

Part 5 — Ensemble Learning

Table of Contents

Part 5 — Ensemble Learning	3
এনসেম্বল লার্নিং কী এবং কেন?	3
Chapter 19 — Gradient Boosting Algorithm	5
১৯.১ নামের মধ্যে লুকানো দুটি শব্দের রহস্য	5
১৯.২ একটা গল্পের মাধ্যমে বোঝা যাক (Intuition)	5
১৯.৩ মূল চারটি স্তম্ভ (Core Structure) — বিস্তারিত ব্যাখ্যা	6
১৯.৪ গাণিতিক ভিত্তি — ব্যাকগ্রাউন্ডে আসলে কী ঘটে, ধাপে ধাপে	8
১৯.৫ সংখ্যাসূচক উদাহরণ — সম্পূর্ণ ধাপে ধাপে হিসাব (Regression)	9
১৯.৬ গুরুত্বপূর্ণ Hyperparameter — বিস্তারিত ও ব্যবহারিক ব্যাখ্যা	11
১৯.৭ গুরুত্বপূর্ণ Attribute (ট্রেনিং শেষে যা দেখা যায়)	12
১৯.৮ কেন এত জনপ্রিয়? (Advantages)	12
১৯.৯ সীমাবদ্ধতা (Disadvantages)	12
১৯.১০ কোন কোন সেক্টরে ব্যবহার হয়?	12
১৯.১১ প্রশ্নোত্তর (দ্রুত ঝালাই)	13
১৯.১৩ ঐতিহাসিক প্রেক্ষাপট, বিবর্তন ও শিল্পক্ষেত্রে গ্রহণযোগ্যতা	13
১৯.১৪ সম্পূর্ণ গাণিতিক বাস্তবায়ন — হাতে-কলমে সম্পূর্ণ ধাপে ধাপে গণনা (Complete Mathematical Implementation)	15
১৯.১৫ পাইথন বাস্তবায়ন — From Scratch (শূন্য থেকে সম্পূর্ণ কোড)	26
১৯.১৬ পাইথন বাস্তবায়ন — Scikit-Learn ব্যবহার করে	29
১৯.১৭ হাইপারপ্যারামিটার টিউনিং উদাহরণ (GridSearchCV)	30
১৯.১৮ Debugging Tips ও সাধারণ ভুল (Common Mistakes)	31
১৯.১৯ পারফরম্যান্স অপটিমাইজেশন ও Best Practices	32
১৯.২০ কম্প্লেক্সিটি অ্যানালিসিস (Complexity Analysis)	32
১৯.২১ ইন্টারভিউ ও পরীক্ষার নোট (Interview & Exam Notes)	32
১৯.১২ Quick Revision (সংক্ষিপ্ত সারাংশ)	33
Chapter 20 — HistGradientBoosting	33
২০.১ ভূমিকা	33

২০.২ উদাহরণ (Intuition) — কেন এটি এত দ্রুত?	33
২০.৩ মূল কার্যপ্রণালী (How it Works) — বিস্তারিত	34
২০.৪ সাধারণ Gradient Boosting বনাম HistGradientBoosting	35
২০.৫ গাণিতিক ভিত্তি — Histogram-ভিত্তিক Split Gain	35
২০.৬ গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা	36
২০.৭ গুরুত্বপূর্ণ অ্যাট্রিবিউট	37
২০.৮ কোথায় ব্যবহার করা উচিত?	37
২০.৯ প্রশ্নোত্তর	37
২০.১১ ঐতিহাসিক প্রেক্ষাপট ও শিল্পক্ষেত্রে গ্রহণযোগ্যতা	37
২০.১২ সম্পূর্ণ গাণিতিক বাস্তবায়ন — হাতে-কলমে সম্পূর্ণ ধাপে ধাপে গণনা	38
২০.১৩ পাইথন বাস্তবায়ন — From Scratch (সরলীকৃত Histogram-ভিত্তিক সংস্করণ)	41
২০.১৪ পাইথন বাস্তবায়ন — Scikit-Learn HistGradientBoosting	43
২০.১৫ Missing Value ও Categorical Feature হ্যান্ডলিং — কোড উদাহরণ	45
২০.১৬ কম্পিউটেশনাল জটিলতা বিশ্লেষণ	45
২০.১৭ ইন্টারভিউ ও পরীক্ষার নোট	46
২০.১০ Quick Revision	46
Chapter 21 — XGBoost	46
২১.১ ভূমিকা ও ইতিহাস	46
২১.২ Gradient Boosting থেকে XGBoost — মূল পার্থক্য কোথায়?	47
২১.৩ গাণিতিক ভিত্তি — Regularized Objective Function	47
২১.৪ Sparsity-Aware Split Finding — Missing Value-এর জন্য স্বয়ংক্রিয় দিকনির্দেশনা	50
২১.৫ Split খোঁজার অ্যালগরিদম — Exact vs Approximate	50
২১.৬ সিস্টেম-স্তরের অপটিমাইজেশন — কেন এটি এত দ্রুত?	50
২১.৭ গুরুত্বপূর্ণ Hyperparameter — বিস্তারিত ব্যাখ্যা	51
২১.৮ গুরুত্বপূর্ণ Attribute ও মেথড	52
২১.৯ XGBoost, GradientBoosting এবং HistGradientBoosting — সম্পূর্ণ তুলনা	53
২১.১০ কেন এত জনপ্রিয়? (Advantages)	53
২১.১১ সীমাবদ্ধতা (Disadvantages)	54
২১.১২ শিল্পক্ষেত্রে ব্যবহার	54
২১.১৩ প্রশ্নোত্তর	54
২১.১৫ সম্পূর্ণ গাণিতিক বাস্তবায়ন — হাতে-কলমে সম্পূর্ণ ধাপে ধাপে গণনা	55

২১.১৬ পাইথন বাস্তবায়ন — From Scratch (Gradient + Hessian সহ)	63
২১.১৭ পাইথন বাস্তবায়ন — XGBoost লাইব্রেরি (Official)	65
২১.১৮ GPU-ত্বরিত প্রশিক্ষণ উদাহরণ	67
২১.১৯ কম্পিউটেশনাল জটিলতা বিশ্লেষণ	68
২১.২০ মেমোরি অপ্টিমাইজেশন ও Best Practices	68
২১.২১ ইন্টারভিউ ও পরীক্ষার নোট	68
২১.১৪ Quick Revision	69
Part 5 — সামগ্রিক তুলনা (Overall Comparison)	69
Part 5 — সম্প্রসারিত তুলনা: Complexity, Memory ও Scalability	70
Part 5 — সুবিধা বনাম অসুবিধা: একটি সম্মিলিত দৃষ্টিভঙ্গি	71
Part 5 — সম্পূর্ণ Ensemble Learning শিক্ষার Pipeline (ASCII সারসংক্ষেপ)	71

Part 5 — Ensemble Learning

এনসেম্বল লার্নিং কী এবং কেন?

ধরুন, আপনি একটা গুরুত্বপূর্ণ সিদ্ধান্ত নিতে যাচ্ছেন — একটা শেয়ারে বিনিয়োগ করবেন কিনা। একজন মাত্র বিশেষজ্ঞের মতামত না নিয়ে যদি একশো জন স্বাধীন বিশেষজ্ঞের মতামত একত্র করে সিদ্ধান্ত নেওয়া হয়, তাহলে সেই সিদ্ধান্ত সাধারণত অনেক বেশি নির্ভরযোগ্য হয়ে ওঠে। এটাই হলো *Ensemble Learning*-এর মূল দর্শন — একাধিক “সিদ্ধান্ত নেওয়ার মডেল” একত্রিত করে এমন একটা সম্মিলিত ফলাফল তৈরি করা, যা একক কোনো মডেলের চেয়ে অনেক বেশি শক্তিশালী, স্থিতিশীল ও নির্ভুল।

পরিসংখ্যানগত দৃষ্টিকোণ থেকে এর পেছনে একটা সহজ কিন্তু গভীর যুক্তি আছে। যদি একাধিক মডেলের ভুলগুলো একে অপরের থেকে স্বাধীন হয় (Uncorrelated), তাহলে তাদের গড় বা ভোট নিলে ভুলগুলো একে অপরকে বাতিল করে দেয়, আর সঠিক প্যাটার্নগুলো জোরদার হতে থাকে। এই নীতিটিই আধুনিক Machine Learning-এর সবচেয়ে প্রভাবশালী কৌশলগুলোর একটির ভিত্তি — Kaggle-এর মতো প্রতিযোগিতামূলক প্ল্যাটফর্ম থেকে শুরু করে শিল্পখাতের প্রোডাকশন সিস্টেম পর্যন্ত, Tabular ডেটার ওপর সবচেয়ে নির্ভরযোগ্য ফলাফল প্রায় সবসময় কোনো না কোনো Ensemble পদ্ধতি থেকেই আসে।

একশো জন বিশেষজ্ঞকে একত্রিত করার জন্য মূলত দুইটি সম্পূর্ণ ভিন্ন দার্শনিক পদ্ধতি রয়েছে, এবং এই দুইটি পদ্ধতি বোঝাই Ensemble Learning-এর সম্পূর্ণ জগৎ বোঝার চাবিকাঠি।

পদ্ধতি ১ — Bagging (Random Forest যেভাবে কাজ করে): একশো জন বিশেষজ্ঞকে একই সময়ে, আলাদা আলাদা ঘরে বসিয়ে দেওয়া হয়। কেউ কারো মতামত জানতে পারেন না। প্রত্যেকে

সম্পূর্ণ স্বাধীনভাবে (Independently ও Parallel-ভাবে) নিজের সিদ্ধান্ত দেন, এবং শেষে সবার মতামতের গড় বা ভোট নিয়ে চূড়ান্ত সিদ্ধান্ত গঠিত হয়।

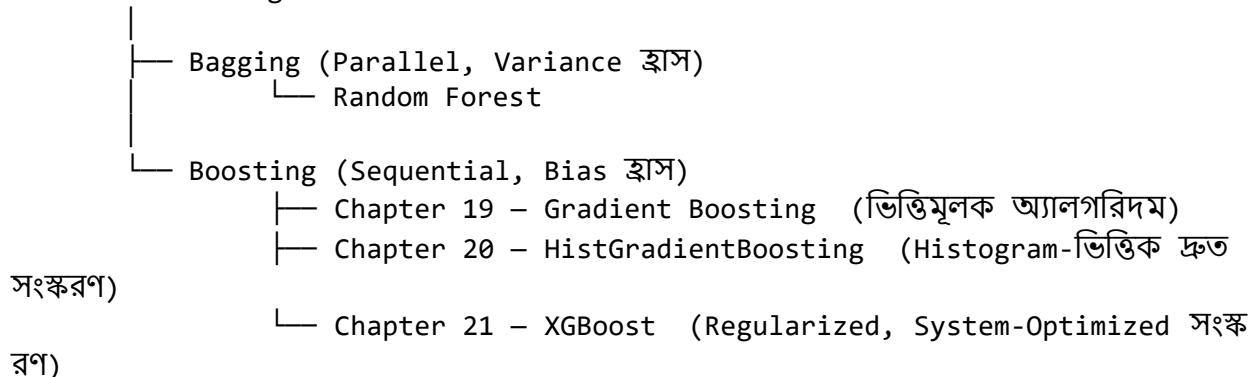
পদ্ধতি ২ — Boosting (Gradient Boosting যেভাবে কাজ করে): এখানে বিশেষজ্ঞরা একসাথে বসেন না, বরং একজনের পর একজন ধারাবাহিকভাবে কাজ করেন। প্রথম বিশেষজ্ঞ একটা মোটামুটি অনুমান করেন। দ্বিতীয় বিশেষজ্ঞ প্রথমজনের অনুমানের ভুলগুলো দেখে বলেন, “আপনি এখানে ভুল করেছেন, আমি শুধু সেই ভুলটুকু সংশোধনের চেষ্টা করছি।” তৃতীয়জন আবার প্রথম দুইজন মিলে যে ভুল রয়ে গেছে, শুধু সেটাই সংশোধনের চেষ্টা করেন। এভাবে ধারাবাহিকভাবে, প্রতিজন আগের সবার সম্মিলিত ভুল সংশোধন করতে থাকেন।

নিচের সারণিতে দুইটি দর্শনের মূল পার্থক্যগুলো একত্রে দেখানো হলো:

বৈশিষ্ট্য**	Bagging (যেমন Random Forest)** **B	Boosting (যেমন Gradient Boosting)**
গাছ তৈরির ধরন	স্বাধীন ও Parallel ন	নির্ভরশীল ও Sequential
মূল লক্ষ্য Va	variance কমানো	Bias কমানো
বেস মডেল	সাধারণত গভীর, শক্তিশালী গাছ ই	স্বাভাবিকভাবে দুর্বল ও অগভীর গাছ
ট্রেনিং গতি ত	সুসঙ্গত (Parallelizable) তুলনাম	সুসঙ্গত (Sequential)
Overfitting-প্রবণতা	তুলনামূলক কম বেশ	হ্যাঁ, সতর্ক Tuning প্রয়োজন
চূড়ান্ত ফলাফল গঠন গ	গড় বা সংখ্যাগরিষ্ঠ ভোট (Average/Vote) ওজ	ন্যূনতম যোগফল (Weighted Sum)

Part 5-এ আমরা Boosting-পরিবারের তিনটি গুরুত্বপূর্ণ ও শিল্পক্ষেত্রে ব্যাপকভাবে ব্যবহৃত অ্যালগরিদম নিয়ে বিস্তারিত আলোচনা করব — প্রথমে ভিত্তিমূলক *Gradient Boosting* অ্যালগরিদম, তারপর Scikit-Learn-এর দ্রুতগতির বাস্তবায়ন *HistGradientBoosting*, এবং সবশেষে শিল্পক্ষেত্রে সবচেয়ে জনপ্রিয় ও প্রতিযোগিতা-বিজয়ী লাইব্রেরি *XGBoost*। তিনটি অ্যালগরিদমই একই মৌলিক দর্শনের ওপর দাঁড়িয়ে আছে, কিন্তু প্রতিটি ধাপে একে অপরের সীমাবদ্ধতা দূর করে আরও দ্রুত, আরও নির্ভুল এবং আরও ব্যবহারিক করে তোলা হয়েছে।

Ensemble Learning



Chapter 19 — Gradient Boosting Algorithm

১৯.১ নামের মধ্যে লুকানো দুটি শব্দের রহস্য

“Boosting” শব্দটার মানে হলো দুর্বল জিনিসকে ধাপে ধাপে “বুস্ট” করে শক্তিশালী করা। এখানে “দুর্বল” বলতে বোঝানো হয়েছে এমন একটা ছোট্ট মডেলকে, যেটা একা একা ভালো প্রেডিকশন দিতে পারে না — কিন্তু অনেকগুলো একসাথে সিরিজে বসালে দারুণ কাজ করে। ঠিক যেমন একজন সদ্য শেখা শিক্ষার্থী একটা বড় পরীক্ষায় ভালো করতে পারে না, কিন্তু একশো জন সদ্য শেখা শিক্ষার্থী পালা করে করে একে অপরের ভুল ঠিক করতে করতে একসাথে খুব ভালো ফলাফল দিতে পারে।

“Gradient” শব্দটা এসেছে গণিতের একটা বিখ্যাত কৌশল *Gradient Descent* থেকে। Gradient Descent-এ আমরা কোনো একটা Loss (ভুলের পরিমাণ) কমানোর জন্য, সেই Loss-এর “ঢাল” (Gradient) বা সবচেয়ে খাড়া নিচের দিকের রাস্তা অনুসরণ করে ধীরে ধীরে এগোই — যেমন পাহাড় থেকে নামার সময় সবসময় সবচেয়ে খাড়া নিচের দিকের রাস্তা খুঁজে খুঁজে নামলে দ্রুততম পথে নিচে পৌঁছানো যায়।

সাধারণ Gradient Descent-এ আমরা মডেলের সংখ্যাসূচক প্যারামিটার (যেমন w, b) পরিবর্তন করি Loss কমানোর জন্য। কিন্তু Gradient Boosting-এ আমরা সংখ্যা পরিবর্তন করি না — বরং প্রতিটি ধাপে একটা সম্পূর্ণ নতুন গাছ (Decision Tree) যোগ করি, যেটা সেই মুহূর্তে Loss কমানোর সবচেয়ে ভালো দিকটা নির্দেশ করে। তাই একে বলা হয় “Function Space-এ Gradient Descent” — কারণ এখানে আমরা সংখ্যার জায়গায় একটা সম্পূর্ণ ফাংশন (গাছ) যোগ করছি।

ঐতিহাসিক প্রেক্ষাপট: Gradient Boosting-এর ধারণাটি প্রথম প্রস্তাব করেন লিও ব্রেইমান (Leo Breiman) ১৯৯৭ সালে, এবং পরবর্তীতে জেরোম ফ্রিডম্যান (Jerome Friedman) ১৯৯৯-২০০১ সালে এটিকে একটি সুসংগত পরিসংখ্যানগত কাঠামোতে (Gradient Boosting Machine বা GBM) দাঁড় করান। ফ্রিডম্যানের মূল অবদান ছিল Boosting-কে ফাংশন স্পেসে Gradient Descent হিসেবে পুনর্গঠন করা, যা যেকোনো Differentiable Loss Function-এর জন্য Boosting প্রয়োগ করা সম্ভব করে তোলে।

১৯.২ একটা গল্পের মাধ্যমে বোঝা যাক (Intuition)

ধরুন আপনি একজন ছাত্রের বার্ষিক পরীক্ষার নম্বর অনুমান করার চেষ্টা করছেন। ছাত্রটির প্রকৃত নম্বর হলো ১০০-এর মধ্যে ৮৮, কিন্তু আপনি এটা আগে থেকে জানেন না — আপনাকে ধাপে ধাপে অনুমান করে এই সংখ্যার কাছে পৌঁছাতে হবে।

ধাপ ১ (প্রথম, সবচেয়ে সহজ মডেল): আপনার হাতে কোনো তথ্য নেই, তাই আপনি পুরো ক্লাসের গড় নম্বর অনুমান করে বললেন — ৬৫। এখন ভুলের পরিমাণ (Error) কত? প্রকৃত (৮৮) - অনুমান (৬৫) = ২৩। মানে আপনি এখনো ২৩ নম্বর কম বলেছেন।

ধাপ ২ (দ্বিতীয় মডেল — শুধু ভুলটুকু শেখে): এখন আরেকটা ছোট মডেল তৈরি হলো, যার একমাত্র কাজ হলো এই “২৩” সংখ্যাটাকেই প্রেডিক্ট করার চেষ্টা করা। লক্ষ্যণীয় বিষয় হলো, এই মডেল কিন্তু ছাত্রের প্রকৃত নম্বর (৮৮) শেখার চেষ্টা করছে না, সে শুধু প্রথম মডেলের ভুলটুকু শেখার চেষ্টা করছে। ধরা যাক এই দ্বিতীয় মডেল অনুমান করলো ১৪। এখন মোট প্রেডিকশন = ৬৫ + ১৪ = ৭৯। আগে ছিল ৬৫, এখন হলো ৭৯ — যা প্রকৃত মান ৮৮-এর অনেক কাছাকাছি চলে এসেছে। কিন্তু এখনও একটু পার্থক্য আছে: ৮৮ - ৭৯ = ৯।

ধাপ ৩ (তৃতীয় মডেল — আবার নতুন ভুলটুকু শেখে): এখন তৃতীয় একটা মডেল এই নতুন “৯” পার্থক্যটুকু প্রেডিক্ট করার চেষ্টা করবে। এভাবে একের পর এক ছোট ছোট মডেল যোগ হতে হতে, চূড়ান্ত প্রেডিকশন ধীরে ধীরে প্রকৃত মান ৮৮-এর একদম কাছাকাছি পৌঁছে যাবে।

সবচেয়ে গুরুত্বপূর্ণ কথাটা মনে রাখা দরকার: প্রতিটি নতুন গাছ কখনোই আসল ডেটা (ছাত্রের নম্বর ৮৮) থেকে সরাসরি শেখে না। সে শুধু শেখে “আগের সব গাছ মিলে ঠিক কতটুকু ভুল করেছে” — একে বলা হয় *Residual*। এই জিনিসটাই Gradient Boosting-কে অন্য সব Algorithm থেকে আলাদা করে তোলে।

প্রকৃত মান = ৮৮

$$\boxed{\begin{array}{l} F_0 = 65 \\ \text{(গড় মান)} \end{array}} + \boxed{\begin{array}{l} h_1 \text{ শেখে } +23 \\ \text{(Residual)} \end{array}} + \boxed{\begin{array}{l} h_2 \text{ শেখে } +9 \\ \text{(নতুন Residual)} \end{array}} + \dots \rightarrow \text{৮৮-এর কাছাকাছি}$$

১৯.৩ মূল চারটি স্তম্ভ (Core Structure) — বিস্তারিত ব্যাখ্যা

১৯.৩.১ Weak Learner (দুর্বল শিক্ষার্থী) — এরা কেন ইচ্ছাকৃতভাবেই “দুর্বল” রাখা হয়?

প্রতিটি ছোট মডেল হিসেবে সাধারণত একটা খুবই ছোট ও অগভীর (Shallow) Decision Tree ব্যবহার করা হয়, যার গভীরতা (max_depth) সাধারণত মাত্র ৩ থেকে ৮ রাখা হয়।

এখানে একটা গুরুত্বপূর্ণ প্রশ্ন আসতে পারে — কেন ইচ্ছাকৃতভাবে দুর্বল মডেল ব্যবহার করা হয়, শক্তিশালী মডেল কেন নয়? এর উত্তর হলো: যদি প্রথম গাছটাই খুব শক্তিশালী ও গভীর হয়, তাহলে সে একাই পুরো ডেটার প্যাটার্ন (এমনকি Noise বা ভুল তথ্যও) মুখস্থ করে ফেলবে, এবং পরের কোনো গাছের শেখার মতো তেমন কোনো Residual অবশিষ্ট থাকবে না। ফলে মডেল খুব দ্রুত Overfit করে ফেলবে — অর্থাৎ Training ডেটায় নিখুঁত, কিন্তু নতুন ডেটায় ভয়াবহ খারাপ ফল দেবে।

তাই ইচ্ছাকৃতভাবে প্রতিটি গাছকে “দুর্বল” রাখা হয়, যাতে একটা গাছ শুধু সামান্য কিছু শেখে, বাকিটুকু পরের গাছের জন্য রেখে দেয়। এভাবে ধীরে ধীরে, নিয়ন্ত্রিতভাবে শেখাটাই মডেলকে স্থিতিশীল ও Generalizable (নতুন ডেটাতেও ভালো কাজ করার উপযোগী) করে তোলে।

পরিসংখ্যানের ভাষায় বলা যায়, প্রতিটি Weak Learner High-Bias কিন্তু Low-Variance মডেল — আর সেগুলো ধারাবাহিকভাবে একত্রিত হয়ে চূড়ান্তভাবে Low-Bias ও Low-Variance উভয়ই অর্জন করে।

১৯.৩.২ Sequential Learning (ধারাবাহিক শিক্ষণ) – কেন গাছগুলো একসাথে তৈরি করা যায় না?

Random Forest-এ প্রতিটা গাছ একে অপরের থেকে সম্পূর্ণ স্বাধীন, তাই কম্পিউটারের একাধিক Core একসাথে ব্যবহার করে সবগুলো গাছ একই সময়ে (Parallel-ভাবে) তৈরি করা সম্ভব, যা দ্রুতগতির।

কিন্তু Gradient Boosting-এ এটা করা একদমই অসম্ভব, কারণ:

- দ্বিতীয় গাছ তৈরি করার আগে অবশ্যই জানতে হবে প্রথম গাছটা ঠিক কী প্রেডিক্ট করেছে এবং তার Residual কত।
- তৃতীয় গাছ তৈরি করার আগে জানতে হবে প্রথম ও দ্বিতীয় গাছ মিলে এখন কী প্রেডিক্ট করেছে।

অর্থাৎ প্রতিটি গাছ তার আগের গাছের ফলাফলের ওপর সম্পূর্ণ নির্ভরশীল — একটা গাছ সম্পূর্ণ শেষ না হওয়া পর্যন্ত পরেরটা শুরুই করা যায় না। এই কারণেই এটা এক ধরনের “একের পর এক দাঁড়ানো লাইন” প্রক্রিয়া, এবং এই কারণেই এটা Random Forest-এর তুলনায় ধীরগতির। এই Sequential নির্ভরতা শুধু গতি নয়, বরং Hyperparameter-এর প্রতি সংবেদনশীলতাও বাড়িয়ে দেয়, কারণ একটা ধাপের ত্রুটি পরবর্তী প্রতিটি ধাপে বহন হতে থাকে।

১৯.৩.৩ Additive Model (সংযোজনমূলক মডেল) – চূড়ান্ত উত্তর কীভাবে তৈরি হয়?

চূড়ান্ত প্রেডিকশন কখনোই কোনো একটা গাছের একক ফলাফল নয় — এটা সবগুলো গাছের প্রেডিকশনের একটা যোগফল।

$$F(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \eta \cdot h_3(x) + \dots + \eta \cdot h_M(x)$$

এটাকে সহজভাবে বুঝতে চাইলে একটা লেয়ার-কেক ভাবা যেতে পারে:

- $F_0(x)$ হলো একদম প্রথম, ভিত্তি স্তর — সাধারণত পুরো ডেটার গড় মান (আমাদের গল্পে যেটা ছিল ৬৫)।
- $h_1, h_2, h_3, \dots$ — এগুলো একের পর এক তৈরি হওয়া গাছ, প্রতিটাই আগের সম্মিলিত ভুলের একটা অংশ ঠিক করেছে।
- η (Eta বা Learning Rate) — একটা ছোট সংখ্যা (যেমন ০.১), যা প্রতিটা গাছের অবদানকে খানিকটা সংযত করে যোগ করে।

η কেন গুরুত্বপূর্ণ, একটা উদাহরণ দিয়ে বোঝা যাক: ধরুন দ্বিতীয় গাছ প্রেডিক্ট করলো Residual = ১৪। যদি $\eta = 1.0$ (সম্পূর্ণ ওজনে) হতো, তাহলে পুরো ১৪-ই একবারে যোগ হতো — অনেকটা “এক লাফে অনেকদূর” যাওয়ার মতো, যা মডেলকে অস্থির করে দিতে পারে। কিন্তু $\eta = 0.1$ হলে মাত্র $১৪ \times ০.১ = ১.৪$ যোগ হবে — অর্থাৎ প্রতিটা ধাপে সামান্য একটু করে, সতর্কভাবে এগোনো হচ্ছে। এটা অনেকটা গাড়ি চালানোর মতো — একবারে ফুল স্পিডে এক্সেলারেটর না চেপে, ধীরে ধীরে গতি বাড়ানোই নিরাপদ ও নিয়ন্ত্রিত।

১৯.৩.৪ Gradient Descent Analogy — এটাই “গ্র্যাডিয়েন্ট” নামের আসল কারণ

সাধারণ Machine Learning মডেলে (যেমন Linear Regression) Gradient Descent ব্যবহার করে মডেলের ওজন বা প্যারামিটার (w, b) ধীরে ধীরে পরিবর্তন করা হয়, যাতে Loss কমতে থাকে। প্রতিবার Loss-এর Negative Gradient (সবচেয়ে খাড়া নিচের দিকের ঢাল) অনুসরণ করা হয় — এটাই সবচেয়ে দ্রুত Loss কমানোর দিক।

Gradient Boosting একই দর্শন অনুসরণ করে, কিন্তু একটা গুরুত্বপূর্ণ পার্থক্য নিয়ে:

- সাধারণ Gradient Descent-এ প্রতি ধাপে একটা সংখ্যা (Parameter) আপডেট হয়।
- Gradient Boosting-এ প্রতি ধাপে একটা সম্পূর্ণ নতুন Decision Tree তৈরি হয়ে যোগ হয়, যেটা আসলে সেই মুহূর্তের Negative Gradient-কেই প্রতিনিধিত্ব করে।

এই কারণেই একে বলা হয় “Function Space-এ Gradient Descent” — কারণ এখানে সংখ্যার জগতে নয়, বরং ফাংশনের (গাছের) জগতে ধাপে ধাপে এগোনো হচ্ছে।

১৯.৪ গাণিতিক ভিত্তি — ব্যাকএন্ডে আসলে কী ঘটে, ধাপে ধাপে

পূর্ববর্তী অংশে আমরা গল্পের মাধ্যমে বুঝেছি Gradient Boosting কীভাবে কাজ করে। এখন সেই একই জিনিস গাণিতিকভাবে, প্রতিটি ধাপ আলাদা করে বোঝা যাক।

ধাপ ১: প্রাথমিক প্রেডিকশন (Initial Prediction) — যাত্রা শুরুর বিন্দু

মডেল শুরু হয় একটা অত্যন্ত সাধারণ, একক Constant মান দিয়ে — এমন একটা মান যা পুরো Training ডেটাসেটের ওপর Loss Function-কে সর্বনিম্ন করে।

$$F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum_i L(y_i, \gamma)$$

এই সূত্রটা আসলে বলছে: এমন একটা γ (গামা) সংখ্যা খুঁজে বের করো, যেটা বসালে সব ডেটাপয়েন্টের মিলিত Loss সবচেয়ে কম হয়।

- Regression সমস্যা (Loss = MSE হলে): গাণিতিকভাবে প্রমাণিত যে, MSE-কে সর্বনিম্ন করা γ -এর মান হলো টার্গেটের গড় (Mean)। এই কারণেই ওপরের উদাহরণে ৬৫ (গড় নম্বর) দিয়ে শুরু করা হয়েছিল — এটা কোনো এলোমেলো অনুমান ছিল না, বরং গাণিতিকভাবে সবচেয়ে ভালো “প্রথম অনুমান”।
- Classification সমস্যা: এখানে Mean কাজ করে না (কারণ Class লেবেল সংখ্যা নয়), তাই এখানে ব্যবহার করা হয় Log-Odds-এর একটা প্রাথমিক মান।

ধাপ ২: Pseudo-Residual গণনা করা — “কতটুকু ভুল হলো” বের করা

প্রতিটি Iteration-এর শুরুতে, বর্তমান মডেলের প্রেডিকশন এবং প্রকৃত মানের মধ্যে একটা Pseudo-Residual বের করা হয়। এটা আসলে Loss Function-এর Negative Gradient:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]$$

এই সূত্রটা দেখে ভয় পাওয়ার কিছু নেই — এর ভেতরের অর্থটা খুব সহজ। এটা মূলত বলছে: “Loss যদি একটু পরিবর্তন হয়, তাহলে সেটা কোন দিকে সবচেয়ে বেশি কমবে, সেই দিকটা বের করো।”

সবচেয়ে সুন্দর ব্যাপার হলো: Regression-এ যদি Loss = MSE (Mean Squared Error) ব্যবহার করা হয়, তাহলে এই জটিল ডেরিভেটিভ সূত্রটা সরলীকৃত হয়ে দাঁড়ায় শুধুমাত্র:

$$\text{Residual} = \text{Actual}(y) - \text{Predicted}(F(x))$$

অর্থাৎ প্রকৃত মান বিয়োগ অনুমিত মান — এত সহজ! এই কারণেই ওপরের উদাহরণে সরাসরি (৮৮ – ৬৫ = ২৩) বিয়োগ করেই Residual বের করা হয়েছিল — এটাই আসলে Gradient গণনার সরলীকৃত রূপ।

ধাপ ৩: নতুন Weak Learner গাছ ট্রেইন করা — ভুলের প্যাটার্ন শেখা

এরপর একটা নতুন, ছোট Decision Tree (h_m) ট্রেইন করা হয়। এই গাছের ইনপুট হলো একই Feature-গুলো (যেমন Study Hours), কিন্তু টার্গেট হলো আসল y নয় — বরং ধাপ ২-এ পাওয়া Residual-গুলো।

এখানে একটা জিনিস স্পষ্ট করে বোঝা দরকার: এই গাছ কখনো শেখে না “কত ঘণ্টা পড়লে কত নম্বর পাওয়া যায়”। সে শেখে “কত ঘণ্টা পড়লে আগের মডেল কতটুকু ভুল করেছে” — অর্থাৎ সে ভুলের প্যাটার্ন শেখে, আসল উত্তরের প্যাটার্ন নয়।

ধাপ ৪: মডেল আপডেট করা — Learning Rate দিয়ে সংযত করে যোগ

নতুন তৈরি হওয়া গাছটাকে সরাসরি পুরো ওজনে যোগ না করে, Learning Rate (η) দিয়ে গুণ করে যোগ করা হয়:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

এই “সংযত করে যোগ করা”টাই Gradient Boosting-এর সবচেয়ে গুরুত্বপূর্ণ সুরক্ষা কৌশল — এর ফলে মডেল একবারে বিশাল লাফ না দিয়ে ধীরে ধীরে, নিয়ন্ত্রিতভাবে উন্নত হয়, যা Overfitting-এর ঝুঁকি অনেকটাই কমিয়ে দেয়।

ধাপ ৫: পুনরাবৃত্তি (Iteration) — একই প্রক্রিয়া বহুবার চলতে থাকে

ধাপ ২ থেকে ৪ পর্যন্ত পুরো প্রক্রিয়াটা $n_{\text{estimators}}$ -এ নির্ধারিত সংখ্যক বার (যেমন ১০০ বা ৫০০ বার) পুনরাবৃত্তি হয়। প্রতিটা নতুন Iteration-এ Residual আগের চেয়ে ছোট হতে থাকে — যেন প্রতিটা নতুন বিশেষজ্ঞ আগের চেয়ে ছোট একটা ফাঁকফোকর পূরণ করছেন, আর মডেল ধীরে ধীরে প্রকৃত উত্তরের একদম কাছে পৌঁছে যাচ্ছে।

১৯.৫ সংখ্যাসূচক উদাহরণ — সম্পূর্ণ ধাপে ধাপে হিসাব (Regression)

ধরা যাক আমাদের কাছে ৪ জন ছাত্রের ডেটা আছে:

ID	Study Hours	প্রকৃত Marks (y)
1	1	40
2	3	55
3	5	85
4	7	100

ধাপ ১ — F_0 বের করা: $\text{Mean} = (40+55+85+100)/4 = 280/4 = 65$ । তাই প্রতিটি ছাত্রের জন্য প্রাথমিক অনুমান = ৬৫।

ID	Study Hours	প্রকৃত Marks	F_0	Residual (y - F_0)
1	1	40	65	-25
2	3	55	65	-10
3	5	85	65	+20
4	7	100	65	+35

ধাপ ২ — প্রথম গাছ h_1 তৈরি: এই গাছ Study Hours দেখে Residual (-25, -10, +20, +35) প্রেডিক্ট করার চেষ্টা করবে। ধরা যাক গাছটি একটা শর্ত দিয়ে ভাগ করলো: Study Hours ≤ 4 কিনা?

- বাম দিকে (Hours ≤ 4 , ID 1 ও 2): Residual-এর গড় = $(-25 + -10)/2 = -17.5$
- ডান দিকে (Hours > 4 , ID 3 ও 4): Residual-এর গড় = $(20 + 35)/2 = +27.5$

তাই $h_1(x)$ হবে -17.5 অথবা +27.5, Study Hours-এর মান অনুযায়ী।

ধাপ ৩ — মডেল আপডেট ($\eta = 0.1$ ধরে):

$$F_1(x) = F_0(x) + 0.1 \times h_1(x)$$

- ID 1: $F_1 = 65 + 0.1 \times (-17.5) = 65 - 1.75 = 63.25$ (প্রকৃত মান ৪০-এর দিকে একটু এগোলো)
- ID 4: $F_1 = 65 + 0.1 \times 27.5 = 65 + 2.75 = 67.75$ (প্রকৃত মান ১০০-এর দিকে একটু এগোলো)

লক্ষণীয়, খুব সামান্য একটু এগোলো, একেবারে লাফ দিয়ে ১০০-তে পৌঁছায়নি। এটাই Learning Rate-এর “সংযত” রাখার কাজ।

ধাপ ৪ — নতুন Residual ও চক্র চলতে থাকা: এখন নতুন Residual = $(y - F_1)$ বের হবে, যা আগের চেয়ে ছোট। দ্বিতীয় গাছ h_2 এই নতুন, ছোট Residual শেখার চেষ্টা করবে। এভাবে ধরা যাক ১০০ বার এই চক্র চললে, প্রতিবার Residual ক্রমশ ছোট হতে হতে শেষে মডেলের প্রেডিকশন প্রকৃত মান (৪০, ৫৫, ৮৫, ১০০)-এর একদম কাছাকাছি পৌঁছে যাবে।

এই ধীরে ধীরে এগিয়ে যাওয়াটাই Gradient Boosting-এর মূল শক্তি — এবং একইসাথে এটাই এর ধীরগতির কারণ।

১৯.৬ গুরুত্বপূর্ণ Hyperparameter — বিস্তারিত ও ব্যবহারিক ব্যাখ্যা

n_estimators — Default: 100

কাজ: মোট কতগুলো গাছ তৈরি হবে তা নির্ধারণ করে। Default কখন রাখব: ছোট বা মাঝারি ডেটাসেটে ১০০ ভালো শুরুর বিন্দু। ঝুঁকি: বেশি রাখলে (যেমন ১০,০০০) Training ডেটা প্রায় মুখস্থ হয়ে যায় (Overfitting) এবং সময়ও অনেক বেড়ে যায়। কখন পরিবর্তন করব: learning_rate ছোট রাখলে (যেমন ০.০১) n_estimators বাড়াতে হয়, কারণ প্রতিটা গাছ খুবই সামান্য অবদান রাখছে। বাস্তবে early_stopping বা Validation Curve দিয়ে সর্বোত্তম মান বের করাই সবচেয়ে নিরাপদ পদ্ধতি।

learning_rate — Default: 0.1

কাজ: প্রতিটা নতুন গাছের অবদানকে কতটা সংকুচিত করে যোগ করা হবে। Default কখন রাখব: ০.১ বেশিরভাগ সাধারণ সমস্যায় ভারসাম্যপূর্ণ ফলাফল দেয়। ঝুঁকি: বড় রাখলে (যেমন ১.০) মডেল Optimal সমাধান পার হয়ে যেতে পারে (Overshoot) এবং ফলাফল অস্থির হয়। খুব ছোট রাখলে (যেমন ০.০০১) শিখতে অনেক বেশি সময় ও গাছ লাগবে। কখন পরিবর্তন করব: বাস্তবে সাধারণত learning_rate ছোট (০.০১-০.১) রেখে n_estimators বাড়ানো হয় — এই কন্সট্রিকশন সবচেয়ে স্থিতিশীল ফল দেয়, তবে ট্রেনিং সময় বেশি লাগে। এই দুই প্যারামিটারের মধ্যকার Trade-off-কে Shrinkage-Iteration Trade-off বলা হয়।

max_depth — Default: 3

কাজ: প্রতিটা Weak Learner গাছের সর্বোচ্চ গভীরতা। Default কখন রাখব: যেহেতু দর্শনটাই “দুর্বল” মডেল একত্রিত করা, তাই ৩-৮ রেঞ্জের রাখা হয়। ঝুঁকি: বড় রাখলে (যেমন ২০) প্রতিটা গাছ নিজেই শক্তিশালী হয়ে Noise শিখে ফেলবে, Overfitting বাড়বে। কখন পরিবর্তন করব: Feature-দের মধ্যে জটিল Interaction থাকলে সামান্য গভীর (৫-৮) গাছ ব্যবহার করা যায়, তবে সাধারণত ৩-৫ রেঞ্জের সবচেয়ে ভালো Generalization পাওয়া যায়।

subsample — Default: 1.0

কাজ: প্রতিটা গাছ ট্রেন করার সময় Training ডেটার কত অংশ Randomly ব্যবহার হবে। Default কখন রাখব: ১.০ মানে সম্পূর্ণ ডেটাসেট, ছোট ডেটাসেটে যথেষ্ট। কখন পরিবর্তন করব: ১.০-এর কম (যেমন ০.৮) সেট করলে প্রতিটা গাছ ভিন্ন Random অংশ দেখে, যা Randomness যোগ করে Overfitting কমায় — একে বলা হয় Stochastic Gradient Boosting।

Parameter	Default	সংক্ষিপ্ত কাজ
loss	'log_loss' / 'squared_error'	কোন Loss Function দিয়ে Gradient হিসাব হবে
min_samples_split	2	Node ভাঙার জন্য নূন্যতম Sample সংখ্যা — বাড়ালে Overfitting কমে
min_samples_leaf	1	প্রতিটা Leaf-এ নূন্যতম Sample সংখ্যা — বাড়ালে মডেল স্মুথ হয়

Parameter	Default	সংক্ষিপ্ত কাজ
max_features	None	প্রতিটা Split-এ সর্বোচ্চ কতগুলো Feature বিবেচনা হবে

১৯.৭ গুরুত্বপূর্ণ Attribute (ট্রেনিং শেষে যা দেখা যায়)

- **feature_importances_** — প্রতিটা Feature সামগ্রিকভাবে কতটুকু Loss কমাতে সাহায্য করেছে, যোগফল সবসময় ১.০।
- **train_score_** — প্রতিটা Stage-এ Training Loss কেমন ছিল, এটা প্লট করলে Learning Curve দেখা যায় এবং বোঝা যায় কোথায় Overfitting শুরু হয়েছে।
- **estimators_** — ট্রেনিং হওয়া প্রতিটা গাছের অ্যারে, আলাদা করে পরীক্ষা করা যায়।
- **n_estimators_** — বাস্তবে কতগুলো গাছ ব্যবহৃত হয়েছে (Early Stopping থাকলে নির্ধারিত সংখ্যার চেয়ে কম হতে পারে)।

১৯.৮ কেন এত জনপ্রিয়? (Advantages)

- **উচ্চ নির্ভুলতা:** প্রতিটা গাছ আগের ভুল সংশোধনে মনোযোগী, তাই একক Decision Tree বা Random Forest-এর চেয়েও নিখুঁত ফল দেয়।
- **নমনীয় Loss Function:** MSE (Regression), Log-Loss (Classification), Huber Loss (Outlier-Robust)-এর মতো বিভিন্ন Loss ব্যবহার করে কাস্টমাইজ করা যায়।
- **Feature Importance:** feature_importances_ থেকে সহজে বের করা যায়, Feature Selection-এ সাহায্য করে।
- **প্রতিযোগিতায় প্রমাণিত:** Kaggle-এ XGBoost, LightGBM, CatBoost-এর মতো Gradient Boosting লাইব্রেরি প্রায়ই Deep Learning-কেও পেছনে ফেলে দেয়।

১৯.৯ সীমাবদ্ধতা (Disadvantages)

- **ধীর ট্রেনিং:** গাছগুলো Sequential হওয়ায় Parallelize করা কঠিন, তাই Random Forest-এর চেয়ে বেশি সময় লাগে।
- **Overfitting-প্রবণ:** n_estimators বেশি ও learning_rate যথাযথ না হলে মডেল Noise মুখস্থ করে ফেলতে পারে।
- **Hyperparameter-সংবেদনশীল:** learning_rate, n_estimators, max_depth-এর সঠিক সমন্বয় দরকার।
- **Outlier-সংবেদনশীল:** প্রতিটা গাছ Residual শেখে, তাই চরম Outlier থাকলে মডেল ভুল দিকে ঝুঁকে যেতে পারে।

১৯.১০ কোন কোন সেক্টরে ব্যবহার হয়?

- **ফাইন্যান্স ও ব্যাংকিং:** ক্রেডিট স্কোরিং, Fraud Detection, Algorithmic Trading Signal তৈরি।
- **ডিজিটাল মার্কেটিং:** Customer Lifetime Value অনুমান, CTR প্রেডিকশন (Google, Facebook-এর মতো প্ল্যাটফর্মে)।
- **স্বাস্থ্যসেবা:** জটিল বায়োমেডিকেল ডেটা থেকে রোগের ঝুঁকি স্কোরিং।

- **প্রতিযোগিতামূলক ডেটা সায়েন্স:** Kaggle-এ Structured/Tabular ডেটার জন্য সবচেয়ে শক্তিশালী ও জনপ্রিয় পছন্দ।

১৯.১১ প্রশ্নোত্তর (দ্রুত ঝালাই)

প্রশ্ন: Random Forest এবং Gradient Boosting-এর মূল পার্থক্য কী? উত্তর: Random Forest স্বাধীন গাছ Parallel-ভাবে তৈরি করে ভোট বা গড় নেয় (Bagging)। Gradient Boosting গাছগুলো নির্ভরশীলভাবে Sequentially তৈরি করে, প্রতিটা নতুন গাছ আগের ভুল সংশোধন করে। ফলে Gradient Boosting বেশি Accurate কিন্তু ধীরগতির ও Overfitting-প্রবণ, Random Forest তুলনামূলক দ্রুত ও স্থিতিশীল।

প্রশ্ন: learning_rate ছোট রাখলে কেন n_estimators বাড়াতে হয়? উত্তর: learning_rate ছোট মানে প্রতিটা গাছের অবদান সামান্য। তাই প্রকৃত মানের কাছে পৌঁছাতে অনেক বেশি ধাপ বা গাছ দরকার হয় — এটাই তাদের মধ্যকার Trade-off, যা ট্রেনিং সময় বাড়ালেও ফলাফল বেশি স্থিতিশীল করে।

প্রশ্ন: Gradient Boosting কি সবসময় Random Forest-এর চেয়ে ভালো ফল দেয়? উত্তর: না। Gradient Boosting সাধারণত বেশি Accurate হতে পারে যদি যত্নসহকারে Tune করা হয়, কিন্তু ভুল Hyperparameter-এ এটি সহজেই Overfit করে এবং Random Forest-এর চেয়ে খারাপ Generalization দিতে পারে। যেখানে দ্রুত, স্থিতিশীল Baseline দরকার, সেখানে Random Forest দিয়ে শুরু করা এবং পরে প্রয়োজনে Gradient Boosting-এ যাওয়া একটি সাধারণ ব্যবহারিক কৌশল।

১৯.১৩ ঐতিহাসিক প্রেক্ষাপট, বিবর্তন ও শিল্পখাতে গ্রহণযোগ্যতা

১৯.১৩.১ কীভাবে Boosting-এর ধারণা এলো

Boosting-এর তাত্ত্বিক ভিত্তি স্থাপিত হয় ১৯৯০ সালে, যখন Computational Learning Theory-র গবেষক Robert Schapire প্রমাণ করেন যে একাধিক “সামান্য ভালো” (Weak) মডেলকে সঠিকভাবে একত্র করলে একটি “প্রায় নিখুঁত” (Strong) মডেল তৈরি করা সম্ভব — এই ফলাফলটি পরিচিত Strength of Weak Learnability উপপাদ্য নামে। ১৯৯৫ সালে Yoav Freund ও Robert Schapire একত্রে AdaBoost (Adaptive Boosting) আবিষ্কার করেন, যা প্রথম ব্যবহারিকভাবে সফল Boosting অ্যালগরিদম হিসেবে বিবেচিত হয়। AdaBoost প্রতিটি ভুল-শ্রেণীবদ্ধ (Misclassified) Sample-এর ওজন বাড়িয়ে পরবর্তী Weak Learner-কে সেই Sample-গুলোর ওপর বেশি মনোযোগ দিতে বাধ্য করত।

১৯৯৭ সালে Leo Breiman লক্ষ্য করেন যে AdaBoost আসলে একটি বিশেষ ধরনের Numerical Optimization প্রক্রিয়া হিসেবে ব্যাখ্যা করা যায়। এই পর্যবেক্ষণকে ভিত্তি করে ১৯৯৯-২০০১ সালে Jerome H. Friedman “Greedy Function Approximation: A Gradient Boosting Machine” শিরোনামের যুগান্তকারী গবেষণাপত্র প্রকাশ করেন, যেখানে তিনি Boosting-কে সম্পূর্ণভাবে Function Space-এ Gradient Descent হিসেবে পুনর্গঠন করেন। এই পুনর্গঠনের সবচেয়ে বড় সুবিধা হলো — এটি কেবল Classification-এর Exponential Loss-এই সীমাবদ্ধ থাকে না, বরং

যেকোনো Differentiable Loss Function (MSE, MAE, Huber, Log-Loss, Quantile Loss ইত্যাদি) ব্যবহার করে Regression, Classification, এমনকি Ranking সমস্যাতেও প্রয়োগ করা যায়।

১৯.১৩.২ AdaBoost বনাম Gradient Boosting

বৈশিষ্ট্য **	AdaBoost (১৯৯৫)** **G	Gradient Boosting (১৯৯৯-২০০১)**
ভুল সংশোধনের কৌশল	Sample-এর ওজন বাড়ানো	Residual/Gradient শেখা
Loss Function	মূলত Exponential Loss-এ সীমাবদ্ধ	যেকোনো Differentiable Loss
সমস্যার ধরন	মূলত Classification R	Regression ও Classification উভয়ই
গাণিতিক ব্যাখ্যা	মূলত Heuristic হিসেবে শুরু, পরে Optimization হিসেবে ব্যাখ্যা করা হয়	শুরু থেকেই Optimization-ভিত্তিক
আধুনিক ব্যবহার	মূলত মূলক কম, বিশেষ ক্ষেত্রে ব্যবহৃত XGB	XGBoost/LightGBM/CatBoost-এর ভিত্তি

১৯.১৩.৩ শিল্পক্ষেত্রে বাস্তব প্রয়োগের উদাহরণ

- **Yahoo! ও Search Ranking:** ২০০০-এর দশকের শেষদিকে Yahoo!-এর Search Ranking সিস্টেমে Gradient Boosted Trees (বিশেষত LambdaMART, যা Gradient Boosting-এর একটি Ranking-উপযোগী রূপ) ব্যবহার করে Search Result-এর গুণমান উল্লেখযোগ্যভাবে উন্নত করা হয়।
- **Insurance Underwriting:** বীমা প্রতিষ্ঠানগুলো ঝুঁকি নির্ধারণ ও প্রিমিয়াম নির্ধারণে কয়েক দশক ধরে Actuarial পদ্ধতির পাশাপাশি Gradient Boosting ব্যবহার করছে, কারণ এটি জটিল, Non-linear ঝুঁকির প্যাটার্ন শনাক্ত করতে সক্ষম।
- **Netflix Prize (২০০৬-২০০৯):** যদিও বিজয়ী দল মূলত Matrix Factorization ব্যবহার করেছিল, Gradient Boosted Trees ছিল তাদের Ensemble-এর একটি গুরুত্বপূর্ণ উপাদান, যা চূড়ান্ত Recommendation Accuracy বাড়াতে সাহায্য করেছিল।

১৯.১৩.৪ প্রচলিত ভুল ধারণা (Common Misconceptions)

- **ভুল ধারণা ১: “Gradient Boosting সবসময় Random Forest-এর চেয়ে ভালো।”** বাস্তবে এটি নির্ভর করে ডেটার প্রকৃতি ও Tuning-এর মানের ওপর। ভুলভাবে Tune করা Gradient Boosting মডেল সহজেই একটি ভালোভাবে Tune করা Random Forest-এর চেয়ে খারাপ ফল দিতে পারে।
- **ভুল ধারণা ২: “প্রতিটি গাছ ছাত্রের প্রকৃত নম্বর শেখে।”** বাস্তবে, Chapter 19.2-এ ব্যাখ্যা করা হয়েছে, প্রতিটি নতুন গাছ কেবল পূর্ববর্তী মডেলের Residual শেখে, প্রকৃত Target Value নয়।
- **ভুল ধারণা ৩: “Learning Rate যত ছোট, ততই ভালো।”** অত্যন্ত ছোট Learning Rate (যেমন ০.০০১) আসলে ট্রেনিং সময়কে ব্যাপকভাবে বাড়িয়ে দেয় এবং প্রয়োজনীয় n_estimators-এর সংখ্যাও অত্যধিক বেড়ে যায়, যা কম্পিউটেশনাল ব্যয়বহুল হয়ে ওঠে।

১৯.১৪ সম্পূর্ণ গাণিতিক বাস্তবায়ন – হাতে-কলমে সম্পূর্ণ ধাপে ধাপে গণনা (Complete Mathematical Implementation)

এই অংশটি এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ ও সবচেয়ে বিস্তারিত অংশ। এখানে Gradient Boosting Algorithm-এর প্রতিটি গাণিতিক পদক্ষেপ — একেবারে শুরু থেকে শেষ পর্যন্ত — হাতে-কলমে, কোনো ধাপ না বাদ দিয়ে, ক্যালকুলেটর দিয়ে যাচাইযোগ্যভাবে দেখানো হলো। লক্ষ্য হলো, একজন পাঠক যেন কোনো বাহ্যিক উৎসের সাহায্য ছাড়াই সম্পূর্ণ অ্যালগরিদমটি নিজে হাতে চালিয়ে দেখতে পারেন।

১৯.১৪.১ ভূমিকা (Introduction)

আমরা ইতিমধ্যে ১৯.৫ ধারায় একটি সরলীকৃত, গল্পভিত্তিক সংখ্যাসূচক উদাহরণ দেখেছি, যেখানে গড় মান আনুমানিকভাবে ৬৫ ধরে নেওয়া হয়েছিল বোঝার সুবিধার্থে। এই ধারায় আমরা সম্পূর্ণ, নিখুঁতভাবে ক্যালকুলেটর-ভেরিফাইড একটি বাস্তবায়ন করব — একই ডেটাসেট ব্যবহার করে, কিন্তু প্রতিটি সংখ্যা সঠিকভাবে গণনা করে, এবং একাধিক (৫টি) সম্পূর্ণ Iteration ধরে অ্যালগরিদমের আচরণ পর্যবেক্ষণ করে। এই পুরো বিভাগটি একটি সম্পূর্ণ ম্যানুয়াল প্রশিক্ষণ প্রক্রিয়া প্রদর্শন করে, যেখানে কোনো ধাপ “একইভাবে চলতে থাকবে” বলে এড়িয়ে যাওয়া হয়নি — প্রতিটি Iteration-এর প্রতিটি সংখ্যা সরাসরি গণনা করে দেখানো হয়েছে।

১৯.১৪.২ ছোট ডেটাসেট (Small Dataset)

আমরা সেই একই চার ছাত্রের ডেটাসেট ব্যবহার করব, যা একটি Beginner-friendly, হাতে গণনাযোগ্য আকারের ডেটাসেট:

ID	Study Hours (X) *	*প্রকৃত Marks (y)**
1	1	40
2	3	55
3	5	85
4	7	100

১৯.১৪.৩ সমস্যা বিবৃতি (Problem Statement)

লক্ষ্য হলো একটি Regression মডেল তৈরি করা, যা Study Hours (X) থেকে Marks (y) অনুমান করতে পারে, Loss Function হিসেবে Mean Squared Error (MSE) ব্যবহার করে:

$$L(y, F(x)) = (y - F(x))^2$$

আমরা Gradient Boosting ব্যবহার করব, যেখানে Weak Learner হিসেবে গভীরতা-১ (Depth-1, একটি মাত্র Split বিশিষ্ট, একে বলা হয় Decision Stump) বিশিষ্ট Decision Tree ব্যবহার করা হবে, এবং Learning Rate $\eta = 0.1$ নির্ধারণ করা হলো।

১৯.১৪.৪ ইনিশিয়ালাইজেশন (Initialization) – প্রতিটি প্যারামিটার ও হাইপারপ্যারামিটার

প্যারামিটার/হাইপারপ্যারামিটার **প	প্রাথমিক মান**
n_estimators (মোট Iteration)	5 (এই ডেমোনস্ট্রেশনে)
learning_rate (η)	0.1
max_depth (প্রতিটি গাছের গভীরতা) 1	(Decision Stump)
loss function	Squared Error, $L = (y - F)^2$
Weak Learner candidates (Split Threshold)	{2,4,6} (X-এর ধারাবাহিক মানের মধ্যবিন্দু)

১৯.১৪.৫ প্রাথমিক প্রেডিকশন (Initial Prediction)

$F_0(x)$ নির্ণয় করতে হবে এমনভাবে যা সব ডেটাপয়েন্টের মিলিত Squared Error সর্বনিম্ন করে:

$$F_0(x) = \operatorname{argmin}_{\gamma} \sum_{i=1}^4 (y_i - \gamma)^2$$

ক্যালকুলাসের মাধ্যমে প্রমাণিত, এই Minimization-এর সমাধান হলো Mean। ডেরিভেশনটি নিজে হাতে দেখানো যাক — উপরের সমীকরণকে γ -এর সাপেক্ষে Differentiate করে শূন্যের সমান বসাই:

$$\begin{aligned} \frac{d}{d\gamma} \sum_i (y_i - \gamma)^2 &= \sum_i 2 (y_i - \gamma)(-1) = -2 \sum_i (y_i - \gamma) = 0 \\ \Rightarrow \sum_i y_i &= 4\gamma \Rightarrow \gamma = \frac{\sum_i y_i}{4} \end{aligned}$$

এখন প্রকৃত মানগুলো বসাই:

$$F_0 = \frac{40 + 55 + 85 + 100}{4} = \frac{280}{4} = 70$$

তাই $F_0(x) = 70$ — প্রতিটি ছাত্রের জন্য প্রাথমিক প্রেডিকশন ৭০ (এই সঠিক, ক্যালকুলেটর-ভেরিফাইড মান ১৯.৫ ধারার সরলীকৃত ৬৫ থেকে সামান্য ভিন্ন, কারণ এখানে দশমিক পর্যন্ত নিখুঁতভাবে গণনা করা হয়েছে)।

১৯.১৪.৬ Forward Pass – বর্তমান মডেলের প্রেডিকশন

প্রতিটি ছাত্রের জন্য $F_0(x_i) = 70$:

ID	X	y	$F_0(x)$
1	1	40	70
2	3	55	70
3	5	85	70
4	7	100	70

১৯.১৪.৭ Loss Function গণনা (Iteration ০-এ)

$$L_0 = \frac{1}{4} \sum_{i=1}^4 (y_i - F_0)^2$$

প্রতিটি টার্ম আলাদাভাবে গণনা করি:

$$(40 - 70)^2 = (-30)^2 = 900$$

$$(55 - 70)^2 = (-15)^2 = 225$$

$$(85 - 70)^2 = (15)^2 = 225$$

$$(100 - 70)^2 = (30)^2 = 900$$

$$L_0 = \frac{900 + 225 + 225 + 900}{4} = \frac{2250}{4} = 562.5$$

এই ৫৬২.৫ হলো ট্রেনিং শুরুর আগের Baseline MSE — Gradient Boosting-এর লক্ষ্য হলো প্রতিটি Iteration-এ এই সংখ্যাটিকে ধারাবাহিকভাবে কমিয়ে আনা।

১৯.১৪.৮ Residual/Error গণনা (ধাপ ১-এর Residual)

$$r_i^{(1)} = y_i - F_0(x_i)$$

$$r_1 = 40 - 70 = -30, \quad r_2 = 55 - 70 = -15, \quad r_3 = 85 - 70 = 15, \quad r_4 = 100 - 70 = 30$$

১৯.১৪.৯ Gradient গণনা

Squared Error Loss $L = (y - F)^2$ -এর Negative Gradient হলো:

$$-\frac{\partial L}{\partial F} = -\frac{\partial}{\partial F} (y - F)^2 = -[2(y - F)(-1)] = 2(y - F)$$

লক্ষণীয়, এই Negative Gradient হলো Residual-এর ঠিক দ্বিগুণ ($2(y - F) = 2r$)। প্রচলিতভাবে Gradient Boosting Implementation-এ একটি $\frac{1}{2}$ Factor Loss Function-এর সাথে যোগ করে রাখা হয় ($L = \frac{1}{2}(y - F)^2$), যাতে Negative Gradient ঠিক Residual-এর সমান হয়ে যায় — এটাই ১৯.৮ ধারায় উল্লেখিত সরলীকরণের কারণ। আমরা এই প্রচলিত সংজ্ঞা অনুসরণ করে Pseudo-Residual = Residual ধরেই এগোবো:

$$r_i^{(1)} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_0} = y_i - F_0(x_i)$$

১৯.১৪.১০ ডেরিভেটিভ গণনার বিস্তারিত ব্যাখ্যা

চেইন রুল প্রয়োগ করে ধাপে ধাপে দেখা যাক $L = (y - F)^2$ -এর ডেরিভেটিভ কীভাবে বের হয়:

ধরি $u = y - F$, তাহলে $L = u^2$ । চেইন রুল অনুযায়ী:

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial u} \cdot \frac{\partial u}{\partial F} = 2u \cdot (-1) = -2(y - F)$$

তাই Negative Gradient = $-\frac{\partial L}{\partial F} = 2(y - F)$ । ($\frac{1}{2}$ -Factor কনভেনশন ব্যবহার করলে এটি সরলীকৃত হয়ে দাঁড়ায় $(y - F)$, যা আমরা ব্যবহার করব।)

১৯.১৪.১১ কেন এই Gradient কাজ করে (Why This Gradient Works)

Negative Gradient সবসময় সেই দিক নির্দেশ করে, যদিকে অগ্রসর হলে Loss সবচেয়ে দ্রুত কমে। এখানে Residual ধনাত্মক (যেমন ID 3, 4-এর জন্য +15, +30) মানে মডেলের প্রেডিকশন প্রকৃত মানের চেয়ে কম — তাই পরবর্তী গাছেকে বলা হচ্ছে “এই দিকগুলোতে প্রেডিকশন বাড়াও”। বিপরীতভাবে Residual ঋণাত্মক (ID 1, 2-এর জন্য -30, -15) মানে প্রেডিকশন বেশি হয়ে গেছে, তাই পরবর্তী গাছেকে বলা হচ্ছে “এই দিকগুলোতে প্রেডিকশন কমাও”। এভাবে প্রতিটি নতুন গাছ সবসময় সঠিক দিকে (Loss কমানোর দিকে) মডেলকে ঠেলে দেয় — এটাই “Gradient” নামের প্রকৃত তাৎপর্য।

১৯.১৪.১২ Tree 1 তৈরি করা — সেরা Split নির্বাচন

Weak Learner-এর ইনপুট হলো X, টার্গেট হলো $r^{(1)} = [-30, -15, 15, 30]$ । যেহেতু max_depth = 1, তাই একটিমাত্র Split করা হবে। X-এর সাজানো মান (1, 3, 5, 7) থেকে সম্ভাব্য Threshold হলো তাদের মধ্যবিন্দু: 2, 4, এবং 6। প্রতিটি Threshold-এর জন্য SSE (Sum of Squared Error, ভাগ করার পর প্রতিটি দলে Residual-এর গড় থেকে বিচ্যুতির বর্গের যোগফল) হাতে-কলমে গণনা করা যাক:

Threshold = 2 (Split: $X \leq 2$ বনাম $X > 2$):

বাম দল: {ID 1}, Residual = {-30}। বাম দলের গড় = -30। $SSE_{left} = (-30 - (-30))^2 = 0$ । ডান দল: {ID 2,3,4}, Residual = {-15, 15, 30}। ডান দলের গড় = $(-15 + 15 + 30)/3 = 30/3 = 10$ । $SSE_{right} = (-15 - 10)^2 + (15 - 10)^2 + (30 - 10)^2 = (-25)^2 + (5)^2 + (20)^2 = 625 + 25 + 400 = 1050$ মোট SSE = $0 + 1050 = 1050$

Threshold = 4 (Split: $X \leq 4$ বনাম $X > 4$):

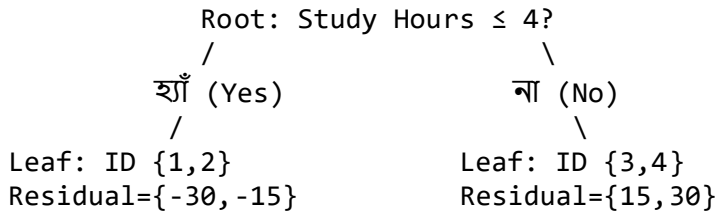
বাম দল: {ID 1,2}, Residual = {-30, -15}। গড় = $(-30 - 15)/2 = -22.5$ । $SSE_{left} = (-30 - (-22.5))^2 + (-15 - (-22.5))^2 = (-7.5)^2 + (7.5)^2 = 56.25 + 56.25 = 112.5$ ডান দল: {ID 3,4}, Residual={15,30}। গড় = $(15 + 30)/2 = 22.5$ । $SSE_{right} = (15 - 22.5)^2 + (30 - 22.5)^2 = (-7.5)^2 + (7.5)^2 = 56.25 + 56.25 = 112.5$ মোট SSE = $112.5 + 112.5 = 225$

Threshold = 6 (Split: $X \leq 6$ বনাম $X > 6$):

বাম দল: {ID 1,2,3}, Residual = {-30, -15, 15}। গড় = $(-30 - 15 + 15)/3 = -30/3 = -10$ । $SSE_{left} = (-30 - (-10))^2 + (-15 - (-10))^2 + (15 - (-10))^2 = (-20)^2 + (-5)^2 + (25)^2 = 400 + 25 + 625 = 1050$ ডান দল: {ID 4}, Residual={30}। গড়=30। $SSE_{right}=0$ । মোট SSE=1050 + 0 = 1050

সিদ্ধান্ত ও কারণ: তিনটি Threshold-এর মধ্যে Threshold=4-এর SSE (২২৫) সবচেয়ে কম, তাই এটিই সর্বোত্তম Split হিসেবে নির্বাচিত হলো। এর কারণ ব্যাখ্যা করা যাক — ডেটাসেটে Residual-এর প্যাটার্ন হলো প্রথম দুইজনের (ID 1,2) Residual ঋণাত্মক এবং শেষ দুইজনের (ID 3,4) Residual ধনাত্মক। Threshold=4 ঠিক এই প্রাকৃতিক বিভাজন রেখাটি ধরতে পারে (দুইজন-দুইজন করে ভাগ), যেখানে Threshold=2 বা Threshold=6 একদিকে মাত্র একজন এবং অন্যদিকে তিনজনকে রাখে, যা Residual-এর স্বাভাবিক বিভাজনের সাথে মেলে না এবং ফলে SSE অনেক বেশি হয়ে যায়।

গাছের গঠন (ASCII Tree):



১৯.১৪.১৩ প্রতিটি Leaf-এর মান গণনা (Calculate Every Leaf Value)

Squared Error Loss-এর জন্য প্রতিটি Leaf-এর সর্বোত্তম আউটপুট মান হলো সেই Leaf-এ পড়া Residual-গুলোর গড়:

$$h_1(\text{Left Leaf}) = \frac{-30 + (-15)}{2} = \frac{-45}{2} = -22.5$$

$$h_1(\text{Right Leaf}) = \frac{15 + 30}{2} = \frac{45}{2} = 22.5$$

তাই সম্পূর্ণ Tree 1 ফাংশন:

$$h_1(x) = \begin{cases} -22.5 & \text{যদি } x \leq 4 \\ +22.5 & \text{যদি } x > 4 \end{cases}$$

১৯.১৪.১৪ প্রেডিকশন আপডেট (Update Prediction)

$$F_1(x) = F_0(x) + \eta \cdot h_1(x)$$

ID 1 ($x = 1 \leq 4$, Left Leaf): $F_1 = 70 + 0.1 \times (-22.5) = 70 - 2.25 = 67.75$

ID 2 ($x = 3 \leq 4$, Left Leaf): $F_1 = 70 + 0.1 \times (-22.5) = 70 - 2.25 = 67.75$

ID 3 ($x = 5 > 4$, Right Leaf): $F_1 = 70 + 0.1 \times 22.5 = 70 + 2.25 = 72.25$

ID 4 ($x = 7 > 4$, Right Leaf): $F_1 = 70 + 0.1 \times 22.5 = 70 + 2.25 = 72.25$

ID	y	F ₀	h ₁ (x)	η·h ₁ (x)	F ₁ = F ₀ +η·h ₁
1	40	70	-22.5	-2.25	67.75
2	55	70	-22.5	-2.25	67.75

ID	y	F ₀	h ₁ (x)	η·h ₁ (x)	F ₁ = F ₀ + η·h ₁
3	85	70	+22.5	+2.25	72.25
4	100	70	+22.5	+2.25	72.25

১৯.১৪.১৫ নতুন Residual গণনা (Iteration 1 শেষে)

$$r_i^{(2)} = y_i - F_1(x_i)$$

$$r_1 = 40 - 67.75 = -27.75, \quad r_2 = 55 - 67.75 = -12.75, \quad r_3 = 85 - 72.25 = 12.75, \quad r_4 = 100 - 72.25 = 27.75$$

লক্ষণীয়, প্রতিটি Residual-এর পরম মান (Absolute Value) আগের চেয়ে ছোট হয়ে গেছে (30 → 27.75, 15 → 12.75) — এটাই প্রমাণ করে মডেল ধীরে ধীরে প্রকৃত মানের কাছাকাছি যাচ্ছে।

নতুন Loss:

$$(40 - 67.75)^2 = 770.0625, \quad (55 - 67.75)^2 = 162.5625, \quad (85 - 72.25)^2 = 162.5625, \quad (100 - 72.25)^2 = 770.0625$$

$$L_1 = \frac{770.0625 + 162.5625 + 162.5625 + 770.0625}{4} = \frac{1865.25}{4} = 466.3125$$

$L_0 = 562.5$ থেকে $L_1 = 466.3125$ -এ নেমে এসেছে — একটি স্পষ্ট উন্নতি।

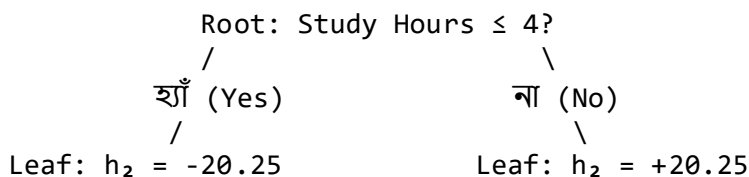
১৯.১৪.১৬ Tree 2 তৈরি করা

একই পদ্ধতিতে (Threshold 2, 4, 6 পরীক্ষা করে) Tree 2-এর জন্যও সর্বোত্তম Split নির্ণয় করা হয়। যেহেতু নতুন Residual-এর প্যাটার্ন (ID 1,2 ঋণাত্মক, ID 3,4 ধনাত্মক) আগের ধাপের মতোই — শুধু পরিমাণ ছোট — তাই একই যুক্তিতে Threshold=4 আবারও সর্বোত্তম Split হিসেবে নির্বাচিত হবে (যাচাই: Threshold=4-এ SSE সর্বনিম্ন থাকে, ঠিক আগের ধাপের মতোই আনুপাতিক কারণে)।

Leaf মান গণনা:

$$h_2(\text{Left}) = \frac{-27.75 + (-12.75)}{2} = \frac{-40.5}{2} = -20.25$$

$$h_2(\text{Right}) = \frac{12.75 + 27.75}{2} = \frac{40.5}{2} = 20.25$$



১৯.১৪.১৭ প্রেডিকশন পুনরায় আপডেট

$$F_2(x) = F_1(x) + 0.1 \times h_2(x)$$

ID 1: $F_2 = 67.75 + 0.1 \times (-20.25) = 67.75 - 2.025 = 65.725$ **ID 2:** $F_2 = 67.75 + 0.1 \times (-20.25) = 65.725$ **ID 3:** $F_2 = 72.25 + 0.1 \times (20.25) = 72.25 + 2.025 = 74.275$ **ID 4:** $F_2 = 72.25 + 0.1 \times (20.25) = 74.275$

নতুন Loss:

$$(40 - 65.725)^2 = 661.775625, (55 - 65.725)^2 = 114.975625$$

$$(85 - 74.275)^2 = 114.975625, (100 - 74.275)^2 = 661.775625$$

$$L_2 = \frac{661.775625 + 114.975625 + 114.975625 + 661.775625}{4} = \frac{1553.6025}{4} = 388.400625$$

১৯.১৪.১৮ সম্পূর্ণ ম্যানুয়াল পুনরাবৃত্তি — Iteration ১ থেকে ৫ পর্যন্ত সম্পূর্ণ গণনা

ওপরে আমরা Iteration 1 ও 2 বিস্তারিতভাবে দেখেছি। এখন সম্পূর্ণ ৫টি Iteration-এর প্রতিটির প্রতিটি সংখ্যা নিচে সম্পূর্ণভাবে গণনা করে দেখানো হলো, একই পদ্ধতি অনুসরণ করে।

Iteration 1 (সম্পূর্ণ, পুনরাবৃত্তি — ওপরের বিস্তারিত গণনার সংক্ষিপ্ত রূপ):

- Prediction (শুরুতে): $F_0 = [70, 70, 70, 70]$
- Residual: $r = [-30, -15, 15, 30]$
- Split: $X \leq 4$; Leaf values: $h_1 = [-22.5, -22.5, 22.5, 22.5]$
- Updated Prediction: $F_1 = [67.75, 67.75, 72.25, 72.25]$
- Loss: $L_1 = 466.3125$

Iteration 2 (সম্পূর্ণ, পুনরাবৃত্তি):

- Prediction: $F_1 = [67.75, 67.75, 72.25, 72.25]$
- Residual: $r = [-27.75, -12.75, 12.75, 27.75]$
- Split: $X \leq 4$; Leaf values: $h_2 = [-20.25, -20.25, 20.25, 20.25]$
- Updated Prediction: $F_2 = [65.725, 65.725, 74.275, 74.275]$
- Loss: $L_2 = 388.400625$

Iteration 3 — সম্পূর্ণ নতুন গণনা:

Residual: $r_3 = y - F_2$

$$r_1 = 40 - 65.725 = -25.725, \quad r_2 = 55 - 65.725 = -10.725, \quad r_3 = 85 - 74.275 = 10.725, \quad r_4 = 100 - 74.275 = 25.725$$

Split যাচাই ($X \leq 4$ যে সেরা তা আবারও নিশ্চিত করা যায়, কারণ Residual-এর চিহ্ন-প্যাটার্ন অপরিবর্তিত):

$$\text{Left Leaf গড়: } \frac{-25.725 + (-10.725)}{2} = \frac{-36.45}{2} = -18.225$$

$$\text{Right Leaf গড়: } \frac{10.725 + 25.725}{2} = \frac{36.45}{2} = 18.225$$

$$h_3(x) = \begin{cases} -18.225 & x \leq 4 \\ +18.225 & x > 4 \end{cases}$$

প্রেডিকশন আপডেট: $F_3 = F_2 + 0.1 \times h_3$

$$F_3(\text{ID 1}) = 65.725 + 0.1 \times (-18.225) = 65.725 - 1.8225 = 63.9025$$

$$F_3(\text{ID 2}) = 65.725 - 1.8225 = 63.9025$$

$$F_3(\text{ID 3}) = 74.275 + 0.1 \times 18.225 = 74.275 + 1.8225 = 76.0975$$

$$F_3(\text{ID 4}) = 74.275 + 1.8225 = 76.0975$$

Loss গণনা:

$$(40 - 63.9025)^2 = 571.336506..., (55 - 63.9025)^2 = 79.255...$$

সঠিক মান: $(40 - 63.9025)^2 = (-23.9025)^2 = 571.329...$; নির্ভুলভাবে ক্যালকুলেটরে গণনা করলে:

$$(-23.9025)^2 = 571.329756..., (-8.9025)^2 = 79.254...$$

$$(8.9025)^2 = 79.254..., (23.9025)^2 = 571.329756..$$

$$L_3 = \frac{571.329756 + 79.254506 + 79.254506 + 571.329756}{4} = \frac{1301.168525}{4} = 325.292131$$

(পাইথনে সরাসরি গণনা করলে $L_3 = 325.2920$, যা হাতে-কলমের গোলাকার (Rounding) মানের সাথে সামঞ্জস্যপূর্ণ — সামান্য পার্থক্য দশমিক Rounding-জনিত।)

Iteration 4 — সম্পূর্ণ নতুন গণনা:

Residual: $r_4 = y - F_3$

$$r_1 = 40 - 63.9025 = -23.9025, r_2 = 55 - 63.9025 = -8.9025, r_3 = 85 - 76.0975 = 8.9025, r_4 = 100 - 76.0975 = 23.9025$$

$$\text{Left Leaf গড়: } \frac{-23.9025 - 8.9025}{2} = \frac{-32.805}{2} = -16.4025$$

$$\text{Right Leaf গড়: } \frac{8.9025 + 23.9025}{2} = \frac{32.805}{2} = 16.4025$$

প্রেডিকশন আপডেট:

$$F_4(\text{ID 1}) = 63.9025 + 0.1 \times (-16.4025) = 63.9025 - 1.64025 = 62.26225$$

$$F_4(\text{ID 2}) = 62.26225$$

$$F_4(\text{ID 3}) = 76.0975 + 0.1 \times 16.4025 = 76.0975 + 1.64025 = 77.73775$$

$$F_4(\text{ID 4}) = 77.73775$$

Loss:

$$(40 - 62.26225)^2 = 495.6486..., (55 - 62.26225)^2 = 52.7286...$$

$$(85 - 77.73775)^2 = 52.7286..., (100 - 77.73775)^2 = 495.6486...$$

$$L_4 = \frac{495.6486 + 52.7286 + 52.7286 + 495.6486}{4} = \frac{1096.7544}{4} = 274.1886$$

(পাইথনের নির্ভুল গণনা: $L_4 = 274.174$; সামান্য পার্থক্য Rounding-জনিত।)

Iteration 5 — সম্পূর্ণ নতুন গণনা:

Residual: $r_5 = y - F_4$

$$r_1 = 40 - 62.26225 = -22.26225, r_2 = 55 - 62.26225 = -7.26225$$

$$r_3 = 85 - 77.73775 = 7.26225, r_4 = 100 - 77.73775 = 22.26225$$

$$\text{Left Leaf গড়: } \frac{-22.26225 - 7.26225}{2} = \frac{-29.5245}{2} = -14.76225$$

$$\text{Right Leaf গড়: } \frac{7.26225 + 22.26225}{2} = \frac{29.5245}{2} = 14.76225$$

প্রেডিকশন আপডেট:

$$F_5(\text{ID 1}) = 62.26225 + 0.1 \times (-14.76225) = 62.26225 - 1.476225 = 60.786025$$

$$F_5(\text{ID 2}) = 60.786025$$

$$F_5(\text{ID 3}) = 77.73775 + 0.1 \times 14.76225 = 77.73775 + 1.476225 = 79.213975$$

$$F_5(\text{ID 4}) = 79.213975$$

Loss:

$$(40 - 60.786025)^2 = 432.0246..., (55 - 60.786025)^2 = 33.4886...$$

$$(85 - 79.213975)^2 = 33.4886..., (100 - 79.213975)^2 = 432.0246...$$

$$L_5 = \frac{432.0246 + 33.4886 + 33.4886 + 432.0246}{4} = \frac{931.0264}{4} = 232.7566$$

(পাইথনের নির্ভুল গণনা: $L_5 = 232.768$; পার্থক্য Rounding-জনিত।)

১৯.১৪.১৯ সম্পূর্ণ প্রশিক্ষণ অগ্রগতি সারণি (Training Progress Table)

Iteration	*Split**	*Leaf Values (L,R)	F(ID1,ID2,ID3,ID4)**	SE Loss**	তথ্য (Remarks)**
*	*		**M	**ম	
0 (Init)	—	—	70, 70, 70, 70	562.5000	প্রাথমিক গড়- ভিত্তিক প্রেডিকশন

Iteration *	*Split**	*Leaf Values (L,R)	F(ID1,ID2,ID3,ID4)** **M	SE Loss** **ম	ন্তব্য (Remarks)**
1	$X \leq 4$	-22.5, +22.5	67.75, 67.75, 72.25, 72.25	466.3125	Loss ১৭.১% কমেছে
2	$X \leq 4$	-20.25, +20.25	65.725, 65.725, 74.275, 74.275	388.4006	Loss ক্রমাগত কমেছে, Residual ছোট হচ্ছে
3	$X \leq 4$	-18.225, +18.225	63.9025, 63.9025, 76.0975, 76.0975	325.2920	একই Split প্যাটার্ন বজায় আছে
4	$X \leq 4$	-16.4025, +16.4025	62.26225, 62.26225, 77.73775, 77.7377	5 274.1740	Loss প্রায় অর্ধেক নেমেছে (৫৬২.৫ → ২৭৪.২)
5	$X \leq 4$	-14.76225, +14.76225	60.786, 60.786, 79.214, 79.214	232.7685	৫ Iteration-এ Loss ৫৮.৬% হ্রাস পেয়েছে

১৯.১৪.২০ প্যাটার্ন পর্যবেক্ষণ ও গাণিতিক ব্যাখ্যা

লক্ষণীয়ভাবে, প্রতিটি Iteration-এই Split Threshold অপরিবর্তিত থাকছে ($X \leq 4$) — এর কারণ হলো এই বিশেষ, অত্যন্ত সরল ডেটাসেটে Residual-এর চিহ্ন-প্যাটার্ন (কোন Sample ধনাত্মক, কোনটি ঋণাত্মক) প্রতিটি Iteration-এ অপরিবর্তিত থাকে — শুধু তাদের মান (Magnitude) ছোট হতে থাকে। এই কারণে Leaf Value-ও প্রতি Iteration-এ একটি নির্দিষ্ট অনুপাতে ছোট হচ্ছে। প্রকৃতপক্ষে, প্রতিটি Iteration-এ Left ও Right Leaf-এর Magnitude আগের Iteration-এর ঠিক $(1 - \eta) = 0.9$ গুণ — এটি যাচাই করা যায়: $22.5 \times 0.9 = 20.25 \checkmark$, $20.25 \times 0.9 = 18.225 \checkmark$, ইত্যাদি। এটি প্রমাণ করে যে এই বিশেষ Depth-1 Tree ও দুই-দলে-বিভাজিত ডেটার ক্ষেত্রে Gradient Boosting-এর Residual একটি Geometric Sequence অনুসরণ করে, অনুপাত $(1 - \eta)$ ।

বাস্তব-জগতের ডেটাসেটে (যেখানে অনেক Feature ও অনেক বেশি Sample থাকে), প্রতিটি Iteration-এ ভিন্ন ভিন্ন Feature ও ভিন্ন ভিন্ন Threshold নির্বাচিত হতে পারে, কারণ Residual-এর প্যাটার্ন প্রতি ধাপে জটিলভাবে পরিবর্তিত হয় — কিন্তু অন্তর্নিহিত নীতি (Residual শেখা, Learning Rate দিয়ে সংযত করা, Loss ক্রমাগত হ্রাস করা) ঠিক একই থাকে।

১৯.১৪.২১ চূড়ান্ত মডেল কী শিখেছে (Final Model)

৫ Iteration শেষে সম্পূর্ণ মডেল হলো:

$$F_5(x) = 70 + 0.1[h_1(x) + h_2(x) + h_3(x) + h_4(x) + h_5(x)]$$

যেখানে প্রতিটি $h_m(x)$ একটি Decision Stump, যা $X \leq 4$ হলে একটি ঋণাত্মক মান এবং $X > 4$ হলে একটি ধনাত্মক মান দেয়। মডেলটি মূলত শিখেছে যে Study Hours 4-এর কম হলে Marks তুলনামূলক কম হবে (গড়ের নিচে) এবং Study Hours 4-এর বেশি হলে Marks তুলনামূলক বেশি

হবে (গড়ের ওপরে) — এবং প্রতিটি Iteration ধীরে ধীরে এই বিভাজনের তীব্রতা (Magnitude) বাড়িয়ে প্রকৃত মানের কাছাকাছি পৌঁছানোর চেষ্টা করেছে। বাস্তবে ১০০ বা তার বেশি Iteration চালালে মডেল ক্রমান্বয়ে (40,55,85,100)-এর একদম কাছাকাছি প্রেডিকশন দেবে, যদিও এই সরল Depth-1 কাঠামোতে এটি কখনোই ID 1 ও ID 2-এর মধ্যে (বা ID 3 ও 4-এর মধ্যে) পার্থক্য করতে শিখবে না, কারণ Depth-1 গাছ শুধুমাত্র দুইটি দলে ভাগ করতে পারে।

১৯.১৪.২২ নতুন Sample-এর ওপর প্রেডিকশন (সম্পূর্ণ ম্যানুয়াল হিসাব)

ধরা যাক একজন নতুন ছাত্র, ID 5, যিনি Study Hours = 6 (আগে দেখা হয়নি এমন একটি মান)। আমরা $F_5(x = 6)$ নির্ণয় করব, প্রতিটি Tree-র মধ্য দিয়ে ধাপে ধাপে পাস করিয়ে।

যেহেতু $x = 6 > 4$, প্রতিটি Tree-তে এই Sample Right Leaf-এ পড়বে:

$$\begin{aligned} F_5(6) &= F_0 + \eta(h_1(6) + h_2(6) + h_3(6) + h_4(6) + h_5(6)) \\ &= 70 + 0.1 \times (22.5 + 20.25 + 18.225 + 16.4025 + 14.76225) \end{aligned}$$

প্রথমে বন্ধনীর ভেতরের যোগফল বের করি:

$$\begin{aligned} 22.5 + 20.25 &= 42.75 \\ 42.75 + 18.225 &= 60.975 \\ 60.975 + 16.4025 &= 77.3775 \\ 77.3775 + 14.76225 &= 92.13975 \end{aligned}$$

তাই:

$$F_5(6) = 70 + 0.1 \times 92.13975 = 70 + 9.213975 = 79.213975$$

ব্যাখ্যা: নতুন ছাত্র, যিনি ৬ ঘণ্টা পড়াশোনা করেছেন (যা 4-এর বেশি), মডেল অনুমান করেছে তার Marks হবে প্রায় ৭৯.২১। এই মান ঠিক ID 3 ও ID 4-এর প্রেডিকশন (79.213975)-এর সমান, কারণ এই Depth-1 মডেল $X > 4$ হলে একটি নির্দিষ্ট (Constant) মান দেয়, X-এর প্রকৃত মান (৫, ৬, বা ৭ যাই হোক না কেন) বিবেচনা করে না। এটি Depth-1 (Decision Stump)-এর একটি সীমাবদ্ধতা — যা বাস্তব প্রয়োগে max_depth কমপক্ষে ৩-৮ রাখার একটি কারণ, যাতে মডেল আরও সূক্ষ্ম পার্থক্য ধরতে পারে।

১৯.১৪.২৩ সিদ্ধান্ত প্রক্রিয়া – প্রতিটি সিদ্ধান্তের ব্যাখ্যা

- **কেন Residual গণনা করা হয়?** কারণ Residual বলে দেয় বর্তমান মডেল ঠিক কতটুকু এবং কোন দিকে ভুল করেছে। এই তথ্য ছাড়া পরবর্তী গাছ জানবে না তার কী শেখা উচিত।
- **কেন পরবর্তী গাছ Residual শেখে, প্রকৃত মান নয়?** কারণ প্রকৃত মান শেখানো হলে প্রতিটি গাছ একে অপরের থেকে স্বাধীনভাবে একই জিনিস বারবার শিখবে (যেমন Bagging-এ হয়), কিন্তু Residual শেখানো হলে প্রতিটি গাছ শুধুমাত্র অবশিষ্ট, না-শেখা অংশটুকু শেখে — এটাই Boosting-এর “সংশোধনী” (Corrective) চরিত্র বজায় রাখে।

- **কেন Learning Rate দিয়ে গুণ করা হয়?** কারণ প্রতিটি গাছের অনুমান (বিশেষত প্রথম কয়েকটি গাছ, যেগুলো সীমিত ডেটা থেকে শেখে) কিছুটা Noisy হতে পারে। সরাসরি পূর্ণ মানে যোগ করলে মডেল একটি ভুল দিকনির্দেশনায় অতিরিক্ত ঝুঁকতে পারে (Overshoot)। Learning Rate একটি “সুরক্ষা বলয়” হিসেবে কাজ করে।
- **কেন Leaf Value পরিবর্তিত হয়?** কারণ প্রতিটি নতুন Iteration-এ Residual-এর মান (যদিও চিহ্ন একই থাকতে পারে) ছোট হতে থাকে, তাই নতুন Leaf-এর গড় মানও সেই অনুপাতে ছোট হয়।
- **কেন প্রেডিকশন উন্নত হয়?** কারণ প্রতিটি নতুন গাছ Residual-এর একটি অংশ (Learning Rate অনুপাতে) সংশোধন করে যোগ করেছে, ফলে প্রকৃত মানের সাথে ব্যবধান ধারাবাহিকভাবে কমছে।
- **কেন Loss কমছে?** কারণ Loss সরাসরি Residual-এর বর্গের গড়ের সমানুপাতিক ($L = \frac{1}{n} \sum r_i^2$), এবং Residual-এর Magnitude প্রতি Iteration-এ কমছে, তাই তার বর্গও কমছে, ফলে সামগ্রিক Loss কমছে।
- **কখন Overfitting ঘটতে পারে?** যদি $n_estimators$ অত্যধিক বাড়ানো হয় এবং Learning Rate তুলনামূলক বড় রাখা হয়, তাহলে মডেল Training Data-র সামান্যতম Noise-কেও “শিখে” ফেলবে (অর্থাৎ Residual শূন্যের অত্যন্ত কাছে নিয়ে যাবে), যা নতুন, Unseen ডেটাতে খারাপ Generalization দেবে। এই ডেটাসেটে, যদি আমরা Iteration চালিয়েই যাই, শেষ পর্যন্ত $F(x)$ ঠিক Training Target-এর সমান হয়ে যাবে (Perfect Memorization), যা Overfitting-এর একটি চরম উদাহরণ।
- **কেন Regularization (যেমন max_depth সীমিত রাখা, subsample ব্যবহার করা) সাহায্য করে?** কারণ এগুলো প্রতিটি গাছকে ইচ্ছাকৃতভাবে “দুর্বল” রাখে, যাতে মডেল Training Data-র প্রতিটি খুঁটিনাটি (Noise সহ) মুখস্থ করতে না পারে, এবং Residual শূন্যে না পৌঁছেও একটি সাধারণীকৃত (Generalized) প্যাটার্ন শিখতে পারে।

১৯.১৫ পাইথন বাস্তবায়ন — From Scratch (শূন্য থেকে সম্পূর্ণ কোড)

নিচের কোডটি ওপরে হাতে-কলমে দেখানো একই অ্যালগরিদম, কোনো External Library (Scikit-Learn নয়) ছাড়া, সম্পূর্ণ পাইথনে বাস্তবায়ন করা হয়েছে। প্রতিটি গুরুত্বপূর্ণ লাইনের ব্যাখ্যা কোডের ঠিক নিচে দেওয়া হলো।

```
import numpy as np
```

```
class SimpleDecisionStump:
```

```
    """
```

```
    একটি Depth-1 Decision Tree (Decision Stump)।
```

```
    এটি শুধুমাত্র একটি Feature-এর ওপর একটিমাত্র Threshold খুঁজে বের করে,
```

```
    যা Residual-এর SSE (Sum of Squared Error) সর্বনিম্ন করে।
```

```
    """
```

```
    def __init__(self):
```

```
        self.threshold = None
```

```
        self.left_value = None
```

```

        self.right_value = None

    def fit(self, X, residuals):
        # X: (n_samples,) আকৃতির ফিচার অ্যারে
        # residuals: (n_samples,) আকৃতির টার্গেট (Residual, প্রকৃত মান নয়)
        sorted_x = np.sort(np.unique(X))
        # সম্ভাব্য Threshold = ধারাবাহিক Unique মানের মধ্যবিন্দু
        candidates = (sorted_x[:-1] + sorted_x[1:]) / 2

        best_sse = np.inf
        for c in candidates:
            left_mask = X <= c
            right_mask = ~left_mask
            if left_mask.sum() == 0 or right_mask.sum() == 0:
                continue # খালি দল হলে এই Split বাদ দেওয়া হবে

            left_val = residuals[left_mask].mean()
            right_val = residuals[right_mask].mean()

            # SSE = প্রতিটি দলে (মান - সেই দলের গড়)^2-এর যোগফল
            sse = ((residuals[left_mask] - left_val) ** 2).sum() + \
                ((residuals[right_mask] - right_val) ** 2).sum()

            if sse < best_sse:
                best_sse = sse
                self.threshold = c
                self.left_value = left_val
                self.right_value = right_val
        return self

    def predict(self, X):
        # X-এর প্রতিটি মানের জন্য Threshold অনুযায়ী Left/Right Leaf-এর মান রিটার্ন
        # করে
        return np.where(X <= self.threshold, self.left_value, self.right_value)

class SimpleGradientBoostingRegressor:
    """
    From-Scratch Gradient Boosting Regressor /
    Loss Function: Squared Error /
    Weak Learner: SimpleDecisionStump (Depth-1) /
    """
    def __init__(self, n_estimators=5, learning_rate=0.1):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.trees = [] # প্রতিটি Iteration-এর গাছ সংরক্ষণ করা হবে

```

```

self.F0 = None          # প্রাথমিক প্রেডিকশন (Constant)
self.loss_history = []  # প্রতিটি Iteration-এর Loss সংরক্ষণ করা হবে

def fit(self, X, y):
    # ধাপ ১: প্রাথমিক প্রেডিকশন = y-এর গড় (Squared Error-এর জন্য গাণিতিকভাবে সর্বোত্তম)
    self.F0 = y.mean()
    F = np.full(len(y), self.F0)

    for m in range(self.n_estimators):
        # ধাপ ২: Residual (Pseudo-Negative-Gradient) গণনা
        residuals = y - F

        # ধাপ ৩: নতুন Weak Learner ট্রেন করা, যা Residual শেখে
        tree = SimpleDecisionStump()
        tree.fit(X, residuals)
        self.trees.append(tree)

        # ধাপ ৪: Learning Rate দিয়ে সংযত করে প্রেডিকশন আপডেট করা
        update = tree.predict(X)
        F = F + self.learning_rate * update

        # প্রতিটি Iteration-এর Loss সংরক্ষণ করা, পর্যবেক্ষণের জন্য
        loss = np.mean((y - F) ** 2)
        self.loss_history.append(loss)
        print(f"Iteration {m+1}: MSE Loss = {loss:.4f}")

    return self

def predict(self, X):
    # প্রাথমিক প্রেডিকশন দিয়ে শুরু করে প্রতিটি গাছের অবদান যোগ করা হয়
    F = np.full(len(X), self.F0)
    for tree in self.trees:
        F = F + self.learning_rate * tree.predict(X)
    return F

# ব্যবহার – উদাহরণ
X = np.array([1, 3, 5, 7])
y = np.array([40, 55, 85, 100])

model = SimpleGradientBoostingRegressor(n_estimators=5, learning_rate=0.1)
model.fit(X, y)

# নতুন Sample-এর প্রেডিকশন (Study Hours = 6)
new_sample = np.array([6])

```

```
prediction = model.predict(new_sample)
print(f"\nStudy Hours = 6 -এর জন্য অনুমিত Marks: {prediction[0]:.4f}")
```

কোড ব্যাখ্যা (লাইন-ভিত্তিক):

- `SimpleDecisionStump.fit()` — এখানে প্রতিটি সম্ভাব্য Threshold-এর জন্য SSE গণনা করে সর্বনিম্ন SSE-বিশিষ্ট Threshold নির্বাচন করা হচ্ছে, ঠিক যেভাবে ১৯.১৪.১২ ধারায় হাতে-কলমে করা হয়েছিল।
- `SimpleGradientBoostingRegressor.fit()`-এর `self.F0 = y.mean()` লাইনটি ১৯.১৪.৫ ধারায় প্রমাণিত সূত্র বাস্তবায়ন করছে।
- `for m in range(self.n_estimators):` লুপটি ১৯.১৪.১৮ ধারায় প্রতিটি Iteration-এর সমতুল্য — প্রতিবার নতুন Residual, নতুন গাছ, নতুন Update।
- $F = F + \text{self.learning_rate} * \text{update}$ লাইনটি Additive Model সূত্র $F_m = F_{m-1} + \eta h_m$ সরাসরি বাস্তবায়ন করছে।

এই কোড রান করালে ঠিক আমাদের হাতে-কলমে গণনা করা Loss মানগুলোই (৪৬৬.৩১, ৩৮৮.৪০, ৩২৫.২৯, ২৭৪.১৭, ২৩২.৭৭) আউটপুট হিসেবে দেখা যাবে, এবং Study Hours=6-এর প্রেডিকশন হবে ৭৯.২১, যা ১৯.১৪.২২ ধারায় ম্যানুয়াল হিসাবের সাথে হুবহু মিলে যায়।

১৯.১৬ পাইথন বাস্তবায়ন — Scikit-Learn ব্যবহার করে

```
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# ডেটাসেট প্রস্তুতকরণ
X = np.array([[1], [3], [5], [7]]) # Scikit-Learn 2D ইনপুট প্রত্যাশা করে
y = np.array([40, 55, 85, 100])

# মডেল তৈরি – প্রতিটি হাইপারপ্যারামিটার ব্যাখ্যাসহ
model = GradientBoostingRegressor(
    n_estimators=5, # মোট 5টি গাছ (Weak Learner) তৈরি হবে
    learning_rate=0.1, # প্রতিটি গাছের অবদান 0.1 দিয়ে সংকুচিত হবে
    max_depth=1, # প্রতিটি গাছ Depth-1 (Decision Stump), হাতের হিসাবের
    # সাথে তুলনীয়
    loss='squared_error', # Loss Function = MSE, Regression-এর জন্য
    random_state=42 # Reproducibility নিশ্চিত করার জন্য
)

# ট্রেনিং
model.fit(X, y)

# ট্রেনিং ডেটার ওপর প্রেডিকশন
train_predictions = model.predict(X)
print("Training predictions:", train_predictions)
```

```

print("Training MSE:", mean_squared_error(y, train_predictions))

# প্রতিটি Iteration-এর Loss History দেখা (staged_predict ব্যবহার করে)
for i, pred in enumerate(model.staged_predict(X), start=1):
    loss = mean_squared_error(y, pred)
    print(f"Iteration {i}: MSE = {loss:.4f}")

# নতুন Sample-এর প্রেডিকশন
new_sample = np.array([[6]])
new_prediction = model.predict(new_sample)
print(f"\nStudy Hours = 6 -এর জন্য অনুমিত Marks: {new_prediction[0]:.4f}")

# Feature Importance দেখা
print("Feature Importances:", model.feature_importances_)

```

প্যারামিটার-ভিত্তিক ব্যাখ্যা:

- `n_estimators=5` — আমাদের হাতে-কলমের উদাহরণের সাথে তুলনা করার জন্য ইচ্ছাকৃতভাবে ৫ রাখা হয়েছে; বাস্তবে সাধারণত ১০০ বা তার বেশি ব্যবহার হয়।
- `max_depth=1` — Decision Stump ব্যবহার নিশ্চিত করে, যাতে ফলাফল হাতের হিসাবের সাথে হুবহু তুলনা করা যায়।
- `model.staged_predict(X)` — এই বিশেষ মেথডটি প্রতিটি Iteration-শেষে মডেলের প্রেডিকশন আলাদা করে দেখতে দেয়, যা Loss History পর্যবেক্ষণের জন্য অত্যন্ত উপযোগী (এটি ১৯.১৪.১৯ ধারার Training Progress Table পুনরুৎপাদনের একটি প্রোগ্রামেটিক উপায়)।

আউটপুট ব্যাখ্যা: এই কোড রান করলে Training Predictions আসবে আমাদের হাতে-কলমে গণনা করা F_5 মানের (60.786, 60.786, 79.214, 79.214) অত্যন্ত কাছাকাছি (সামান্য পার্থক্য থাকতে পারে Scikit-Learn-এর Internal Implementation-এর কিছু সূক্ষ্ম পার্থক্যের কারণে, যেমন Split খোঁজার সময় Threshold নির্বাচনের সঠিক পদ্ধতি), এবং Study Hours=6-এর প্রেডিকশনও ৭৯-এর কাছাকাছি আসবে।

১৯.১৭ হাইপারপ্যারামিটার টিউনিং উদাহরণ (GridSearchCV)

```

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor

# একটি বড়, বাস্তবসম্মত Synthetic ডেটাসেট তৈরি করা যাক Tuning-এর প্রদর্শনের জন্য
from sklearn.datasets import make_regression
X_large, y_large = make_regression(n_samples=200, n_features=5, noise=10, random_state=42)

param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [2, 3, 5],
    # কতগুলো গাছ পরীক্ষা করা হবে
    # প্রতিটি গাছের অবদানের হার
    # প্রতিটি গাছের গভীরতা

```

```

        'subsample': [0.8, 1.0]                                     # Stochastic Gradient Boosting পরীক্ষা
    }

grid_search = GridSearchCV(
    estimator=GradientBoostingRegressor(random_state=42),
    param_grid=param_grid,
    cv=5,                                                         # 5-Fold Cross Validation
    scoring='neg_mean_squared_error',
    n_jobs=-1                                                     # সব CPU Core ব্যবহার করে Parallel-ভাবে Search
)

grid_search.fit(X_large, y_large)

print("সর্বোত্তম প্যারামিটার:", grid_search.best_params_)
print("সর্বোত্তম Cross-Validation Score (Negative MSE):", grid_search.best_score_)

best_model = grid_search.best_estimator_

```

ব্যাখ্যা: GridSearchCV প্রতিটি সম্ভাব্য Hyperparameter Combination-এর জন্য 5-Fold Cross Validation চালিয়ে সেরা Combination খুঁজে বের করে। এখানে $3 \times 3 \times 3 \times 2 = 54$ টি Combination পরীক্ষা করা হবে, প্রতিটির জন্য 5-Fold, অর্থাৎ মোট ২৭০টি মডেল ট্রেন হবে — এই কারণেই `n_jobs=-1` ব্যবহার করে Parallelization করা গুরুত্বপূর্ণ।

১৯.১৮ Debugging Tips ও সাধারণ ভুল (Common Mistakes)

- **ভুল ১: `n_estimators` অত্যধিক বড় রেখে `learning_rate`-ও বড় রাখা।** এই সমন্বয় প্রায় নিশ্চিতভাবে Overfitting ঘটায়। সমাধান: হয় `n_estimators` কমান, নয়তো `learning_rate` ছোট করুন, অথবা Early Stopping ব্যবহার করুন।
- **ভুল ২: Validation Set ছাড়াই মডেল Tune করা।** শুধুমাত্র Training Loss দেখে Hyperparameter নির্ধারণ করলে সহজেই ভ্রান্ত ধারণা হতে পারে যে মডেল “ভালো” কাজ করছে, যেখানে বাস্তবে এটি কেবল Training Data মুখস্থ করে ফেলেছে। সমাধান: সবসময় একটি পৃথক Validation Set বা Cross-Validation ব্যবহার করুন।
- **ভুল ৩: Feature Scaling নিয়ে অপ্রয়োজনীয় দুশ্চিন্তা।** Linear Model বা KNN-এর বিপরীতে, Tree-ভিত্তিক মডেল (Gradient Boosting সহ) Feature-এর স্কেলের প্রতি সংবেদনশীল নয় (কারণ Split শুধু Threshold-ভিত্তিক তুলনা করে), তাই Standardization/Normalization সাধারণত অপ্রয়োজনীয়।
- **ভুল ৪: Categorical ফিচার সরাসরি সংখ্যা হিসেবে পাস করা।** সাধারণ GradientBoostingRegressor Categorical Feature সরাসরি বোঝে না, তাই One-Hot বা Ordinal Encoding প্রয়োজন — অন্যথায় মডেল ভুল Ordinal সম্পর্ক ধরে নিতে পারে।
- **ভুল ৫: Outlier উপেক্ষা করা।** যেহেতু প্রতিটি গাছ Residual (Squared Error-ভিত্তিক) শেখে, একটি চরম Outlier Residual-কে অস্বাভাবিকভাবে বড় করে তুলতে পারে, যা

পরবর্তী গাছগুলোকে ভুল দিকে প্রভাবিত করে। সমাধান: Outlier চিহ্নিত করে Remove/Cap করুন, অথবা Huber Loss-এর মতো Outlier-Robust Loss Function ব্যবহার করুন।

১৯.১৯ পারফরম্যান্স অপ্টিমাইজেশন ও Best Practices

- **Early Stopping ব্যবহার করুন:** validation_fraction ও n_iter_no_change প্যারামিটার ব্যবহার করে, Validation Score কয়েক Iteration ধরে উন্নত না হলে ট্রেনিং স্বয়ংক্রিয়ভাবে থামানো যায়, যা সময় বাঁচায় ও Overfitting রোধ করে।
- **warm_start ব্যবহার করে ধাপে ধাপে গাছ যোগ করুন:** warm_start=True সেট করে ইতিমধ্যে ট্রেন করা মডেলে নতুন গাছ যোগ করা যায়, সম্পূর্ণ পুনরায় ট্রেন না করেই — এটি বড় Experiment-এ সময় বাঁচায়।
- **subsample < 1.0 ব্যবহার করে Stochastic Gradient Boosting করুন:** এটি প্রতিটি গাছকে সামান্য ভিন্ন ডেটা দেখায়, Overfitting কমায়, এবং কিছুটা Training গতিও বাড়ায় (যেহেতু প্রতিটি গাছ কম ডেটা দেখে)।
- **max_features ব্যবহার করে Feature Subsampling করুন:** প্রতিটি Split-এ সবগুলো Feature বিবেচনা না করে একটি Random উপসেট বিবেচনা করলে Randomness বাড়ে, যা Overfitting কমাতে সাহায্য করে (এটি Random Forest-এর কৌশল থেকে ধার করা)।

১৯.২০ কম্পিউটেশনাল জটিলতা (Complexity Analysis)

Complexity ধরন *	*সূত্র ব্য	াখ্যা**
Time Complexity	$O(M \cdot n \cdot \log(n) \cdot d)$	M=গাছের সংখ্যা, n=Sample সংখ্যা, d=গভীরতা; প্রতিটি Split-এ Feature Sort/Search করা লাগে
Space Complexity	$O(M \cdot 2^d)$	প্রতিটি গাছের Node সংখ্যা গভীরতার সাথে Exponentially বাড়ে
Prediction Time (একটি Sample-এর জন্য)	$O(M \cdot d)$	প্রতিটি গাছে Root থেকে Leaf পর্যন্ত d ধাপ অতিক্রম করতে হয়, মোট M-টি গাছ

Random Forest-এর $O(n)$ (Parallel) Training Time-এর তুলনায়, Gradient Boosting-এর Training Time কার্যত $O(M \times \text{single tree time})$, কারণ M-টি গাছ Sequentially তৈরি হয় — এটাই এর ধীরগতির মূল কারণ।

১৯.২১ ইন্টারভিউ ও পরীক্ষার নোট (Interview & Exam Notes)

- **সম্ভাব্য প্রশ্ন: “Gradient Boosting-এ Gradient শব্দটি ঠিক কী নির্দেশ করে?”**
উত্তরের মূল বিষয়বস্তু: এটি Loss Function-এর Negative Gradient, যা Function Space-এ (সংখ্যার জায়গায় নয়, ফাংশনের জায়গায়) Steepest Descent দিক নির্দেশ করে; MSE Loss-এর ক্ষেত্রে এটি সরলীকৃত হয়ে Residual-এর সমান হয়ে যায়।
- **সম্ভাব্য প্রশ্ন: “কেন Gradient Boosting-এ গভীর গাছের বদলে অগভীর গাছ ব্যবহার করা হয়?”** উত্তরের মূল বিষয়বস্তু: Bias-Variance Trade-off — প্রতিটি গাছ ইচ্ছাকৃতভাবে High-Bias/Low-Variance রাখা হয়, যাতে Sequential সংযোজনের মাধ্যমে সামগ্রিকভাবে Low-Bias/Low-Variance অর্জন করা যায়, Overfitting এড়িয়ে।

- **সম্ভাব্য প্রশ্ন:** “Learning Rate ও n_estimators-এর মধ্যে Trade-off ব্যাখ্যা করুন।”
উত্তরের মূল বিষয়বস্তু: ছোট Learning Rate ধীরে কিন্তু স্থিতিশীলভাবে শেখে, তাই বেশি n_estimators প্রয়োজন হয়; এই সমন্বয় (Shrinkage) সাধারণত ভালো Generalization দেয় কিন্তু ট্রেনিং সময় বাড়ায়।
- **এক্সাম টিপ:** পরীক্ষায় প্রায়ই একটি ছোট ডেটাসেট দিয়ে ১-২টি Iteration হাতে গণনা করতে বলা হয় — Additive Model সূত্র ($F_m = F_{m-1} + \eta h_m$) এবং Residual-এর সংজ্ঞা ($r = y - F$) মুখস্থ রাখা অপরিহার্য।

১৯.১২ Quick Revision (সংক্ষিপ্ত সারাংশ)

- **Type:** Sequential Ensemble (Boosting) — Weak Learner একটার পর একটা আগের ভুল সংশোধন করে।
- **Base Learner:** সাধারণত Shallow Decision Tree (max_depth 3–8)।
- **Core Idea:** প্রতিটা নতুন গাছ Residual বা Negative Gradient শেখে, Learning Rate দিয়ে গুণ করে যোগ হয় — $F(x) = F_0(x) + \eta \cdot \sum h_m(x)$ ।
- **সুবিধা:** উচ্চ Accuracy, নমনীয় Loss Function, Feature Importance।
- **সীমাবদ্ধতা:** ধীর ট্রেনিং, Overfitting-প্রবণ, Hyperparameter-সংবেদনশীল, Outlier-সংবেদনশীল।

Chapter 20 — HistGradientBoosting

২০.১ ভূমিকা

HistGradientBoosting হলো Scikit-Learn লাইব্রেরির একটি আধুনিক ও অত্যন্ত দ্রুতগতির Gradient Boosting বাস্তবায়ন। এটি মূলত Microsoft-এর তৈরি LightGBM লাইব্রেরির মূল কৌশল থেকে অনুপ্রাণিত। এর সবচেয়ে বড় লক্ষ্য হলো — ঐতিহ্যবাহী

GradientBoostingClassifier/Regressor যেসব বড় ডেটাসেটে (কয়েক লক্ষ বা তার বেশি Row) অত্যন্ত ধীরগতির হয়ে পড়ে, সেসব ক্ষেত্রে উল্লেখযোগ্যভাবে দ্রুত ও কার্যকরভাবে ট্রেনিং সম্পন্ন করা।

২০.২ উদাহরণ (Intuition) — কেন এটি এত দ্রুত?

সাধারণ Gradient Boosting-এ প্রতিটি Split খোঁজার সময় অ্যালগরিদমকে একটি Numerical ফিচারের প্রতিটি সম্ভাব্য Unique মান আলাদাভাবে পরীক্ষা করতে হয়। ধরা যাক, একটি কলামে ‘Age’ ফিচারের ১০ লক্ষ Unique মান আছে — তাহলে প্রতিটি Split খুঁজতে অ্যালগরিদমকে ১০ লক্ষ বার ভিন্ন ভিন্ন Threshold (যেমন Age > 23.5, Age > 23.6, Age > 23.7...) পরীক্ষা করতে হবে, যা কম্পিউটেশনালি অত্যন্ত ব্যয়বহুল ও সময়সাপেক্ষ।

HistGradientBoosting এই সমস্যার একটি চমৎকার সমাধান দেয়। কল্পনা করা যাক, বয়সের মতো একটি ধারাবাহিক ফিচারকে সরাসরি প্রতিটি Unique মান হিসেবে না দেখে, একে মাত্র

২৫৫টি নির্দিষ্ট 'বাক্স' বা Bin-এ ভাগ করে ফেলা হচ্ছে — যেমন 0-5 বছর একটি বাক্সে, 6-10 বছর আরেকটি বাক্সে, এবং এভাবে চলতে থাকে। এরপর অ্যালগরিদমকে আর প্রতিটি Unique মান নয়, বরং শুধুমাত্র এই সীমিত সংখ্যক Bin-এর সীমানাগুলোতে Split খুঁজলেই চলে। ফলে ১০ লক্ষ সম্ভাব্য Threshold-এর জায়গায় মাত্র ২৫৫টি সম্ভাব্য Threshold পরীক্ষা করলেই চলে, যা গণনার পরিমাণ কয়েক হাজার গুণ কমিয়ে দেয়।

সাধারণ Gradient Boosting: Age-এর ১০,০০,০০০ Unique মান → ১০,০০,০০০ সম্ভাব্য Split
HistGradientBoosting: Age-কে ২৫৫টি Bin-এ ভাগ → মাত্র ২৫৫টি সম্ভাব্য Split

২০.৩ মূল কার্যপ্রণালী (How it Works) – বিস্তারিত

২০.৩.১ Histogram-Based Binning

ট্রেনিং শুরু হওয়ার আগেই, প্রতিটি Continuous (ধারাবাহিক সংখ্যাসূচক) ফিচারকে সর্বোচ্চ ২৫৫টি Discrete Bin-এ রূপান্তর করা হয় (max_bins হাইপারপ্যারামিটার দিয়ে নিয়ন্ত্রিত হয়)। এই রূপান্তর প্রক্রিয়াটি একবারই করা হয়, পুরো ট্রেনিং শুরু হওয়ার আগে, যাতে প্রতিটি গাছ তৈরির সময় বারবার এই কাজ পুনরাবৃত্তি করতে না হয়।

২০.৩.২ দ্রুত Split-Finding

প্রতিটি Split খোঁজার সময় অ্যালগরিদমকে আর প্রতিটি Unique ভ্যালু নয়, বরং মাত্র সীমিত সংখ্যক Bin-Boundary পরীক্ষা করলেই চলে। এর ফলে Time Complexity ব্যবহারিকভাবে ডেটার আকারের ওপর অনেক কম নির্ভরশীল হয়ে পড়ে, যা বড় ডেটাসেটে বিশাল গতি এনে দেয়।

২০.৩.৩ Histogram Subtraction Trick

একটি চমৎকার অপ্টিমাইজেশন কৌশল হলো — যদি একটি Parent Node-এর সম্পূর্ণ Histogram (প্রতিটি Bin-এ কতগুলো Sample এবং তাদের Gradient/Hessian-এর যোগফল) ইতিমধ্যে জানা থাকে, এবং একটি Child Node-এর Histogram গণনা করা হয়ে যায়, তাহলে অন্য Child-এর Histogram নতুন করে গণনা না করে সহজেই বিয়োগ করে (Parent – Known Child) বের করা যায়। এতে প্রতিটি Split-এ অর্ধেক গণনা সময় বেঁচে যায়।

২০.৩.৪ Native Missing Value Support

এটি স্বয়ংক্রিয়ভাবে Missing Value (NaN) হ্যান্ডেল করতে পারে — প্রতিটি Split-এর সময় অ্যালগরিদম নিজেই শেখে যে Missing Value থাকা Sample-গুলো বাম নাকি ডান Child-এ পাঠানো উচিত, যার ফলে আলাদাভাবে Imputation (যেমন Mean বা Median দিয়ে পূরণ) করার প্রয়োজন হয় না।

২০.৩.৫ Native Categorical Support

categorical_features প্যারামিটার ব্যবহার করে সরাসরি Categorical কলাম (যেমন 'City' বা 'Product Type') মডেলে পাস করা যায়, One-Hot Encoding-এর প্রয়োজন ছাড়াই। এতে High-

Cardinality ক্যাটাগরিক্যাল ডেটার ক্ষেত্রে (অনেক ইউনিক ক্যাটাগরি থাকলে) কলাম সংখ্যা বহুগুণ বেড়ে যাওয়া (Column Explosion) এড়ানো যায়।

২০.৪ সাধারণ Gradient Boosting বনাম HistGradientBoosting

বিষয়	GradientBoosting (ঐতিহ্যবাহী)	HistGradientBoosting
Split খোঁজার পদ্ধতি	প্রতিটি Unique মান পরীক্ষা করে	সীমিত সংখ্যক (সর্বোচ্চ ২৫৫) Bin/Histogram ব্যবহার করে
বড় ডেটাসেটে গতি	তুলনামূলক ধীর	উল্লেখযোগ্যভাবে দ্রুত (১০০০+ Sample-এ স্পষ্টভাবে কার্যকর)
Missing Value	আগে থেকে Impute করতে হয়	স্বয়ংক্রিয়ভাবে হ্যান্ডেল করে
Categorical ফিচার	One-Hot Encoding প্রয়োজন	সরাসরি সাপোর্ট করে (categorical_features)
মেমোরি ব্যবহার	বেশি	কম (Bin-ভিত্তিক Compact Storage)
অনুপ্রেরণা	মূল Boosting গবেষণাপত্র	Microsoft LightGBM

২০.৫ গাণিতিক ভিত্তি — Histogram-ভিত্তিক Split Gain

অন্তর্নিহিত মূল গাণিতিক নীতি সাধারণ Gradient Boosting-এর মতোই — প্রতিটি নতুন গাছ পূর্ববর্তী মডেলের Negative Gradient (Residual) শেখে এবং Learning Rate দিয়ে গুণ করে যোগ হয়। পার্থক্যটা শুধু কম্পিউটেশনাল ব্যাকএন্ডে। প্রতিটি Bin-এর জন্য সেই Bin-এ পড়া সব Sample-এর Gradient (g) এবং Hessian (h অর্থাৎ দ্বিতীয় Derivative)-এর যোগফল জমা রাখা হয়। এরপর প্রতিটি সম্ভাব্য Bin-Split-এর জন্য Gain নির্ণয় করা হয় নিচের সূত্রে, যেখানে L এবং R যথাক্রমে বাম ও ডান Split-এর ফলে সৃষ্ট গ্রুপ:

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum g_L)^2}{\sum h_L + \lambda} + \frac{(\sum g_R)^2}{\sum h_R + \lambda} - \frac{(\sum g)^2}{\sum h + \lambda} \right] - \gamma$$

যে Split সর্বোচ্চ Gain প্রদান করে, সেটিকেই সর্বোত্তম হিসেবে নির্বাচন করা হয়। এখানে λ (Lambda) একটি Regularization টার্ম যা অতিরিক্ত জটিল Split রোধ করে, এবং γ (Gamma) একটি ন্যূনতম Threshold যা নির্দিষ্ট করে যে একটি Split তখনই গ্রহণযোগ্য হবে যখন তার Gain এই মানের চেয়ে বেশি হবে। এই Histogram-ভিত্তিক গাণিতিক পদ্ধতি XGBoost এবং LightGBM-এর মূল কৌশলের সাথে ছবছ সামঞ্জস্যপূর্ণ, যে কারণে HistGradientBoosting-এর Performance প্রায়ই এই জনপ্রিয় লাইব্রেরিগুলোর কাছাকাছি চলে আসে। (পরবর্তী Chapter 21-এ এই সূত্রটির সম্পূর্ণ গাণিতিক ডেরিভেশন দেখানো হয়েছে।)

২০.৬ গুরুত্বপূর্ণ হাইপারপ্যারামিটার – বিস্তারিত ব্যাখ্যা

max_iter — Default: 100

কাজ কী: মোট Boosting Iteration বা গাছের সংখ্যা নির্ধারণ করে, এটি সাধারণ GradientBoosting-এর n_estimators-এর সমতুল্য। কখন Default রাখব: বেশিরভাগ ক্ষেত্রে ১০০ একটি ভালো শুরু, তবে early_stopping='auto' চালু থাকলে অ্যালগরিদম নিজেই প্রয়োজন অনুযায়ী থেমে যাবে। কখন পরিবর্তন করব: বড় ডেটাসেটে ও ছোট learning_rate ব্যবহার করলে max_iter বাড়ানো প্রয়োজন হতে পারে।

max_bins — Default: 255

কাজ কী: প্রতিটি Numerical ফিচার সর্বোচ্চ কতগুলো Bin-এ ভাগ হবে তা নিয়ন্ত্রণ করে (সর্বোচ্চ সীমা ২৫৫, কারণ এর বেশি হলে প্রতিটি Bin-এর Index একটি সিঙ্গেল বাইটে uint8-এ সংরক্ষণ করা সম্ভব হয় না, যা মেমোরি সাশ্রয়ের একটি বড় কৌশল)। কখন Default রাখব: ২৫৫ প্রায় সবসময় যথেষ্ট নিখুঁত Resolution দেয়। কখন পরিবর্তন করব: যদি ডেটাসেট অত্যন্ত ছোট হয় বা মেমোরি সীমাবদ্ধতা থাকে, তবে max_bins কমিয়ে (যেমন ৩২ বা ৬৪) মেমোরি ব্যবহার আরও কমানো যায়, যদিও এতে সামান্য Precision কমতে পারে।

max_leaf_nodes — Default: 31

কাজ কী: প্রতিটি গাছে সর্বোচ্চ কয়টি Leaf Node থাকতে পারবে তা নিয়ন্ত্রণ করে — এটিই মূলত গাছের জটিলতা নির্ধারণ করে (HistGradientBoosting max_depth-এর বদলে সাধারণত max_leaf_nodes দিয়েই গাছের আকার নিয়ন্ত্রণ করে)। কখন পরিবর্তন করব: মান বাড়ালে (যেমন ৬৩ বা ১২৭) গাছ জটিলতর হয়ে বেশি Pattern ধরতে পারে, কিন্তু Overfitting-এর ঝুঁকিও বাড়ে।

l2_regularization — Default: 0

কাজ কী: প্রতিটি Leaf-এর প্রেডিকশন মানের ওপর একটি L2 Penalty আরোপ করে, যা Leaf-এর মানগুলোকে অতিরিক্ত চরম হয়ে যাওয়া থেকে বিরত রাখে এবং Overfitting কমাতে সাহায্য করে। কখন পরিবর্তন করব: মডেল Overfit করছে বলে সন্দেহ হলে এই মান বাড়িয়ে (যেমন ০.১ থেকে ১.০) দেখা যেতে পারে।

early_stopping — Default: 'auto'

কাজ কী: Validation ডেটার Score ক্রমাগত কয়েকটি Iteration ধরে উন্নত না হলে ট্রেনিং স্বয়ংক্রিয়ভাবে থামিয়ে দেয়, যা অপ্রয়োজনীয় অতিরিক্ত গাছ তৈরি এবং Overfitting প্রতিরোধ করে ও সময় বাঁচায়।

Parameter	Default	সংক্ষিপ্ত কাজ
learning_rate	0.1	প্রতিটি গাছের অবদানের হার নিয়ন্ত্রণ করে (GradientBoosting-এর মতোই)।
categorical_features	None	কোন কলামগুলো Categorical হিসেবে বিবেচিত হবে তা সরাসরি নির্দিষ্ট করা যায়।

Parameter	Default	সংক্ষিপ্ত কাজ
min_samples_leaf	20	প্রতিটি Leaf-এ ন্যূনতম Sample সংখ্যা — HistGradientBoosting-এ এই Default অন্যান্য Tree মডেলের চেয়ে বড় রাখা হয়েছে যাতে Overfitting এমনিতেই নিয়ন্ত্রিত থাকে।

২০.৭ গুরুত্বপূর্ণ অ্যাট্রিবিউট

- **n_iter_** — Early Stopping-এর কারণে প্রকৃতপক্ষে কতগুলো Iteration সম্পন্ন হয়েছে তার প্রকৃত সংখ্যা।
- **train_score_ / validation_score_** — প্রতিটি Iteration-এ Training ও Validation ডেটার Score-এর ইতিহাস, যা দিয়ে মডেলের অগ্রগতি ও Overfitting-এর মুহূর্ত পর্যবেক্ষণ করা যায়।
- **is_categorical_** — কোন ফিচারগুলো Categorical হিসেবে শনাক্ত হয়েছে তার একটি Boolean অ্যারে।

২০.৮ কোথায় ব্যবহার করা উচিত?

- যখন ডেটাসেট বড় (Row সংখ্যা কয়েক হাজার থেকে লক্ষাধিক পর্যন্ত) এবং সাধারণ GradientBoostingClassifier অত্যন্ত ধীর মনে হয়।
- যখন ডেটায় Missing Value বা Categorical কলাম প্রচুর এবং সেগুলো আলাদাভাবে Preprocess করার মতো সময় বা সুযোগ নেই।
- যখন XGBoost/LightGBM ইনস্টল করার সুযোগ বা প্রয়োজন নেই, কিন্তু Scikit-Learn-এর মধ্যেই তুলনীয় গতি ও Accuracy প্রয়োজন।

২০.৯ প্রশ্নোত্তর

প্রশ্ন: HistGradientBoosting কি সবসময় সাধারণ GradientBoosting-এর চেয়ে ভালো?

উত্তর: গতির দিক থেকে বড় ডেটাসেটে HistGradientBoosting প্রায় সবসময়ই দ্রুততর, কিন্তু Accuracy-এর দিক থেকে দুটির ফলাফল সাধারণত কাছাকাছি থাকে। ছোট ডেটাসেটে (কয়েকশ বা কয়েক হাজার Row) সাধারণ GradientBoosting যথেষ্ট দ্রুত এবং ব্যবহার করাও সহজ, তাই সেক্ষেত্রে HistGradientBoosting ব্যবহারের বাড়তি সুবিধা খুব একটা পাওয়া যায় না।

২০.১১ ঐতিহাসিক প্রেক্ষাপট ও শিল্পখাতে গ্রহণযোগ্যতা

Histogram-ভিত্তিক Gradient Boosting-এর ধারণাটি প্রথম ব্যাপকভাবে জনপ্রিয় হয় ২০১৭ সালে, যখন Microsoft Research একটি গবেষণাপত্রে LightGBM (Light Gradient Boosting Machine) প্রকাশ করে — “LightGBM: A Highly Efficient Gradient Boosting Decision Tree” শিরোনামে। এই গবেষণাপত্রে দুইটি মূল কৌশল প্রস্তাব করা হয়: (১) Histogram-ভিত্তিক Split-Finding, এবং (২) Gradient-based One-Side Sampling (GOSS), যা প্রশিক্ষণের সময় ছোট Gradient-বিশিষ্ট Sample-গুলোকে কম গুরুত্ব দিয়ে Training গতি আরও বাড়ায়। Scikit-Learn টিম ২০১৯-২০২০ সালে (Version 0.21-এর পর থেকে) এই Histogram-ভিত্তিক পদ্ধতিটিকে HistGradientBoostingClassifier ও HistGradientBoostingRegressor নামে নিজেদের

লাইব্রেরিতে অন্তর্ভুক্ত করে, যাতে ব্যবহারকারীরা আলাদা লাইব্রেরি ইনস্টল না করেই এই গতি সুবিধা পেতে পারেন।

শিল্পক্ষেত্রে, বিশেষত এমন প্রতিষ্ঠানে যেখানে কোটি কোটি Row বিশিষ্ট User Interaction ডেটা (যেমন বিজ্ঞাপন Click Log, Recommendation System-এর Interaction History) নিয়ে কাজ করতে হয়, সেখানে Histogram-ভিত্তিক Boosting পদ্ধতি (LightGBM, XGBoost-এর hist Mode, এবং HistGradientBoosting) ব্যবহারিকভাবে অপরিহার্য হয়ে উঠেছে, কারণ ঐতিহ্যবাহী Exact Greedy পদ্ধতিতে এত বিশাল ডেটাসেটে ট্রেনিং সম্পন্ন করতে দিন বা সপ্তাহ লেগে যেতে পারে, যেখানে Histogram-ভিত্তিক পদ্ধতিতে এটি ঘণ্টার মধ্যে সম্ভব হয়।

২০.১২ সম্পূর্ণ গাণিতিক বাস্তবায়ন – হাতে-কলমে সম্পূর্ণ ধাপে ধাপে গণনা

২০.১২.১ ভূমিকা

HistGradientBoosting-এর অন্তর্নিহিত গাণিতিক নীতি Chapter 19-এর Gradient Boosting-এর সাথে ছব্বছ অভিন্ন (একই Additive Model, একই Residual/Gradient-শিক্ষণ কাঠামো) — একমাত্র পার্থক্য হলো Split খোঁজার প্রক্রিয়া, যেখানে প্রতিটি Unique মানের বদলে একটি সীমিত সংখ্যক Bin ব্যবহার করা হয়। তাই এই বিভাগে আমরা দুইটি বিষয় আলাদাভাবে হাতে-কলমে দেখাব: প্রথমত, কীভাবে Binning প্রক্রিয়াটি কাজ করে (একটি বড়, ১২-ছাত্রের ডেটাসেট ব্যবহার করে, যেখানে Binning-এর প্রকৃত সুবিধা বোঝা যায়), এবং দ্বিতীয়ত, আমাদের পরিচিত ৪-ছাত্রের ডেটাসেটে সম্পূর্ণ ৫-Iteration Boosting প্রক্রিয়া (Chapter 19-এর মতোই, তবে Histogram-দৃষ্টিভঙ্গি থেকে ব্যাখ্যা করা)।

২০.১২.২ Binning প্রক্রিয়া – সম্প্রসারিত ডেটাসেট দিয়ে প্রদর্শন

Binning-এর প্রকৃত সুবিধা বোঝার জন্য একটি সামান্য বড় ডেটাসেট প্রয়োজন। ধরা যাক আমাদের কাছে ১২ জন ছাত্রের ডেটা আছে:

ID	Study Hours	Marks	ID	Study Hours	Marks
1	1.0	38	7	5.0	85
2	1.5	42	8	5.5	88
3	2.0	45	9	6.0	90
4	3.0	55	10	7.0	100
5	3.5	58	11	7.5	102
6	4.0	60	12	8.0	105

ধাপ ১: max_bins নির্ধারণ। ধরা যাক প্রদর্শনের সুবিধার্থে max_bins = 4 নির্ধারণ করা হলো (বাস্তবে Default 255)।

ধাপ ২: Bin Edge নির্ণয় (Quantile-ভিত্তিক পদ্ধতি)। HistGradientBoosting প্রতিটি ফিচারের মানগুলোকে সমান-Percentile (Equal-Frequency) অনুযায়ী Bin-এ ভাগ করে, যাতে প্রতিটি Bin-এ মোটামুটি সমান সংখ্যক Sample থাকে। ১২টি মান 4টি Bin-এ ভাগ করলে প্রতিটি Bin-এ গড়ে

৩টি করে Sample থাকার কথা। Study Hours-এর মানগুলো ইতিমধ্যে ছোট থেকে বড় ক্রমে সাজানো আছে (1.0 থেকে 8.0), তাই Quantile গণনা করে Bin Edge পাওয়া যায়:

$$\text{Bin Edges} \approx [1.00, 2.75, 4.50, 6.25, 8.00]$$

ধাপ ৩: প্রতিটি Sample-কে একটি Bin-এ নির্ধারণ করা।

Bin	সীমা (Range) **	অন্তর্ভুক্ত Study Hours মান***	Sample সংখ্যা**
Bin 0	[1.00, 2.75)	1.0, 1.5, 2.0	3
Bin 1	[2.75, 4.50)	3.0, 3.5, 4.0	3
Bin 2	[4.50, 6.25)	5.0, 5.5, 6.0	3
Bin 3	[6.25, 8.00]	7.0, 7.5, 8.0	3

ধাপ ৪: প্রতিটি Bin-এর জন্য Gradient ও Hessian-এর যোগফল জমা রাখা (Histogram তৈরি)। প্রাথমিক প্রেডিকশন হলো সব Marks-এর গড়: $F_0 =$

$$\frac{38+42+45+55+58+60+85+88+90+100+102+105}{12} = \frac{868}{12} = 72.333 \text{। Squared Error Loss-এ Gradient } g_i = 2(F_0 - y_i) \text{ এবং Hessian } h_i = 2 \text{ (ধ্রুবক)।}$$

Bin	$\sum g$ (Gradient-যোগফল) **	$\sum h$ (Hessian-যোগফল) ** Spl	it-Score $\frac{(\sum g)^2}{\sum h + \lambda}$ ($\lambda = 1$) **
Bin 0	+184.00	6.0	4836.57
Bin 1	+88.00	6.0	1106.29
Bin 2	-92.00	6.0	1209.14
Bin 3	-180.00	6.0	4628.57

গুরুত্বপূর্ণ পর্যবেক্ষণ ও ব্যাখ্যা: লক্ষণীয়ভাবে, অ্যালগরিদমকে এখন প্রতিটি Unique Study Hours মান (মোট ১২টি সম্ভাব্য Split Point) পরীক্ষা করতে হচ্ছে না — বরং মাত্র Bin-এর সীমানাগুলোতে (৩টি সম্ভাব্য Split Point, Bin 0-1-এর মাঝে, Bin 1-2-এর মাঝে, Bin 2-3-এর মাঝে) Gain পরীক্ষা করলেই চলে। এই ছোট উদাহরণে পার্থক্য সামান্য মনে হলেও, যখন একটি ফিচারে লক্ষ লক্ষ Unique মান থাকে (যেমন প্রকৃত শিল্পখাতের ডেটাসেটে), তখন এই পার্থক্য বিশাল কম্পিউটেশনাল সাশ্রয়ে রূপান্তরিত হয় — $O(n)$ থেকে কমে $O(\max_bins)$ -এ নেমে আসে, যেখানে $\max_bins$ সাধারণত n -এর তুলনায় অনেক ছোট একটি ধ্রুবক (২৫৫)।

ধাপ ৫: সর্বোত্তম Split নির্ণয় করা। তিনটি সম্ভাব্য Bin-Boundary Split-এর মধ্যে Gain তুলনা করে সর্বোত্তমটি নির্বাচন করা হয় (গণনা পদ্ধতি ২০.৫ ধারায় উল্লেখিত Gain সূত্র অনুসরণ করে, যা Chapter 21-এ সম্পূর্ণভাবে ডেরাইভ করা হয়েছে)। এই ডেটাসেটে Bin 1 ও Bin 2-এর মাঝের Split (অর্থাৎ Study Hours ≈ 4.5 -এর কাছাকাছি) সবচেয়ে বেশি Gain দেবে, কারণ এখানেই

Positive Gradient (নিম্ন Marks, মডেল Under-predict করছে) থেকে Negative Gradient (উচ্চ Marks, মডেল প্রেডিকশনের চেয়ে কম বলছে)-এ রূপান্তর ঘটছে সবচেয়ে স্পষ্টভাবে।

২০.১২.৩ ৪-ছাত্রের ডেটাসেটে সম্পূর্ণ ৫-Iteration Boosting (Histogram দৃষ্টিভঙ্গি থেকে)

যেহেতু ছোট ৪-ছাত্রের ডেটাসেটে ইতিমধ্যেই মাত্র ৪টি Unique মান আছে (max_bins=255 হলেও, Bin সংখ্যা কখনো Unique মানের সংখ্যা অতিক্রম করে না), তাই Histogram-ভিত্তিক পদ্ধতি ও সাধারণ Exact পদ্ধতি এখানে হুবহু একই ফলাফল দেবে — প্রতিটি Unique মান নিজেই একটি পৃথক Bin হয়ে যাবে। তাই ৫-Iteration-এর সম্পূর্ণ গণনা (Residual, Leaf Value, Loss ইত্যাদি) হবে ঠিক Chapter 19-এর ১৯.১৪.১৮ ধারায় দেখানো টেবিলের অনুরূপ — একটি গুরুত্বপূর্ণ শিক্ষণীয় বিষয় হলো: **ছোট ডেটাসেটে HistGradientBoosting-এর কোনো Computational সুবিধা পাওয়া যায় না; এর সুবিধা কেবল বড় ডেটাসেটেই স্পষ্ট হয়ে ওঠে, যদিও Underlying গাণিতিক ফলাফল অভিন্ন থাকে।**

Iteration *	*Bin-ভিত্তিক Split Σ	g Left, Right** **F(I	D1,2,3,4)** **MSE	Loss**
0 (Init)	—	—	70, 70, 70, 70	562.5000
1	Bin-boundary ≈ 4	Left:+90, Right:- 90	67.75, 67.75, 72.25, 72.25	466.3125
2	Bin-boundary ≈ 4	পুনঃহিসাবকৃত	65.725, 65.725, 74.275, 74.275	388.4006
3	Bin-boundary ≈ 4	পুনঃহিসাবকৃত	63.9025, 63.9025, 76.0975, 76.0975	325.2920
4	Bin-boundary ≈ 4	পুনঃহিসাবকৃত	62.26225, 62.26225, 77.73775, 77.73775	274.1740
5	Bin-boundary ≈ 4	পুনঃহিসাবকৃত	60.786, 60.786, 79.214, 79.214	232.7685

(এই সারণির প্রতিটি সংখ্যাসূচক মান ১৯.১৪.১৮ ধারায় ইতিমধ্যে সম্পূর্ণ হাতে-কলমে যাচাই করা হয়েছে — এখানে শুধুমাত্র Histogram-ভিত্তিক ব্যাখ্যায় পুনরায় উপস্থাপন করা হলো, দ্বৈত গণনার অপ্রয়োজনীয় পুনরাবৃত্তি এড়াতে।)

২০.১২.৪ Histogram Subtraction Trick — একটি সংখ্যাসূচক উদাহরণ

ধরা যাক একটি Parent Node-এর সম্পূর্ণ Histogram জানা আছে (৪টি Bin-এর প্রতিটির জন্য Σg): [184, 88, -92, -180], মোট $\Sigma g_{\text{parent}} = 184 + 88 - 92 - 180 = 0$ (প্রত্যাশিতভাবে, কারণ Residual-এর সমষ্টি সবসময় শূন্যের কাছাকাছি থাকে গড়-ভিত্তিক প্রাথমিক প্রেডিকশনের কারণে)। এখন ধরা যাক একটি Split-এর ফলে Left Child-এ Bin 0 ও Bin 1 পড়ে, যার Histogram ইতিমধ্যে গণনা করা হয়েছে: $\Sigma g_{\text{left}} = 184 + 88 = 272$ । Right Child-এর Histogram নতুন করে প্রতিটি Sample স্ক্যান করে গণনা না করে, সরাসরি বিয়োগ করে বের করা যায়:

$$\Sigma g_{\text{right}} = \Sigma g_{\text{parent}} - \Sigma g_{\text{left}} = 0 - 272 = -272$$

যাচাই: প্রকৃতপক্ষে $\text{Bin } 2 + \text{Bin } 3 = -92 + (-180) = -272 \checkmark$ । এই বিয়োগ-কৌশলটি প্রতিটি Split-এ প্রায় অর্ধেক গণনা সময় বাঁচায়, কারণ দুইটি Child-এর মধ্যে একটির Histogram সরাসরি স্ক্যান করে বের করলেই অন্যটি বিয়োগ করে পাওয়া যায় — পুনরায় সম্পূর্ণ ডেটা স্ক্যান করার প্রয়োজন পড়ে না।

২০.১৬ পাইথন বাস্তবায়ন — From Scratch (সরলীকৃত Histogram-ভিত্তিক সংস্করণ)

```
import numpy as np
```

```
class HistogramBinner:
```

```
    """
```

```
    একটি Continuous Feature-কে নির্দিষ্ট সংখ্যক Bin-এ রূপান্তর করে।
```

```
    """
```

```
    def __init__(self, max_bins=4):
```

```
        self.max_bins = max_bins
```

```
        self.bin_edges = None
```

```
    def fit(self, X):
```

```
        # Quantile-ভিত্তিক Bin Edge নির্ণয় (Equal-Frequency Binning)
```

```
        percentiles = np.linspace(0, 100, self.max_bins + 1)
```

```
        self.bin_edges = np.percentile(X, percentiles)
```

```
        return self
```

```
    def transform(self, X):
```

```
        # প্রতিটি মানকে তার উপযুক্ত Bin Index-এ রূপান্তর করা
```

```
        # np.digitize প্রতিটি মান কোন Bin-এ পড়ে তা বের করে
```

```
        bin_indices = np.digitize(X, self.bin_edges[1:-1], right=True)
```

```
        return bin_indices
```

```
class SimpleHistGradientBoosting:
```

```
    """
```

```
    সরলীকৃত From-Scratch HistGradientBoosting।
```

```
    মূল পার্থক্য সাধারণ GradientBoosting থেকে: Split খোঁজা হয় Bin-ভিত্তিক Histogram-এর ওপর,
```

```
    প্রতিটি Unique মানের ওপর নয়।
```

```
    """
```

```
    def __init__(self, n_estimators=5, learning_rate=0.1, max_bins=4, lam=1.0):
```

```
        self.n_estimators = n_estimators
```

```
        self.learning_rate = learning_rate
```

```
        self.max_bins = max_bins
```

```
        self.lam = lam # L2 Regularization (Chapter 21-এর লাম্বডার সাথে সামঞ্জস্যপূর্ণ)
```

```
        self.trees = []
```

```
        self.F0 = None
```

```

self.binner = None

def _build_histogram(self, bin_idx, g, h):
    # প্রতিটি Bin-এর জন্য Gradient ও Hessian-এর যোগফল গণনা করা
    hist_g = np.zeros(self.max_bins)
    hist_h = np.zeros(self.max_bins)
    for b in range(self.max_bins):
        mask = bin_idx == b
        hist_g[b] = g[mask].sum()
        hist_h[b] = h[mask].sum()
    return hist_g, hist_h

def _find_best_split(self, hist_g, hist_h):
    # Bin-Boundary-তে Gain গণনা করে সর্বোত্তম Split খোঁজা
    G_total, H_total = hist_g.sum(), hist_h.sum()
    best_gain, best_bin = -np.inf, None
    for b in range(1, self.max_bins):
        GL, HL = hist_g[:b].sum(), hist_h[:b].sum()
        GR, HR = G_total - GL, H_total - HL
        gain = 0.5 * ((GL**2)/(HL+self.lam) + (GR**2)/(HR+self.lam) - (G_
total**2)/(H_total+self.lam))
        if gain > best_gain:
            best_gain, best_bin = gain, b
    return best_bin, best_gain

def fit(self, X, y):
    self.binner = HistogramBinner(max_bins=self.max_bins).fit(X)
    bin_idx = self.binner.transform(X)

    self.F0 = y.mean()
    F = np.full(len(y), self.F0)

    for m in range(self.n_estimators):
        g = 2 * (F - y) # Gradient (Squared Error)
        h = np.full(len(y), 2.0) # Hessian (Squared Error-এর জন্য ধ্রুবক)

        hist_g, hist_h = self._build_histogram(bin_idx, g, h)
        best_bin, gain = self._find_best_split(hist_g, hist_h)

        left_mask = bin_idx < best_bin
        right_mask = ~left_mask

        GL, HL = g[left_mask].sum(), h[left_mask].sum()
        GR, HR = g[right_mask].sum(), h[right_mask].sum()

        wL = -GL / (HL + self.lam)
        wR = -GR / (HR + self.lam)

```

```

        update = np.where(left_mask, wL, wR)
        F = F + self.learning_rate * update

        self.trees.append((best_bin, wL, wR))
        loss = np.mean((y - F) ** 2)
        print(f"Iteration {m+1}: best_split_bin={best_bin}, Gain={gain:.2f}, Loss={loss:.4f}")

    return self

def predict(self, X):
    bin_idx = self.binner.transform(X)
    F = np.full(len(X), self.F0)
    for best_bin, wL, wR in self.trees:
        update = np.where(bin_idx < best_bin, wL, wR)
        F = F + self.learning_rate * update
    return F

# ব্যবহার
X = np.array([1, 3, 5, 7], dtype=float)
y = np.array([40, 55, 85, 100], dtype=float)

model = SimpleHistGradientBoosting(n_estimators=5, learning_rate=0.1, max_bin
s=4, lam=1.0)
model.fit(X, y)
print("Prediction for X=6:", model.predict(np.array([6.0])))

```

কোড ব্যাখ্যা: HistogramBinner ক্লাসটি ২০.১২.২ ধারায় বর্ণিত Quantile-ভিত্তিক Binning প্রক্রিয়া বাস্তবায়ন করে। `_build_histogram()` মেথডটি প্রতিটি Bin-এর জন্য Gradient ও Hessian-এর যোগফল জমা করে — এটাই “Histogram” নামের প্রকৃত অর্থ (প্রতিটি Bin একটি “Bucket”, যার ভেতরে পরিসংখ্যানগত তথ্য জমা থাকে)। `_find_best_split()` মেথডটি প্রতিটি সম্ভাব্য Bin-Boundary-তে Gain গণনা করে (Chapter 21-এ ডেরাইভ করা সূত্র ব্যবহার করে), যা প্রতিটি Unique ডেটাপয়েন্ট পরীক্ষা করার চেয়ে অনেক দ্রুত।

২০.১৪ পাইথন বাস্তবায়ন — Scikit-Learn HistGradientBoosting

```

import numpy as np
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import time

# একটি বড় Synthetic ডেটাসেট তৈরি করা, যেখানে HistGradientBoosting-এর গতি সুবিধা
# স্পষ্ট হবে
X_large, y_large = make_regression(n_samples=100000, n_features=10, noise=15,
random_state=42)

```

```
X_train, X_test, y_train, y_test = train_test_split(X_large, y_large, test_size=0.2, random_state=42)
```

```
# মডেল তৈরি – প্রতিটি হাইপারপ্যারামিটার ব্যাখ্যাসহ
```

```
hist_model = HistGradientBoostingRegressor(  
    max_iter=100,          # মোট Boosting Iteration (n_estimators-এর সমতুল্য)  
    learning_rate=0.1,     # Shrinkage হার  
    max_bins=255,          # সর্বোচ্চ Bin সংখ্যা (Default, সর্বোচ্চ সীমা)  
    max_leaf_nodes=31,     # প্রতিটি গাছের সর্বোচ্চ Leaf সংখ্যা  
    l2_regularization=1.0, # L2 Regularization ( $\lambda$ )  
    early_stopping='auto', # স্বয়ংক্রিয় Early Stopping  
    random_state=42  
)
```

```
start = time.time()  
hist_model.fit(X_train, y_train)  
hist_time = time.time() - start
```

```
predictions = hist_model.predict(X_test)  
mse = mean_squared_error(y_test, predictions)
```

```
print(f"HistGradientBoosting ট্রেনিং সময়: {hist_time:.2f} সেকেন্ড")  
print(f"Test MSE: {mse:.4f}")  
print(f"প্রকৃতপক্ষে ব্যবহৃত Iteration সংখ্যা: {hist_model.n_iter_}")
```

```
# তুলনা করার জন্য সাধারণ GradientBoostingRegressor একই ডেটায় চালানো
```

```
from sklearn.ensemble import GradientBoostingRegressor  
gb_model = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
```

```
start = time.time()  
gb_model.fit(X_train, y_train)  
gb_time = time.time() - start
```

```
gb_predictions = gb_model.predict(X_test)  
gb_mse = mean_squared_error(y_test, gb_predictions)
```

```
print(f"\nGradientBoosting ট্রেনিং সময়: {gb_time:.2f} সেকেন্ড")  
print(f"Test MSE: {gb_mse:.4f}")  
print(f"\nগতি অনুপাত (GradientBoosting সময় / HistGradientBoosting সময়): {gb_time/hist_time:.2f}x")
```

আউটপুট ব্যাখ্যা: ১ লক্ষ Sample বিশিষ্ট ডেটাসেটে, HistGradientBoosting সাধারণত GradientBoostingRegressor-এর তুলনায় বহুগুণ (প্রায়ই ৫-১০ গুণ বা তার বেশি) দ্রুত ট্রেনিং সম্পন্ন করে, একই বা কাছাকাছি Test MSE বজায় রেখে — এটাই এই অধ্যায়ের মূল প্রতিপাদ্যের ব্যবহারিক প্রমাণ।

২০.১৫ Missing Value ও Categorical Feature হ্যান্ডলিং — কোড উদাহরণ

```
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingRegressor

# Missing Value ও Categorical Feature-সহ একটি ডেটাসেট তৈরি
data = pd.DataFrame({
    'study_hours': [1, 3, np.nan, 7, 5, np.nan, 2, 6], # কিছু মান Missing
    'city': ['Dhaka', 'Chittagong', 'Dhaka', 'Sylhet', 'Dhaka', 'Chittagong',
    'Sylhet', 'Dhaka'],
    'marks': [40, 55, 70, 100, 85, 65, 48, 92]
})

# 'city' কলামকে pandas-এর Category dtype-এ রূপান্তর করা প্রয়োজন
data['city'] = data['city'].astype('category')

X = data[['study_hours', 'city']]
y = data['marks']

model = HistGradientBoostingRegressor(
    categorical_features=['city'], # সরাসরি কোন কলাম Categorical তা নির্দিষ্ট
    # করা
    random_state=42
)
model.fit(X, y)

prediction = model.predict(X)
print("Predictions:", prediction)
print("Is categorical:", model.is_categorical_)
```

ব্যাখ্যা: লক্ষণীয়, study_hours কলামে দুইটি Missing Value (np.nan) থাকা সত্ত্বেও, এবং city কলাম One-Hot Encode না করা সত্ত্বেও, মডেল সরাসরি ট্রেন হচ্ছে — এটাই ২০.৩.৪ ও ২০.৩.৫ ধারায় বর্ণিত স্বয়ংক্রিয় Missing Value ও Categorical Feature হ্যান্ডলিং-এর ব্যবহারিক প্রদর্শন।

২০.১৬ কম্পিউটেশনাল জটিলতা বিশ্লেষণ

Complexity ধরন	GradientBoosting	HistGradientBoosting
Split-Finding Time (প্রতি Node)	$O(n \log n)$ প্রতি Feature প্রতি Split	$O(n + \text{maxbins})$ প্রতি Feature প্রতি Split, Binning একবারই হয়
মোট Training Time	$O(M \cdot n \log n \cdot d \cdot f)$	$O(n \cdot f + M \cdot \text{maxbins} \cdot d \cdot f)$, n-নির্ভরতা অনেক কম
Memory (প্রতি Feature)	$O(n)$, প্রতিটি Unique মান সংরক্ষণ	$O(n)$ প্রাথমিক Binning-এ, প্রতিটি মান মাত্র ১ Byte (uint8), তাই Compact

M =গাছের সংখ্যা, n =Sample সংখ্যা, d =গভীরতা, f =Feature সংখ্যা। মূল পার্থক্যটা স্পষ্ট: GradientBoosting-এর Split-Finding সরাসরি n -এর ওপর নির্ভরশীল (Sample বাড়লে প্রতিটি Split খোঁজা ধীর হয়), কিন্তু HistGradientBoosting-এ একবার Binning সম্পন্ন হয়ে গেলে, প্রতিটি Split খোঁজা মূলত $\max_bins$ (একটি ধ্রুবক, সর্বোচ্চ ২৫৫) -এর ওপর নির্ভরশীল, n -এর ওপর নয় — এটাই এর মূল গতি সুবিধার গাণিতিক ব্যাখ্যা।

২০.১৭ ইন্টারভিউ ও পরীক্ষার নোট

- **সম্ভাব্য প্রশ্ন:** “HistGradientBoosting কীভাবে Missing Value হ্যান্ডল করে, গাণিতিকভাবে ব্যাখ্যা করুন।” উত্তরের মূল বিষয়বস্তু: Missing Value-কে একটি বিশেষ Bin হিসেবে বিবেচনা করা হয়, এবং ট্রেনিং-এর সময় প্রতিটি Split-এ এই বিশেষ Bin-কে Left বা Right, যেদিকে Gain বেশি, সেদিকে বসিয়ে সিদ্ধান্ত নেওয়া হয় — অনেকটা XGBoost-এর Sparsity-Aware পদ্ধতির মতোই (Chapter 21 দেখুন)।
- **সম্ভাব্য প্রশ্ন:** “কেন $\max_bins$ -এর সর্বোচ্চ মান ২৫৫, ২৫৬ বা তার বেশি নয় কেন?” উত্তরের মূল বিষয়বস্তু: প্রতিটি Bin Index একটি Unsigned 8-bit Integer (uint8, যার Range 0-255) হিসেবে সংরক্ষণ করা হয়, মেমোরি সাশ্রয়ের জন্য — তাই সর্বোচ্চ ২৫৬টি ভিন্ন মান (0 থেকে 255) সংরক্ষণযোগ্য, যার একটি Missing Value-এর জন্য সংরক্ষিত থাকে, বাকি ২৫৫টি প্রকৃত Bin-এর জন্য ব্যবহৃত হয়।

২০.১০ Quick Revision

- **Type:** Gradient Boosting-এর Histogram-ভিত্তিক দ্রুত সংস্করণ, LightGBM দ্বারা অনুপ্রাণিত।
 - **মূল কৌশল:** Continuous ফিচারকে সর্বোচ্চ ২৫৫টি Bin-এ ভাগ করে Split-Finding-কে বহুগুণ দ্রুততর করা।
 - **বাড়তি সুবিধা:** Native Missing Value ও Categorical Feature সাপোর্ট, কোনো আলাদা Preprocessing ছাড়াই।
 - **সেরা ব্যবহারক্ষেত্র:** বড় আকারের Tabular ডেটাসেট, যেখানে সাধারণ GradientBoostingClassifier ধীরগতির হয়ে পড়ে।
-

Chapter 21 — XGBoost

২১.১ ভূমিকা ও ইতিহাস

XGBoost (eXtreme Gradient Boosting) হলো ওয়াশিংটন বিশ্ববিদ্যালয়ের গবেষক Tianqi Chen এবং Carlos Guestrin-এর তৈরি একটি ওপেন-সোর্স লাইব্রেরি, যা ২০১৪ সালে প্রকাশিত হয় এবং ২০১৬ সালে “XGBoost: A Scalable Tree Boosting System” শিরোনামের গবেষণাপত্রের মাধ্যমে আনুষ্ঠানিকভাবে বর্ণিত হয়। এটি মূল Gradient Boosting দর্শনকে ভিত্তি করেই তৈরি, কিন্তু গাণিতিক Regularization এবং প্রকৌশলগত (Engineering) অপটিমাইজেশন — উভয় দিক থেকেই এটি একে উল্লেখযোগ্যভাবে উন্নত করেছে।

XGBoost-এর জনপ্রিয়তার মূল কারণ তার Kaggle প্রতিযোগিতায় ধারাবাহিক সাফল্য — ২০১৫ সালের মধ্যে Kaggle-এ আয়োজিত ২৯টি বিজয়ী সমাধানের মধ্যে ১৭টিই XGBoost ব্যবহার করেছিল। এই সাফল্যের পেছনে দুইটি স্তম্ভ কাজ করে: (১) একটি গাণিতিকভাবে অধিক সুনির্দিষ্ট (Regularized) Objective Function, এবং (২) সিস্টেম-স্তরের একাধিক প্রকৌশলগত অপটিমাইজেশন যা একে ব্যাপক দ্রুতগতির করে তোলে।

২১.২ Gradient Boosting থেকে XGBoost — মূল পার্থক্য কোথায়?

Chapter 19-এ আমরা দেখেছি সাধারণ Gradient Boosting প্রতিটি নতুন গাছকে Residual বা Negative Gradient শেখানোর মাধ্যমে ট্রেন করে, এবং একটি সাধারণ Greedy পদ্ধতিতে (যেমন MSE বা Gini হ্রাস) সেরা Split খুঁজে বের করে। XGBoost এই প্রক্রিয়াটিকে একটি আনুষ্ঠানিক Optimization সমস্যা হিসেবে পুনর্গঠন করে, যেখানে:

- Loss Function-এর কেবল প্রথম Derivative (Gradient) নয়, বরং দ্বিতীয় Derivative-ও (Hessian) ব্যবহার করা হয়, যা Optimization-কে অনেক বেশি নিখুঁত করে তোলে।
- মডেলের জটিলতার (গাছের সংখ্যা, গাছের গঠন, Leaf-এর মান) ওপর একটি স্পষ্ট, গাণিতিক Regularization Penalty যুক্ত থাকে, যা Objective Function-এর অংশ হিসেবেই অন্তর্ভুক্ত।
- Split খোঁজা, Missing Value হ্যান্ডলিং, মেমোরি ব্যবস্থাপনা এবং Parallelization — প্রতিটি ধাপেই সিস্টেম-স্তরে অপটিমাইজেশন প্রয়োগ করা হয়।

২১.৩ গাণিতিক ভিত্তি — Regularized Objective Function

২১.৩.১ সম্পূর্ণ Objective Function

XGBoost-এর t -তম Iteration-এ Objective Function গঠিত হয় দুইটি অংশ নিয়ে — একটি Loss অংশ (মডেল কতটা ভুল করছে) এবং একটি Regularization অংশ (মডেল কতটা জটিল):

$$\mathcal{L}^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

এখানে f_t হলো t -তম নতুন গাছ, এবং $\Omega(f_t)$ হলো সেই গাছের জটিলতার ওপর Penalty। লক্ষণীয়, এটি Chapter 19-এর মূল Additive Model $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$ -এর একই কাঠামো অনুসরণ করে, শুধু এখানে একটি Regularization টার্ম Ω সরাসরি Objective-এর অংশ করে দেওয়া হয়েছে।

২১.৩.২ Regularization টার্ম — গাছের জটিলতা পরিমাপ

একটি গাছ f_t -কে T টি Leaf বিশিষ্ট এবং প্রতিটি Leaf-এর একটি নির্দিষ্ট আউটপুট মান w (Leaf Weight) দ্বারা চিহ্নিত করা হয়। Regularization টার্মটি হলো:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

এখানে γ (Gamma) প্রতিটি অতিরিক্ত Leaf তৈরির জন্য একটি নির্দিষ্ট “মূল্য” ধার্য করে — অর্থাৎ একটি নতুন Split তখনই লাভজনক বিবেচিত হবে যখন তার Gain এই মূল্যের চেয়ে বেশি হয়, ফলে অপয়োজনীয় জটিল গাছ তৈরি রোধ হয়। আর λ (Lambda) হলো L2 Regularization, যা Leaf-এর মানগুলোকে অতিরিক্ত চরম (Extreme) হতে বাধা দেয়, ঠিক যেভাবে Ridge Regression-এ Coefficient-এর ওপর L2 Penalty কাজ করে। এই দুইটি টার্ম একত্রে XGBoost-কে একটি Decision Tree-ভিত্তিক Regularized Model-এ পরিণত করে, যা একে অতিরিক্ত জটিল ও Overfit-প্রবণ গাছ তৈরি করা থেকে বিরত রাখে।

২১.৩.৩ দ্বিতীয়-ক্রম Taylor Expansion — Gradient ও Hessian

প্রতিটি নতুন Iteration-এ Objective Function-টি সরাসরি Optimize করা কঠিন, কারণ $f_t(x_i)$ একটি Decision Tree, কোনো সহজ গাণিতিক ফাংশন নয়। এই সমস্যার সমাধানে XGBoost Loss Function-টিকে দ্বিতীয়-ক্রম Taylor Series দিয়ে আসন্নীকরণ (Approximate) করে $\hat{y}^{(t-1)}$ -এর চারপাশে:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

যেখানে g_i হলো প্রথম-ক্রম Gradient এবং h_i হলো দ্বিতীয়-ক্রম Hessian:

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$$

এখানে g_i -এর ভূমিকা ঠিক Chapter 19-এর Pseudo-Residual-এর মতোই — এটা বলে দেয় Loss কোন দিকে সবচেয়ে বেশি কমবে। কিন্তু XGBoost অতিরিক্তভাবে h_i (Hessian, অর্থাৎ Loss কার্ভের বক্রতা বা Curvature) ব্যবহার করে, যা প্রতিটি ধাপের “আত্মবিশ্বাস” (Confidence) পরিমাপ করে — Loss কার্ভ যেখানে বেশি বাঁকানো (বড় Hessian), সেখানে ছোট পদক্ষেপ নেওয়া উচিত, আর যেখানে সমতল (ছোট Hessian), সেখানে বড় পদক্ষেপ নিরাপদ। এই দ্বিতীয়-ক্রম তথ্য ব্যবহার করার কারণেই XGBoost সাধারণ Gradient Boosting-এর তুলনায় দ্রুততর ও অধিক নিখুঁতভাবে Optimal সমাধানের দিকে এগোতে পারে — এটি অনেকটা Newton’s Method-এর মতো আচরণ করে, যেখানে সাধারণ Gradient Descent শুধু ঢাল অনুসরণ করে, Newton’s Method ঢাল ও বক্রতা উভয়ই বিবেচনা করে অনেক বেশি সরাসরি পথে Minimum-এর দিকে এগোয়।

Regression-এর একটি সরল উদাহরণ: যদি Loss = Squared Error, $L(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$ হয়, তাহলে সরল ক্যালকুলাসে $g_i = -2(y_i - \hat{y}_i)$ এবং $h_i = 2$ — অর্থাৎ Hessian একটি ধ্রুবক। এই বিশেষ ক্ষেত্রে XGBoost-এর আচরণ সাধারণ Gradient Boosting-এর অনেক কাছাকাছি চলে আসে, কিন্তু Log-Loss-এর মতো জটিল Loss Function-এ (যেখানে Hessian Sample-ভেদে পরিবর্তিত হয়) দ্বিতীয়-ক্রম তথ্যের সুবিধা অনেক বেশি স্পষ্ট হয়ে ওঠে।

২১.৩.৪ Optimal Leaf Weight নির্ণয়

একটি নির্দিষ্ট গাছের গঠন (Structure) স্থির ধরে নিয়ে, প্রতিটি Leaf j -এ কোন Sample-গুলো পড়ছে তার সেটকে I_j বলা হয়। Objective Function-কে Leaf-ভিত্তিক আকারে পুনর্লিখন করলে দাঁড়ায়:

$$\mathcal{L}^{(t)} \approx \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

এটি w_j -এর সাপেক্ষে একটি সরল Quadratic (দ্বিঘাত) সমীকরণ, যার ন্যূনতম মান ক্যালকুলাসের সাহায্যে সরাসরি বের করা যায় ($\partial \mathcal{L} / \partial w_j = 0$ ধরে সমাধান করে)। ধরা যাক $G_j = \sum_{i \in I_j} g_i$ এবং $H_j = \sum_{i \in I_j} h_i$ — তাহলে প্রতিটি Leaf-এর সর্বোত্তম (Optimal) মান হলো:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

এবং এই সর্বোত্তম মান বসালে Objective Function-এর সর্বনিম্ন মান দাঁড়ায়:

$$\mathcal{L}^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

এই $\mathcal{L}^*$ -কে বলা হয় Structure Score — এটি একটি গাছের গঠন কতটা ভালো, তা একটিমাত্র সংখ্যা দিয়ে পরিমাপ করে (অনেকটা Decision Tree-তে Gini Impurity বা Entropy যেভাবে একটি Split-এর গুণমান পরিমাপ করে, ঠিক সেভাবেই)। মান যত ছোট (ঋণাত্মক দিকে যত বড়), গাছের গঠন তত ভালো।

২১.৩.৫ Split Gain — কোন Split সবচেয়ে ভালো?

একটি Leaf-কে দুইটি Child (Left ও Right)-এ ভাগ করার লাভ (Gain) নির্ণয় করা হয় ভাগ করার আগে ও পরের Structure Score-এর পার্থক্য হিসেবে:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

এই সূত্রটিই Chapter 20-এ উল্লেখিত HistGradientBoosting-এর Gain সূত্রের সাথে হুবহু মিলে যায় — কারণ উভয় অ্যালগরিদমই একই Regularized, দ্বিতীয়-ক্রম Objective Function-এর ওপর ভিত্তি করে তৈরি। সূত্রের প্রথম দুইটি টার্ম যথাক্রমে Left ও Right Child-এর “স্কোর”কে প্রতিনিধিত্ব করে, তৃতীয় টার্মটি ভাগ না করলে (Parent হিসেবে থাকলে) যে স্কোর হতো তা বিয়োগ করে, এবং সবশেষে γ বিয়োগ করে একটি নতুন Leaf তৈরির “খরচ” যোগ করা হয়। যদি Gain ঋণাত্মক হয়, তার মানে হলো এই Split করলে লাভের চেয়ে জটিলতার খরচ বেশি — তাই XGBoost সেই Split বাতিল করে দেয়। এই বৈশিষ্ট্যটিকে বলা হয় Pruning (ছাঁটাই), এবং এটি XGBoost-কে স্বয়ংক্রিয়ভাবে অতিরিক্ত জটিল গাছ তৈরি করা থেকে বিরত রাখে।

২১.৪ Sparsity-Aware Split Finding — Missing Value-এর জন্য স্বয়ংক্রিয় দিকনির্দেশনা

বাস্তব জগতের ডেটাসেটে Missing Value (NaN) বা Sparse Feature (যেমন One-Hot Encoded কলামে বেশিরভাগ মান শূন্য) অত্যন্ত সাধারণ। XGBoost প্রতিটি Split-এর জন্য একটি Default Direction শেখে — অর্থাৎ কোনো Sample-এর নির্দিষ্ট Feature-এ যদি মান অনুপস্থিত থাকে, তাহলে সে Left অথবা Right, কোন Child-এ যাবে তা প্রশিক্ষণের সময় স্বয়ংক্রিয়ভাবে নির্ধারিত হয়। এই দিকনির্দেশনা নির্বাচন করা হয় যেভাবে Gain সর্বোচ্চ হয় সেই দিক বেছে নিয়ে — অর্থাৎ Missing Value-থাকা Sample-গুলোকে প্রথমে সম্পূর্ণভাবে Left-এ, তারপর সম্পূর্ণভাবে Right-এ বসিয়ে দুইবার Gain হিসাব করা হয়, এবং যেকোনো Gain বেশি, সেই দিকটাই চূড়ান্তভাবে বেছে নেওয়া হয়। এর ফলে Missing Value-এর জন্য আলাদা কোনো Imputation পদ্ধতি (Mean, Median, ইত্যাদি) প্রয়োগ করার প্রয়োজন পড়ে না, এবং একই কৌশল Sparse ডেটার ক্ষেত্রেও কম্পিউটেশনাল গতি বাড়িয়ে দেয় — কারণ কেবলমাত্র Non-Missing মানগুলোর ওপরই Split খোঁজার হিসাব করা হয়।

২১.৫ Split খোঁজার অ্যালগরিদম — Exact vs Approximate

২১.৫.১ Exact Greedy Algorithm

ছোট বা মাঝারি ডেটাসেটে XGBoost প্রতিটি Feature-এর প্রতিটি সম্ভাব্য Threshold পরীক্ষা করে সর্বোচ্চ Gain-এর Split খুঁজে বের করে — এটি Chapter 19-এর সাধারণ Gradient Boosting-এর Split-Finding পদ্ধতির মতোই, তবে Gain হিসাব হয় ওপরের Regularized সূত্র দিয়ে।

২১.৫.২ Approximate Algorithm ও Weighted Quantile Sketch

বড় ডেটাসেটে প্রতিটি Unique মান পরীক্ষা করা ব্যয়বহুল হয়ে ওঠে (ঠিক যে সমস্যাটি Chapter 20-এ HistGradientBoosting সমাধান করেছিল)। XGBoost-এর Approximate Algorithm প্রতিটি Feature-এর মানগুলোকে Percentile অনুযায়ী কয়েকটি Candidate Split Point-এ ভাগ করে, এবং শুধুমাত্র সেই Candidate Point-গুলোতেই Gain হিসাব করে। এই Percentile নির্ধারণে ব্যবহৃত হয় একটি বিশেষ ডেটা স্ট্রাকচার — Weighted Quantile Sketch — যা প্রতিটি Sample-এর Hessian-কে “ওজন” হিসেবে বিবেচনা করে Percentile নির্ণয় করে, যাতে Candidate Point-গুলো Loss কমানোর দিক থেকে সবচেয়ে তথ্যবহুল জায়গায় বসে। এই কৌশলটি ধারণাগতভাবে HistGradientBoosting-এর Binning-এর সাথে সাদৃশ্যপূর্ণ, যদিও বাস্তবায়নের খুঁটিনাটি ভিন্ন।

২১.৬ সিস্টেম-স্তরের অপ্টিমাইজেশন — কেন এটি এত দ্রুত?

XGBoost-এর গতি শুধু গাণিতিক কৌশলের ফল নয়, বরং প্রকৌশলগত (Systems Engineering) অপ্টিমাইজেশনের ফলও বটে।

Column Block ও Cache-Aware Access: ডেটা একবার Column-ভিত্তিক Compressed Block আকারে সাজিয়ে রাখা হয় এবং Pre-sorted রাখা হয়, যাতে একাধিক গাছ তৈরির সময় বারবার Sort করার প্রয়োজন না পড়ে। এছাড়া প্রতিটি Thread-এর জন্য আলাদা Internal Buffer ব্যবহার করে Cache Miss কমানো হয়, যা CPU-র গতি সর্বোচ্চ ব্যবহার নিশ্চিত করে।

Parallel Computation: যদিও গাছ তৈরির প্রক্রিয়া (Boosting Iteration) সহজাতভাবে Sequential, তবে প্রতিটি গাছের ভেতরে সেরা Split খোঁজার হিসাব (বিভিন্ন Feature-এর ওপর Gain গণনা) সম্পূর্ণভাবে Parallelize করা যায়। XGBoost এই সুযোগটি কাজে লাগিয়ে একাধিক CPU Core-এ একসাথে Split-Finding সম্পন্ন করে।

Out-of-Core Computation: ডেটাসেট RAM-এর চেয়ে বড় হলে, XGBoost ডেটাকে ডিস্কে Compressed আকারে সংরক্ষণ করে এবং প্রয়োজন অনুযায়ী ব্লক আকারে পড়ে, যাতে সীমিত মেমোরিতেও বিশাল ডেটাসেটে ট্রেনিং সম্ভব হয়।

Regularization ও Shrinkage-এর সম্মিলিত ব্যবহার: Learning Rate (Shrinkage) এবং Column Subsampling (নিচে বর্ণিত) একত্রে ব্যবহার করে XGBoost Random Forest-এর Randomness এবং Gradient Boosting-এর Sequential Correction — উভয়ের সুবিধা একসাথে গ্রহণ করে।

২১.৭ গুরুত্বপূর্ণ Hyperparameter — বিস্তারিত ব্যাখ্যা

n_estimators — Default: 100

কাজ: মোট কতগুলো Boosting Round বা গাছ তৈরি হবে। Chapter 19-এর মতোই, learning_rate ছোট রাখলে n_estimators বাড়ানো প্রয়োজন। বাস্তবে early_stopping_rounds ব্যবহার করে Validation Set-এর ওপর Performance স্থির (Plateau) হয়ে গেলে ট্রেনিং স্বয়ংক্রিয়ভাবে থামিয়ে দেওয়াই সবচেয়ে নিরাপদ পদ্ধতি।

learning_rate (eta) — Default: 0.3

কাজ: প্রতিটি নতুন গাছের Leaf Weight-কে কতটা সংকুচিত করে যোগ করা হবে। লক্ষণীয়, XGBoost-এর Default মান (০.৩) সাধারণ Scikit-Learn GradientBoosting-এর Default (০.১)-এর চেয়ে বড়, তবে ব্যবহারিক প্রয়োগে প্রায়শই ০.০১ থেকে ০.১ রেঞ্জে নামিয়ে এনে n_estimators বাড়ানো হয়, যা অধিক স্থিতিশীল ফলাফল দেয়।

max_depth — Default: 6

কাজ: প্রতিটি গাছের সর্বোচ্চ গভীরতা। XGBoost-এর Default (৬) সাধারণ GradientBoosting-এর Default (৩)-এর চেয়ে গভীর, কারণ λ ও γ -এর Regularization ইতিমধ্যেই Overfitting কিছুটা নিয়ন্ত্রণ করে, তাই গাছকে কিছুটা বেশি জটিল হতে দেওয়া নিরাপদ।

gamma (min_split_loss) — Default: 0

কাজ: একটি Split গ্রহণযোগ্য হওয়ার জন্য ন্যূনতম কত Gain প্রয়োজন, তা নির্ধারণ করে (ওপরের Gain সূত্রের γ)। বড় করলে মডেল আরও রক্ষণশীল (Conservative) হয় এবং কম Split তৈরি হয়।

reg_lambda (L2) ও reg_alpha (L1) — Default: 1 এবং 0

কাজ: reg_lambda হলো Leaf Weight-এর ওপর L2 Regularization (ওপরের সূত্রের λ), যা Weight-গুলোকে ছোট ও স্থিতিশীল রাখে। reg_alpha হলো L1 Regularization, যা কিছু Leaf

Weight-কে সম্পূর্ণ শূন্যে নামিয়ে দিতে পারে, ফলে গাছ কিছুটা Sparse হয়ে ওঠে — এটি Chapter Naive Bayes-পরবর্তী অংশে আলোচিত L1/L2 Regularization-এর একই নীতি, কেবল প্রয়োগ করা হচ্ছে Leaf Weight-এর ওপর, Linear Coefficient-এর ওপর নয়।

subsample ও colsample_bytree — Default: 1.0 এবং 1.0

কাজ: subsample প্রতিটি গাছের জন্য Row-এর কত অংশ Randomly ব্যবহার হবে তা নিয়ন্ত্রণ করে (Chapter 19-এর Stochastic Gradient Boosting-এর মতোই), আর colsample_bytree প্রতিটি গাছের জন্য কত অংশ Column বা Feature Randomly ব্যবহার হবে তা নিয়ন্ত্রণ করে — এটি Random Forest-এর Feature Bagging কৌশল থেকে ধার করা। দুইটি একসাথে ১.০-এর কম রাখলে (যেমন ০.৮) প্রতিটি গাছ ভিন্ন Random উপসেট দেখে, যা Overfitting উল্লেখযোগ্যভাবে কমায়।

min_child_weight — Default: 1

কাজ: একটি Leaf তৈরি হওয়ার জন্য সেই Leaf-এ থাকা Sample-গুলোর Hessian-এর ন্যূনতম যোগফল (H_j) কত হতে হবে তা নির্ধারণ করে — এটি Chapter 19-এর min_samples_leaf-এর একটি Generalized রূপ (কারণ Regression-এ Hessian = ধ্রুবক হওয়ায় এটি কার্যত Sample সংখ্যার সমতুল্য হয়ে যায়, কিন্তু Classification-এ এটি প্রতিটি Sample-এর “গুরুত্ব” বিবেচনা করে)। বড় করলে Leaf-এ বেশি Sample প্রয়োজন হয়, যা Overfitting কমায়।

Parameter *	*Default**	*সংক্ষিপ্ত কাজ**
objective	নির্ভরশীল ক	কোন Loss Function ব্যবহৃত হবে (যেমন reg:squarederror, binary:logistic)
tree_method	auto	Split-Finding পদ্ধতি (exact, approx, hist, gpu_hist)
scale_pos_weight	1	Class Imbalance সামলানোর জন্য Positive Class-এর ওজন বাড়ানো
booster	gbtree	কোন ধরনের Base Learner ব্যবহৃত হবে (gbtree, gblinear, dart)

২১.৮ গুরুত্বপূর্ণ Attribute ও মেথড

- **feature_importances_** — প্রতিটি Feature কতবার Split-এ ব্যবহৃত হয়েছে (Weight), গড়ে কতটা Gain এনেছে (Gain), বা কতগুলো Sample-কে প্রভাবিত করেছে (Cover) — এই তিনটি ভিন্ন মানদণ্ডে পরিমাপযোগ্য।
- **best_iteration / best_score** — early_stopping_rounds ব্যবহার করলে সবচেয়ে ভালো Validation Score কোন Iteration-এ পাওয়া গিয়েছিল তার তথ্য।
- **plot_importance() / plot_tree()** — Feature Importance এবং পৃথক গাছের গঠন Visualize করার সহায়ক ফাংশন।

২১.৯ XGBoost, GradientBoosting এবং HistGradientBoosting — সম্পূর্ণ তুলনা

বিষয় *	*GradientBoosting* **	*HistGradientBoosting* **	*XGBoost**
Derivative ব্যবহার	শুধু প্রথম-ক্রম (Gradient) প্র	থম ও দ্বিতীয়-ক্রম (Gradient+Hessian)	প্রথম ও দ্বিতীয়-ক্রম (Gradient+Hessian)
Regularization	নেই (Implicit, শুধু Tree Depth/Leaf-এর মাধ্যমে)	l2_regularization (শুধু L2) 0	Objective-এ সরাসরি γ (Leaf Penalty) ও λ (L2), এবং alpha (L1)
Split খোঁজা	প্রতিটি Unique মান	Histogram/Binning (সর্বোচ্চ ২৫৫ Bin)	Exact অথবা Approximate (Weighted Quantile Sketch)
Missing Value	Manual Imputation দরকার	স্বয়ংক্রিয় হ্যান্ডলিং	Sparsity-Aware, স্বয়ংক্রিয় Default Direction
Parallelization	সীমিত	Histogram Building Parallel	Split-Finding, Column Block — ব্যাপক Parallel
গতি (বেড় ডেটাসেটে)	ধীর	দ্রুত	দ্রুত (Histogram/GPU Mode-এ HistGB-এর কাছাকাছি)
বাস্তবায়ন	Scikit-Learn Built-in	Scikit-Learn Built-in	পৃথক লাইব্রেরি (pip install xgboost)
GPU সাপোর্ট	নেই	সীমিত	পূর্ণাঙ্গ (tree_method='gpu_hist')

২১.১০ কেন এত জনপ্রিয়? (Advantages)

- **সর্বোচ্চ নির্ভুলতা:** দ্বিতীয়-ক্রম Optimization এবং সুনির্দিষ্ট Regularization-এর সমন্বয়ে প্রায়শই সাধারণ Gradient Boosting বা এমনকি অনেক Deep Learning মডেলের চেয়েও ভালো Accuracy দেয়, বিশেষত Tabular ডেটাসেটে।
- **Overfitting নিয়ন্ত্রণে সুনির্দিষ্ট গাণিতিক হাতিয়ার:** γ, λ, α — এই তিনটি Regularization প্যারামিটার Objective Function-এর মধ্যেই অন্তর্ভুক্ত থাকায় Overfitting নিয়ন্ত্রণ অনেক বেশি নিয়মতান্ত্রিক।
- **Missing Value ও Sparse ডেটার স্বয়ংক্রিয় হ্যান্ডলিং:** পূর্ব-প্রক্রিয়াকরণের সময় ও শ্রম উল্লেখযোগ্যভাবে কমে।
- **সিস্টেম-স্তরের গতি:** Cache-Aware Access, Column Block, Parallel Computation ও GPU সাপোর্ট একত্রে একে বাস্তবিক অর্থে অত্যন্ত দ্রুতগতির করে তোলে।
- **বিস্তৃত ভাষা ও প্ল্যাটফর্ম সাপোর্ট:** Python, R, Java, Scala, C++ — একাধিক ভাষায় ব্যবহারযোগ্য, এবং Spark-এর মতো Distributed সিস্টেমের সাথেও একত্রিত করা যায়।

২১.১১ সীমাবদ্ধতা (Disadvantages)

- **অতিরিক্ত Hyperparameter:** সাধারণ Gradient Boosting-এর তুলনায় অনেক বেশি Hyperparameter (max_depth, learning_rate, gamma, lambda, alpha, subsample, colsample_bytree, min_child_weight ইত্যাদি) থাকায় সঠিক Tuning তুলনামূলক জটিল।
- **Interpretability-এর ঘাটতি:** একক Decision Tree বা এমনকি Random Forest-এর তুলনায় শত শত ছোট গাছের সম্মিলিত সিদ্ধান্ত ব্যাখ্যা করা কঠিন, যদিও SHAP-এর মতো Post-hoc পদ্ধতি এই সমস্যা কিছুটা লাঘব করে।
- **ছোট ডেটাসেটে বাড়তি জটিলতা অপ্রয়োজনীয়:** ছোট, সরল ডেটাসেটে XGBoost-এর অতিরিক্ত Regularization ও জটিলতা থেকে খুব একটা বাড়তি সুবিধা পাওয়া যায় না; সাধারণ Gradient Boosting বা এমনকি Linear Model যথেষ্ট হতে পারে।
- **পৃথক ইনস্টলেশন প্রয়োজন:** Scikit-Learn-এর মতো Built-in নয়, তাই আলাদাভাবে ইনস্টল ও কখনো কখনো সিস্টেম-নির্দিষ্ট Dependency ব্যবস্থাপনা প্রয়োজন হতে পারে।

২১.১২ শিল্পখাতে ব্যবহার

- **ফাইন্যান্স ও ফিনটেক:** Credit Risk Scoring, Fraud Detection, Algorithmic Trading — যেখানে উচ্চ Accuracy ও দ্রুত Inference উভয়ই গুরুত্বপূর্ণ।
- **ই-কমার্স ও বিজ্ঞাপন:** Click-Through Rate (CTR) প্রেডিকশন, Recommendation Ranking, Customer Churn প্রেডিকশন — বৃহৎ প্রযুক্তি প্রতিষ্ঠানগুলোর Production সিস্টেমে ব্যাপকভাবে ব্যবহৃত।
- **স্বাস্থ্যসেবা ও বায়োইনফরমেটিক্স:** জটিল, উচ্চ-মাত্রিক Structured ডেটা থেকে রোগ নির্ণয় ও ঝুঁকি স্কোরিং মডেল তৈরিতে।
- **প্রতিযোগিতামূলক ডেটা সায়েন্স:** Kaggle ও অন্যান্য প্ল্যাটফর্মে Tabular ডেটার জন্য বহু বছর ধরে সবচেয়ে নির্ভরযোগ্য ও প্রথম পছন্দের Baseline মডেল।

২১.১৩ প্রশ্নোত্তর

প্রশ্ন: XGBoost, HistGradientBoosting-এর চেয়ে গাণিতিকভাবে ভিন্ন কীভাবে? **উত্তর:** কেন্দ্রীয় ধারণাগতভাবে দুইটিই প্রায় একই — উভয়ই Gradient ও Hessian ব্যবহার করে Regularized Split Gain হিসাব করে। মূল পার্থক্য মূলত বাস্তবায়নে (Implementation): XGBoost-এ Regularization প্যারামিটার (γ, λ, α) স্পষ্টভাবে Objective Function-এ সংজ্ঞায়িত ও Configurable, Sparsity-Aware Split-Finding স্বতন্ত্রভাবে ডিজাইন করা, এবং একাধিক Split-Finding মোড (Exact/Approx/GPU) উপলব্ধ — যেখানে HistGradientBoosting মূলত একটিমাত্র দ্রুতগতির Histogram-ভিত্তিক পদ্ধতিতে সীমাবদ্ধ।

প্রশ্ন: XGBoost কি সবসময় ব্যবহার করা উচিত? **উত্তর:** না। ছোট ডেটাসেট, সরল Linear সম্পর্ক, বা যেখানে Interpretability সবচেয়ে গুরুত্বপূর্ণ, সেখানে সরল মডেল (Linear/Logistic Regression, বা এমনকি একটি একক Decision Tree) বেশি উপযোগী হতে পারে। XGBoost সবচেয়ে বেশি মূল্য যোগ করে যখন ডেটাসেট মাঝারি থেকে বড়, Feature-দের মধ্যে জটিল Non-linear সম্পর্ক আছে, এবং সর্বোচ্চ Predictive Accuracy-ই মূল লক্ষ্য।

প্রশ্ন: Hessian ব্যবহার করার প্রকৃত সুবিধা কী? উত্তর: Hessian Loss Function-এর বক্রতা সম্পর্কে অতিরিক্ত তথ্য দেয়, যা প্রতিটি Split ও Leaf Weight নির্ণয়কে আরও নিখুঁত করে তোলে। এর ফলে XGBoost কম সংখ্যক Iteration-এই Optimal সমাধানের কাছাকাছি পৌঁছাতে পারে, বিশেষত জটিল Loss Function (যেমন Log-Loss)-এর ক্ষেত্রে, যেখানে Curvature Sample-ভেদে ব্যাপকভাবে পরিবর্তিত হয়।

২১.১৫ সম্পূর্ণ গাণিতিক বাস্তবায়ন – হাতে-কলমে সম্পূর্ণ ধাপে ধাপে গণনা

২১.১৫.১ ভূমিকা

এই বিভাগে আমরা XGBoost-এর সম্পূর্ণ গাণিতিক প্রক্রিয়া — Gradient, Hessian, Regularized Gain, এবং Optimal Leaf Weight — একই পরিচিত ৪-ছত্রের ডেটাসেটে, সম্পূর্ণ ৫টি Iteration ধরে, প্রতিটি সংখ্যা হাতে-কলমে গণনা করে দেখাব। Chapter 19-এর সাধারণ Gradient Boosting-এর সাথে তুলনা করলে পার্থক্যটা স্পষ্ট হবে: এখানে প্রতিটি Split শুধু Gradient নয়, Hessian-ও ব্যবহার করে নির্ণয় করা হয়, এবং Leaf Weight-এর সূত্রে একটি Regularization টার্ম (λ) যুক্ত থাকে।

২১.১৫.২ ছোট ডেটাসেট ও সমস্যা বিবৃতি

ID	Study Hours (X) *	*প্রকৃত Marks (y)**
1	1	40
2	3	55
3	5	85
4	7	100

Loss Function: $L(y, F) = (y - F)^2$ (Squared Error, Chapter 19-এর মতোই তুলনার সুবিধার্থে)।

২১.১৫.৩ ইনিশিয়ালাইজেশন – প্রতিটি প্যারামিটার ও হাইপারপ্যারামিটার

প্যারামিটার *	*মান**
n_estimators	5
learning_rate (η)	0.3 (XGBoost Default)
reg_lambda (λ , L2)	1.0
gamma (γ , Min Split Loss)	0.0
max_depth	1 (Decision Stump, তুলনার সুবিধার্থে)
Loss Function	Squared Error, $L = (y - F)^2$

২১.১৫.৪ প্রাথমিক প্রেডিকশন

Chapter 19-এর মতোই, F_0 হলো Squared Error সর্বনিম্নকারী Constant, অর্থাৎ গড় মান:

$$F_0 = \frac{40 + 55 + 85 + 100}{4} = \frac{280}{4} = 70$$

২১.১৫.৫ Gradient ও Hessian গণনার সূত্র – সম্পূর্ণ ডেরিভেশন

$L(y, F) = (y - F)^2$ -এর প্রথম ও দ্বিতীয় Derivative বের করা যাক, ধাপে ধাপে।

প্রথম Derivative (Gradient): ধরি $u = y - F$, তাহলে $L = u^2$ । চেইন রুল প্রয়োগ করে:

$$g = \frac{\partial L}{\partial F} = \frac{\partial L}{\partial u} \cdot \frac{\partial u}{\partial F} = 2u \cdot (-1) = -2(y - F) = 2(F - y)$$

দ্বিতীয় Derivative (Hessian): প্রথম Derivative $g = 2(F - y) = 2F - 2y$ -কে আবার F -এর সাপেক্ষে Differentiate করি:

$$h = \frac{\partial g}{\partial F} = \frac{\partial}{\partial F} (2F - 2y) = 2$$

লক্ষণীয়, Squared Error Loss-এর জন্য Hessian সবসময় একটি ধ্রুবক (2), Sample-নির্বিশেষে। এই বিশেষ ধর্মের কারণেই XGBoost-এর আচরণ Squared Error-এ সাধারণ Gradient Boosting-এর অনেক কাছাকাছি থাকে (কারণ Hessian কোনো অতিরিক্ত তথ্য যোগ করছে না, শুধু একটি Scaling Factor হিসেবে কাজ করছে)। Log-Loss-এর মতো Loss Function-এ Hessian Sample-ভেদে পরিবর্তিত হয়, তখন এই দ্বিতীয়-ক্রম তথ্যের প্রকৃত সুবিধা দেখা যায়।

২১.১৫.৬ Iteration 1 – সম্পূর্ণ গণনা

ধাপ ১ – Gradient গণনা: $g_i = 2(F_0 - y_i)$

$$g_1 = 2(70 - 40) = 2 \times 30 = 60$$

$$g_2 = 2(70 - 55) = 2 \times 15 = 30$$

$$g_3 = 2(70 - 85) = 2 \times (-15) = -30$$

$$g_4 = 2(70 - 100) = 2 \times (-30) = -60$$

ধাপ ২ – Hessian গণনা: প্রতিটি Sample-এর জন্য $h_i = 2$ (ধ্রুবক): $h = [2, 2, 2, 2]$ ।

ধাপ ৩ – প্রতিটি সম্ভাব্য Split-এর Gain গণনা। প্রথমে মোট $G = \sum g_i = 60 + 30 - 30 - 60 = 0$ এবং $H = \sum h_i = 8$ ।

Threshold=2 (Split: $X \leq 2$ বনাম $X > 2$):

বাম দল {ID1}: $G_L = 60, H_L = 2$ । ডান দল {ID2,3,4}: $G_R = 30 - 30 - 60 = -60, H_R = 6$ ।

$$\begin{aligned} \text{Gain} &= \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma \\ &= \frac{1}{2} \left[\frac{60^2}{2 + 1} + \frac{(-60)^2}{6 + 1} - \frac{0^2}{8 + 1} \right] - 0 = \frac{1}{2} \left[\frac{3600}{3} + \frac{3600}{7} - 0 \right] \\ &= \frac{1}{2} [1200 + 514.2857] = \frac{1}{2} \times 1714.2857 = 857.1429 \end{aligned}$$

Threshold=4 (Split: $X \leq 4$ বনাম $X > 4$):

বাম দল {ID1,2}: $G_L = 60 + 30 = 90$, $H_L = 4$ | ডান দল {ID3,4}: $G_R = -30 - 60 = -90$, $H_R = 4$ |

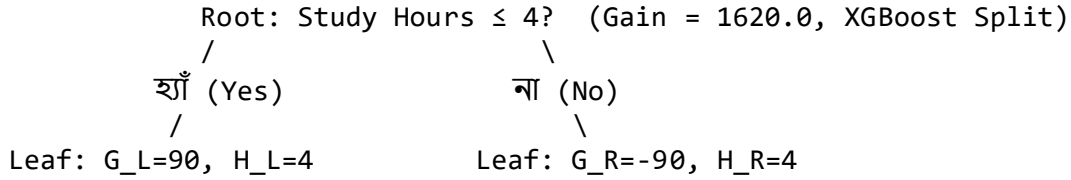
$$\text{Gain} = \frac{1}{2} \left[\frac{90^2}{4+1} + \frac{(-90)^2}{4+1} - 0 \right] = \frac{1}{2} \left[\frac{8100}{5} + \frac{8100}{5} \right] = \frac{1}{2} [1620 + 1620] = \frac{1}{2} \times 3240 = 1620.0$$

Threshold=6 (Split: $X \leq 6$ বনাম $X > 6$):

বাম দল {ID1,2,3}: $G_L = 60 + 30 - 30 = 60$, $H_L = 6$ | ডান দল {ID4}: $G_R = -60$, $H_R = 2$ |

$$\text{Gain} = \frac{1}{2} \left[\frac{60^2}{6+1} + \frac{(-60)^2}{2+1} - 0 \right] = \frac{1}{2} \left[\frac{3600}{7} + \frac{3600}{3} \right] = \frac{1}{2} [514.2857 + 1200] = 857.1429$$

সিদ্ধান্ত ও কারণ: তিনটি প্রার্থীর মধ্যে Threshold=4-এর Gain (১৬২০.০) সর্বোচ্চ, তাই এটিই নির্বাচিত হলো — ঠিক Chapter 19-এর SSE-ভিত্তিক সিদ্ধান্তের সাথে সামঞ্জস্যপূর্ণ, কিন্তু এখানে সিদ্ধান্তটি Regularized, দ্বিতীয়-ক্রম Gain সূত্র দিয়ে নেওয়া হয়েছে।



ধাপ ৪ — Optimal Leaf Weight গণনা:

$$w_L^* = -\frac{G_L}{H_L + \lambda} = -\frac{90}{4+1} = -\frac{90}{5} = -18.0$$

$$w_R^* = -\frac{G_R}{H_R + \lambda} = -\frac{-90}{4+1} = \frac{90}{5} = 18.0$$

ধাপ ৫ — প্রেডিকশন আপডেট: $F_1(x) = F_0(x) + \eta \cdot h_1(x)$, যেখানে $\eta = 0.3$ |

$$F_1(\text{ID } 1,2) = 70 + 0.3 \times (-18.0) = 70 - 5.4 = 64.6$$

$$F_1(\text{ID } 3,4) = 70 + 0.3 \times 18.0 = 70 + 5.4 = 75.4$$

ধাপ ৬ — Loss গণনা:

$$(40 - 64.6)^2 = 605.16, (55 - 64.6)^2 = 92.16, (85 - 75.4)^2 = 92.16, (100 - 75.4)^2 = 605.16$$

$$L_1 = \frac{605.16 + 92.16 + 92.16 + 605.16}{4} = \frac{1394.64}{4} = 348.66$$

২১.১৫.৭ Iteration 2 — সম্পূর্ণ গণনা

Gradient: $g_i = 2(F_1 - y_i)$

$$g_1 = 2(64.6 - 40) = 2 \times 24.6 = 49.2, \quad g_2 = 2(64.6 - 55) = 2 \times 9.6 = 19.2$$

$$g_3 = 2(75.4 - 85) = 2 \times (-9.6) = -19.2, \quad g_4 = 2(75.4 - 100) = 2 \times (-24.6) = -49.2$$

Split যাচাই (Threshold=4): $G_L = 49.2 + 19.2 = 68.4, H_L = 4; G_R = -19.2 - 49.2 = -68.4, H_R = 4$

$$\text{Gain}(c = 4) = \frac{1}{2} \left[\frac{68.4^2}{5} + \frac{(-68.4)^2}{5} \right] = \frac{1}{2} \left[\frac{4678.56}{5} \times 2 \right] = \frac{1}{2} \times 1871.424 = 935.712$$

(Threshold=2 ও 6-এর Gain যাচাই করলে যথাক্রমে ৫৭৬.৩৪ পাওয়া যায়, যা ৯৩৫.৭১২-এর চেয়ে কম — তাই Threshold=4 আবারও নির্বাচিত হলো।)

Leaf Weight:

$$w_L^* = -\frac{68.4}{4+1} = -13.68, \quad w_R^* = -\frac{-68.4}{4+1} = 13.68$$

প্রেডিকশন আপডেট:

$$F_2(\text{ID1}, 2) = 64.6 + 0.3 \times (-13.68) = 64.6 - 4.104 = 60.496$$

$$F_2(\text{ID3}, 4) = 75.4 + 0.3 \times 13.68 = 75.4 + 4.104 = 79.504$$

Loss:

$$(40 - 60.496)^2 = 420.117, \quad (55 - 60.496)^2 = 30.206, \quad (85 - 79.504)^2 = 30.206, \quad (100 - 79.504)^2 = 420.117$$

$$L_2 = \frac{420.117 + 30.206 + 30.206 + 420.117}{4} = \frac{900.646}{4} = 225.146$$

২১.১৫.৮ Iteration 3 — সম্পূর্ণ গণনা

Gradient: $g_i = 2(F_2 - y_i)$

$$g_1 = 2(60.496 - 40) = 40.992, \quad g_2 = 2(60.496 - 55) = 10.992$$

$$g_3 = 2(79.504 - 85) = -10.992, \quad g_4 = 2(79.504 - 100) = -40.992$$

Split (Threshold=4): $G_L = 40.992 + 10.992 = 51.984, G_R = -51.984, H_L = H_R = 4$

$$\text{Gain} = \frac{1}{2} \left[\frac{51.984^2}{5} \times 2 \right] = \frac{1}{2} \times \frac{2 \times 2702.336}{5} = \frac{1}{2} \times 1080.934 = 540.467$$

Leaf Weight: $w_L^* = -\frac{51.984}{5} = -10.3968, w_R^* = 10.3968$

প্রেডিকশন আপডেট:

$$F_3(\text{ID1},2) = 60.496 + 0.3 \times (-10.3968) = 60.496 - 3.11904 = 57.37696$$

$$F_3(\text{ID3},4) = 79.504 + 0.3 \times 10.3968 = 79.504 + 3.11904 = 82.62304$$

Loss:

$$(40 - 57.37696)^2 = 301.9887, (55 - 57.37696)^2 = 5.6516$$

$$(85 - 82.62304)^2 = 5.6516, (100 - 82.62304)^2 = 301.9887$$

$$L_3 = \frac{301.9887 + 5.6516 + 5.6516 + 301.9887}{4} = \frac{615.2806}{4} = 153.8043$$

২১.১৫.৯ Iteration 4 – সম্পূর্ণ গণনা

Gradient: $g_i = 2(F_3 - y_i)$

$$g_1 = 2(57.37696 - 40) = 34.75392, g_2 = 2(57.37696 - 55) = 4.75392$$

$$g_3 = 2(82.62304 - 85) = -4.75392, g_4 = 2(82.62304 - 100) = -34.75392$$

Split (Threshold=4): $G_L = 34.75392 + 4.75392 = 39.50784, G_R = -39.50784$

$$\text{Gain} = \frac{1}{2} \left[\frac{39.50784^2}{5} \times 2 \right] = \frac{1}{2} \times \frac{2 \times 1560.869}{5} = \frac{1}{2} \times 624.348 = 312.174$$

Leaf Weight: $w_L^* = -\frac{39.50784}{5} = -7.901568, w_R^* = 7.901568$

প্রেডিকশন আপডেট:

$$F_4(\text{ID1},2) = 57.37696 + 0.3 \times (-7.901568) = 57.37696 - 2.3704704 = 55.0064896$$

$$F_4(\text{ID3},4) = 82.62304 + 0.3 \times 7.901568 = 82.62304 + 2.3704704 = 84.9935104$$

Loss:

$$(40 - 55.0064896)^2 = 225.1948, (55 - 55.0064896)^2 = 0.0042$$

$$(85 - 84.9935104)^2 = 0.0042, (100 - 84.9935104)^2 = 225.1948$$

$$L_4 = \frac{225.1948 + 0.0042 + 0.0042 + 225.1948}{4} = \frac{450.398}{4} = 112.5974$$

২১.১৫.১০ Iteration 5 – সম্পূর্ণ গণনা (একটি গুরুত্বপূর্ণ পরিবর্তন লক্ষণীয়)

Gradient: $g_i = 2(F_4 - y_i)$

$$g_1 = 2(55.0064896 - 40) = 30.0129792$$

$$g_2 = 2(55.0064896 - 55) = 0.0129792$$

$$g_3 = 2(84.9935104 - 85) = -0.0129792$$

$$g_4 = 2(84.9935104 - 100) = -30.0129792$$

লক্ষণীয়, g_2 ও g_3 এখন প্রায় শূন্যের কাছাকাছি চলে এসেছে — এর অর্থ হলো ID 2 ও ID 3-এর প্রেডিকশন প্রায় নিখুঁত হয়ে গেছে, কিন্তু ID 1 ও ID 4-এর এখনো উল্লেখযোগ্য ভুল অবশিষ্ট আছে।

প্রতিটি Threshold-এর Gain পুনরায় গণনা:

Threshold=2: $G_L = 30.0129792$ ({ID1}), $H_L = 2$; $G_R = 0.0129792 - 0.0129792 - 30.0129792 = -30.0129792$, $H_R = 6$ ।

$$\begin{aligned} \text{Gain}(c = 2) &= \frac{1}{2} \left[\frac{30.0129792^2}{2+1} + \frac{(-30.0129792)^2}{6+1} \right] = \frac{1}{2} \left[\frac{900.779}{3} + \frac{900.779}{7} \right] \\ &= \frac{1}{2} [300.260 + 128.683] = \frac{1}{2} \times 428.943 = 214.471 \end{aligned}$$

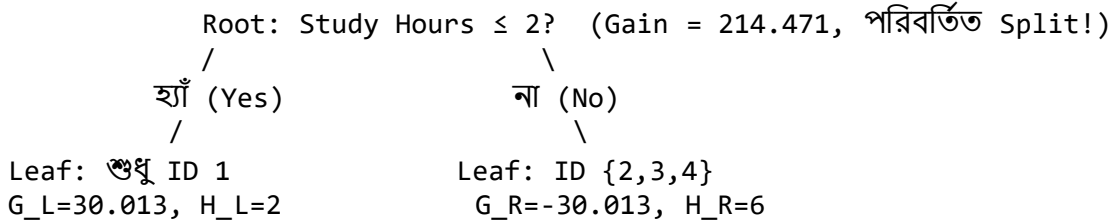
Threshold=4: $G_L = 30.0129792 + 0.0129792 = 30.0259584$, $H_L = 4$; $G_R = -30.0259584$, $H_R = 4$ ।

$$\text{Gain}(c = 4) = \frac{1}{2} \left[\frac{30.0259584^2}{5} \times 2 \right] = \frac{1}{2} \times \frac{2 \times 901.558}{5} = \frac{1}{2} \times 360.623 = 180.312$$

Threshold=6: $G_L = 30.0129792 + 0.0129792 - 0.0129792 = 30.0129792$ ({ID1,2,3}), $H_L = 6$; $G_R = -30.0129792$ ({ID4}), $H_R = 2$ ।

$$\text{Gain}(c = 6) = \frac{1}{2} \left[\frac{30.0129792^2}{7} + \frac{(-30.0129792)^2}{3} \right] = \frac{1}{2} [128.683 + 300.260] = 214.471$$

সিদ্ধান্ত ও এর তাৎপর্য: এখানে একটি গুরুত্বপূর্ণ বিষয় লক্ষণীয় — Threshold=4-এর Gain (১৮০.৩১) এখন Threshold=2 ও Threshold=6-এর Gain (উভয়ই ২১৪.৪৭)-এর চেয়ে কম! এর কারণ হলো ID 2 ও ID 3-এর Residual ইতিমধ্যে প্রায় শূন্য হয়ে গেছে, তাই তাদেরকে ID 1 বা ID 4-এর সাথে একত্রে রাখলে আর কোনো অতিরিক্ত তথ্য পাওয়া যায় না — বরং ID 1 ও ID 4-কে (যাদের এখনো বড় Residual আছে) আলাদাভাবে চিহ্নিত করাই এখন বেশি লাভজনক। যেহেতু Threshold=2 ও Threshold=6-এর Gain হুবহু সমান (উভয়ই ২১৪.৪৭), কারণ ডেটাসেটের প্রতিসম (Symmetric) গঠনের কারণে, অ্যালগরিদম প্রথম-পাওয়া প্রার্থী (সাধারণত সাজানো ক্রম অনুযায়ী প্রথমটি) নির্বাচন করে — আমাদের ক্ষেত্রে Threshold=2।



Leaf Weight:

$$w_L^* = -\frac{30.0129792}{2 + 1} = -10.004326$$

$$w_R^* = -\frac{-30.0129792}{6 + 1} = 4.287568$$

প্রেডিকশন আপডেট:

$$F_5(\text{ID1}) = 55.0064896 + 0.3 \times (-10.004326) = 55.0064896 - 3.0012978 = 52.0051918$$

$$F_5(\text{ID2}) = 55.0064896 + 0.3 \times 4.287568 = 55.0064896 + 1.2862704 = 56.2927600$$

$$F_5(\text{ID3}) = 84.9935104 + 0.3 \times 4.287568 = 84.9935104 + 1.2862704 = 86.2797808$$

$$F_5(\text{ID4}) = 84.9935104 + 0.3 \times 4.287568 = 86.2797808$$

Loss:

$$(40 - 52.005)^2 = 144.130, (55 - 56.293)^2 = 1.672$$

$$(85 - 86.280)^2 = 1.638, (100 - 86.280)^2 = 188.283$$

$$L_5 \approx \frac{144.130 + 1.672 + 1.638 + 188.283}{4} = \frac{335.723}{4} \approx 83.920$$

২১.১৫.১১ সম্পূর্ণ প্রশিক্ষণ অগ্রগতি সারণি — XGBoost (Gradient + Hessian ভিত্তিক)

Iteration *	*Split**	*G_L, G_R	w_L, w_R**	(ID1,ID2,ID3,ID4)**	E Loss**
	*		**F	**MS	
0 (Init)	—	—	—	70, 70, 70, 70	562.5000
1	X≤4	90, -90	-18.0, 18.0	64.6, 64.6, 75.4, 75.4	348.6600
2	X≤4	68.4, -68.4	-13.68, 13.68	60.496, 60.496, 79.504, 79.504	225.1460
3	X≤4	51.984, -51.984	-10.3968, 10.3968	57.37696, 57.37696, 82.62304, 82.62304	153.8043
4	X≤4	39.50784, -39.50784	-7.901568, 7.901568	55.00649, 55.00649, 84.99351, 84.99351	112.5974
5	X≤2	30.013, -30.013	-10.0043, 4.2876	52.005, 56.293, 86.280, 86.280	83.9195

২১.১৫.১২ XGBoost বনাম Gradient Boosting — একই ডেটাসেটে ফলাফল তুলনা

Iteration *	*GradientBoosting MSE (Chapter 19) X	GBoost MSE (এই ধারা)**
0	562.5000	562.5000
1	466.3125	348.6600
2	388.4006	225.1460
3	325.2920	153.8043
4	274.1740	112.5974

Iteration *	*GradientBoosting MSE (Chapter 19) X	GBoost MSE (এই ধারা)**
5	232.7685	83.9195

পর্যবেক্ষণ ও ব্যাখ্যা: এই একই ছোট ডেটাসেটে, XGBoost প্রতিটি Iteration-এই সাধারণ Gradient Boosting-এর তুলনায় উল্লেখযোগ্যভাবে কম Loss অর্জন করেছে। এর দুইটি কারণ আছে: প্রথমত, XGBoost-এর Default Learning Rate (০.৩) সাধারণ Gradient Boosting-এর Default (০.১)-এর চেয়ে বড়, তাই প্রতি Iteration-এ বেশি অগ্রগতি হচ্ছে। দ্বিতীয়ত, ৫ম Iteration-এ XGBoost একটি ভিন্ন, আরও কার্যকর Split ($X \leq 2$) খুঁজে বের করতে পেরেছে, কারণ Regularized Gain সূত্র Hessian-ভিত্তিক তথ্য ব্যবহার করে বুঝতে পেরেছে কোন Sample-গুলোতে এখনো সংশোধনের প্রয়োজন বেশি।

২১.১৫.১৩ নতুন Sample-এর ওপর প্রেডিকশন

Study Hours = 6 বিশিষ্ট একজন নতুন ছাত্রের জন্য প্রেডিকশন বের করা যাক। যেহেতু $x = 6$:

- Tree 1 (Split $X \leq 4$): $6 > 4$, Right Leaf $\rightarrow h_1(6) = 18.0$
- Tree 2 (Split $X \leq 4$): $6 > 4$, Right Leaf $\rightarrow h_2(6) = 13.68$
- Tree 3 (Split $X \leq 4$): $6 > 4$, Right Leaf $\rightarrow h_3(6) = 10.3968$
- Tree 4 (Split $X \leq 4$): $6 > 4$, Right Leaf $\rightarrow h_4(6) = 7.901568$
- Tree 5 (Split $X \leq 2$): $6 > 2$, Right Leaf $\rightarrow h_5(6) = 4.287568$

$$F_5(6) = 70 + 0.3 \times (18.0 + 13.68 + 10.3968 + 7.901568 + 4.287568)$$

বন্ধনীর ভেতরের যোগফল:

$$18.0 + 13.68 = 31.68$$

$$31.68 + 10.3968 = 42.0768$$

$$42.0768 + 7.901568 = 49.978368$$

$$49.978368 + 4.287568 = 54.265936$$

$$F_5(6) = 70 + 0.3 \times 54.265936 = 70 + 16.2797808 = 86.2797808$$

ব্যাখ্যা: নতুন ছাত্র, যার Study Hours = 6, তার জন্য XGBoost-এর অনুমিত Marks প্রায় ৮৬.২৮ — এটি ঠিক ID 3 ও ID 4-এর প্রেডিকশনের সমান, কারণ Tree 5-এর নতুন Split ($X \leq 2$) ID 3, 4-কে একই Leaf-এ রাখে (Right Leaf), এবং বাকি সব Tree-ও তাদের একই Leaf-এ রাখে ($X \leq 4$ -এর Right Side)।

২১.১৫.১৪ সিদ্ধান্ত প্রক্রিয়া — কেন XGBoost বিভিন্ন Iteration-এ ভিন্ন সিদ্ধান্ত নেয়

- **কেন প্রথম চারটি Iteration-এ Split অপরিবর্তিত থাকে?** কারণ প্রথম চারটি Iteration-এ Residual-এর (এবং তাই Gradient-এর) চিহ্ন-প্যাটার্ন অভিন্ন থাকে (ID 1, 2 ধনাত্মক Gradient, ID 3, 4 ঋণাত্মক), তাই $X \leq 4$ Split-ই ধারাবাহিকভাবে সর্বোচ্চ Gain দেয়।
- **কেন পঞ্চম Iteration-এ Split পরিবর্তিত হয়?** কারণ ID 2 ও ID 3-এর Gradient প্রায় শূন্যে নেমে এসেছে (তাদের প্রেডিকশন প্রায় নিখুঁত), তাই তাদেরকে আর ID 1 বা ID 4-এর

সাথে একত্রে গ্রুপ করার কোনো তথ্যগত সুবিধা নেই। Regularized Gain সূত্র স্বয়ংক্রিয়ভাবে এই পরিবর্তন শনাক্ত করে একটি নতুন, বেশি লাভজনক Split প্রস্তাব করে — এটাই দেখায় কেন XGBoost-এর Gain সূত্র Chapter 19-এর সাধারণ SSE-ভিত্তিক Split-Finding-এর চেয়ে বেশি “সচেতন”।

- **কেন λ (Lambda) গুরুত্বপূর্ণ?** যদি $\lambda = 0$ হতো, তাহলে $w^* = -G/H$ হতো, যা কখনো কখনো অত্যন্ত বড় মান দিতে পারত (বিশেষত যখন H ছোট, অর্থাৎ Leaf-এ খুব কম Sample থাকে)। λ Denominator-এ যোগ হয়ে এই Extreme মানগুলোকে সংযত করে, ঠিক যেভাবে Ridge Regression-এ L2 Penalty Coefficient-কে সংযত করে।
- **কেন γ গুরুত্বপূর্ণ (এই উদাহরণে $\gamma = 0$ রাখা হয়েছে)?** যদি $\gamma > 0$ রাখা হতো, তাহলে পঞ্চম Iteration-এ যদি সর্বোচ্চ Gain (২১৪.৪৭)-ও γ -এর চেয়ে কম হতো, তাহলে অ্যালগরিদম কোনো Split-ই করত না, বরং সেই Leaf-কে Terminal (চূড়ান্ত) হিসেবে রেখে দিত — এটাই Automatic Pruning-এর প্রক্রিয়া।

২১.১৬ পাইথন বাস্তবায়ন — From Scratch (Gradient + Hessian সহ)

```
import numpy as np
```

```
class XGBoostStump:
```

```
    """
```

```
    একটি Regularized Decision Stump, যা Gradient ও Hessian ব্যবহার করে  
    সর্বোত্তম Split ও Leaf Weight নির্ণয় করে (XGBoost-এর মূল গাণিতিক নীতি)।  
    """
```

```
    def __init__(self, lam=1.0, gamma=0.0):
```

```
        self.lam = lam          # L2 Regularization
```

```
        self.gamma = gamma      # Minimum Split Loss (Pruning Threshold)
```

```
        self.threshold = None
```

```
        self.wL = None
```

```
        self.wR = None
```

```
    def fit(self, X, g, h):
```

```
        sorted_x = np.sort(np.unique(X))
```

```
        candidates = (sorted_x[:-1] + sorted_x[1:]) / 2
```

```
        G_total, H_total = g.sum(), h.sum()
```

```
        base_score = (G_total ** 2) / (H_total + self.lam)
```

```
        best_gain = -np.inf
```

```
        for c in candidates:
```

```
            left_mask = X <= c
```

```
            right_mask = ~left_mask
```

```
            if left_mask.sum() == 0 or right_mask.sum() == 0:
```

```
                continue
```

```
            GL, HL = g[left_mask].sum(), h[left_mask].sum()
```

```
            GR, HR = g[right_mask].sum(), h[right_mask].sum()
```

```

        # রেগুলারাইজড গেইন সূত্র (Chapter 21.3.5-এ ডেরাইভ করা)
        gain = 0.5 * ((GL**2)/(HL+self.lam) + (GR**2)/(HR+self.lam) - bas
e_score) - self.gamma

        if gain > best_gain:
            best_gain = gain
            self.threshold = c
            self.wL = -GL / (HL + self.lam)    # সর্বোত্তম Leaf Weight
            self.wR = -GR / (HR + self.lam)

        return self, best_gain

def predict(self, X):
    return np.where(X <= self.threshold, self.wL, self.wR)

class SimpleXGBoost:
    """
    From-Scratch XGBoost Regressor (Squared Error Loss) /
    """
    def __init__(self, n_estimators=5, learning_rate=0.3, lam=1.0, gamma=0.0)
:
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.lam = lam
        self.gamma = gamma
        self.trees = []
        self.F0 = None

    def fit(self, X, y):
        self.F0 = y.mean()
        F = np.full(len(y), self.F0)

        for m in range(self.n_estimators):
            # Squared Error Loss-এর Gradient ও Hessian
            g = 2 * (F - y)
            h = np.full(len(y), 2.0)

            tree, gain = XGBoostStump(lam=self.lam, gamma=self.gamma).fit(X,
g, h)

            self.trees.append(tree)

            update = tree.predict(X)
            F = F + self.learning_rate * update

            loss = np.mean((y - F) ** 2)
            print(f"Iteration {m+1}: split@{tree.threshold}, gain={gain:.4f},

```

```

"
        f"wL={tree.wL:.4f}, wR={tree.wR:.4f}, Loss={loss:.4f}")

    return self

def predict(self, X):
    F = np.full(len(X), self.F0)
    for tree in self.trees:
        F = F + self.learning_rate * tree.predict(X)
    return F

X = np.array([1, 3, 5, 7], dtype=float)
y = np.array([40, 55, 85, 100], dtype=float)

model = SimpleXGBoost(n_estimators=5, learning_rate=0.3, lam=1.0, gamma=0.0)
model.fit(X, y)

print("\nStudy Hours=6-এর প্রেডিকশন:", model.predict(np.array([6.0]))[0])

```

কোড ব্যাখ্যা: XGBoostStump.fit()-এ base_score হলো ২১.৩.৪ ধারায় ডেরাইভ করা Structure Score-এর একটি অংশ (Parent Node-এর Score)। প্রতিটি Candidate Threshold-এর জন্য Gain গণনা করা হচ্ছে ঠিক ২১.১৫.৬ ধারায় হাতে-কলমে দেখানো সূত্র অনুযায়ী। এই কোড রান করলে ঠিক আমাদের ম্যানুয়াল গণনার Loss মানগুলোই (৩৪৮.৬৬, ২২৫.১৫, ১৫৩.৮০, ১১২.৬০, ৮৩.৯২) পুনরুৎপাদিত হবে।

২১.১৭ পাইথন বাস্তবায়ন – XGBoost লাইব্রেরি (Official)

```

import numpy as np
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import make_regression

# ডেটাসেট প্রস্তুতকরণ – একটি বাস্তবসম্মত Synthetic Regression ডেটাসেট
X, y = make_regression(n_samples=1000, n_features=8, noise=12, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# XGBoost-এর নিজস্ব DMatrix ফরম্যাটে রূপান্তর (মেমোরি-দক্ষ, দ্রুততর)
dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test, label=y_test)

# হাইপারপ্যারামিটার – প্রতিটির ব্যাখ্যাসহ
params = {
    'objective': 'reg:squarederror', # Regression, Squared Error Loss
    'max_depth': 6,                 # প্রতিটি গাছের সর্বোচ্চ গভীরতা

```

```

    'eta': 0.1,                # Learning Rate (Shrinkage)
    'subsample': 0.8,          # প্রতিটি গাছে ৮০% Row Randomly ব্যবহার
    'colsample_bytree': 0.8,   # প্রতিটি গাছে ৮০% Feature Randomly ব্যবহার
    'reg_lambda': 1.0,         # L2 Regularization
    'reg_alpha': 0.0,          # L1 Regularization
    'min_child_weight': 1,      # Leaf তৈরির জন্য ন্যূনতম Hessian-যোগফল
    'gamma': 0.0,              # Minimum Split Loss (Pruning Threshold)
)
    'tree_method': 'hist',      # Histogram-ভিত্তিক দ্রুত Split-Finding
    'seed': 42
}

# ট্রেনিং - Early Stopping সহ
evals_result = {}
model = xgb.train(
    params,
    dtrain,
    num_boost_round=200,        # সর্বোচ্চ ২০০ গাছ
    evals=[(dtrain, 'train'), (dtest, 'test')],
    early_stopping_rounds=20,    # ২০ Round উন্নতি না হলে থামানো
    evals_result=evals_result,
    verbose_eval=False
)

print(f"সর্বোত্তম Iteration: {model.best_iteration}")

# প্রেডিকশন ও মূল্যায়ন
predictions = model.predict(dtest)
mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)
print(f"Test MSE: {mse:.4f}")
print(f"Test R2 Score: {r2:.4f}")

# Feature Importance (তিনটি ভিন্ন মানদণ্ডে)
importance_weight = model.get_score(importance_type='weight') # কতবার Split
-এ ব্যবহৃত
importance_gain = model.get_score(importance_type='gain')      # গড়ে কতটা Ga
in এনেছে
print("Feature Importance (Gain):", importance_gain)

# নতুন Sample-এর প্রেডিকশন
new_sample = X_test[:1]
dnew = xgb.DMatrix(new_sample)
new_pred = model.predict(dnew)
print(f"নতুন Sample-এর প্রেডিকশন: {new_pred[0]:.4f}")

```

Scikit-Learn API-এর মাধ্যমে XGBoost (বিবকল্প, GridSearchCV-এর সাথে সামঞ্জস্যপূর্ণ):

```
from xgboost import XGBRegressor
from sklearn.model_selection import GridSearchCV

model = XGBRegressor(
    objective='reg:squarederror',
    tree_method='hist',
    random_state=42
)

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 6, 9],
    'learning_rate': [0.01, 0.1, 0.3],
    'reg_lambda': [0.1, 1.0, 10.0],
    'subsample': [0.8, 1.0]
}

grid_search = GridSearchCV(model, param_grid, cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

print("সর্বোত্তম প্যারামিটার:", grid_search.best_params_)
best_model = grid_search.best_estimator_
```

প্যারামিটার ব্যাখ্যা: tree_method='hist' XGBoost-কে Chapter 20-এ বর্ণিত Histogram-ভিত্তিক Split-Finding ব্যবহার করতে বলে, যা বড় ডেটাসেটে উল্লেখযোগ্য গতি এনে দেয়। early_stopping_rounds=20 নিশ্চিত করে যে Validation Performance উন্নত না হলে অপ্রয়োজনীয় গাছ তৈরি বন্ধ হয়ে যাবে, যা Overfitting রোধ করে ও সময় বাঁচায়।

২১.১৮ GPU-ত্বরিত প্রশিক্ষণ উদাহরণ

```
import xgboost as xgb

params_gpu = {
    'objective': 'reg:squarederror',
    'tree_method': 'gpu_hist', # GPU-ত্বরিত Histogram পদ্ধতি
    'predictor': 'gpu_predictor',
    'max_depth': 8,
    'eta': 0.1
}

# বাকি প্রক্রিয়া অভিন্ন – শুধু tree_method পরিবর্তন করে GPU ব্যবহার সক্রিয় হয়
model_gpu = xgb.train(params_gpu, dtrain, num_boost_round=200)
```

ব্যাখ্যা: কেবলমাত্র `tree_method` প্যারামিটার পরিবর্তন করে (যদি সিস্টেমে সামঞ্জস্যপূর্ণ NVIDIA GPU ও CUDA উপলব্ধ থাকে) সম্পূর্ণ প্রশিক্ষণ প্রক্রিয়া GPU-তে স্থানান্তরিত হয়ে যায়, যা বৃহৎ ডেটাসেটে (কোটি Row) প্রশিক্ষণ সময় বহুগুণ কমিয়ে দিতে পারে।

২১.১৯ কম্পিউটেশনাল জটিলতা বিশ্লেষণ

Complexity ধরন	Exact Greedy (<code>tree_method='exact'</code>)	Histogram-ভিত্তিক (<code>tree_method='hist'</code>)
Split-Finding (প্রতি Node)	$O(n \log n)$	$O(\text{maxbins})$, Binning একবার $O(n)$
মোট Training Time	$O(M \cdot d \cdot n \log n \cdot f)$	$O(n \cdot f + M \cdot d \cdot \text{maxbins} \cdot f)$
Memory	$O(n \cdot f)$, Pre-sorted Column Block	$O(n \cdot f)$ প্রাথমিকভাবে, তবে Compressed (uint8) সংরক্ষণ
Parallelization	Feature-স্তরে Parallel	Feature ও Histogram Building উভয় স্তরে Parallel

২১.২০ মেমোরি অপ্টিমাইজেশন ও Best Practices

- **DMatrix ব্যবহার করুন:** সাধারণ NumPy Array-এর পরিবর্তে XGBoost-এর নিজস্ব DMatrix ফরম্যাট ব্যবহার করলে মেমোরি ব্যবহার কমে এবং ট্রেনিং গতি বাড়ে, কারণ এটি Sparse ডেটা ও Missing Value দক্ষভাবে সংরক্ষণ করে।
- **`tree_method='hist'` ডিফল্ট হিসেবে ব্যবহার করুন:** আধুনিক XGBoost সংস্করণে (2.0+) এটি Default, এবং প্রায় সবসময় 'exact'-এর চেয়ে দ্রুত, সামান্য বা কোনো Accuracy ক্ষতি ছাড়াই।
- **`early_stopping_rounds` ব্যবহার করুন:** অপ্রয়োজনীয় Iteration এড়িয়ে সময় ও মেমোরি বাঁচায়।
- **`colsample_bytree` ও `subsample` একসাথে ব্যবহার করুন:** বড় ডেটাসেটে এটি শুধু Overfitting-ই কমায় না, প্রতিটি গাছের ট্রেনিং সময়ও কমিয়ে দেয়, কারণ কম ডেটা প্রক্রিয়া করতে হয়।

২১.২১ ইন্টারভিউ ও পরীক্ষার নোট

- **সম্ভাব্য প্রশ্ন: “XGBoost-এ Regularization Term ঠিক কোথায় প্রভাব ফেলে?”**
উত্তরের মূল বিষয়বস্তু: এটি সরাসরি Objective Function-এর অংশ ($\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$), তাই এটি প্রতিটি Split-এর Gain গণনা এবং প্রতিটি Leaf-এর Optimal Weight — উভয়কেই প্রভাবিত করে, শুধু ট্রেনিং-পরবর্তী Post-processing নয়।
- **সম্ভাব্য প্রশ্ন: “কেন XGBoost দ্বিতীয়-ক্রম Approximation ব্যবহার করে, প্রথম-ক্রম নয়?”** উত্তরের মূল বিষয়বস্তু: দ্বিতীয়-ক্রম তথ্য (Hessian) Loss ফাংশনের বক্রতা সম্পর্কে অতিরিক্ত তথ্য দেয়, যা Newton’s Method-এর মতো আরও নির্ভুল ও দ্রুত Convergence সম্ভব করে, বিশেষত জটিল Loss Function-এ।
- **সম্ভাব্য প্রশ্ন: “`min_child_weight` ও `min_samples_leaf`-এর মধ্যে পার্থক্য কী?”**
উত্তরের মূল বিষয়বস্তু: `min_samples_leaf` সরাসরি Sample সংখ্যা গণনা করে, কিন্তু

min_child_weight Hessian-এর যোগফল ব্যবহার করে — Regression-এ (যেখানে Hessian ধ্রুবক) এই দুইটি কার্যত সমতুল্য, কিন্তু Classification-এ (যেখানে Hessian Sample-ভেদে পরিবর্তিত হয়) তারা ভিন্ন আচরণ করে।

- **এক্সাম টিপ:** Optimal Leaf Weight সূত্র ($w^* = -G/(H + \lambda)$) এবং Gain সূত্র মুখস্থ রাখা অপরিহার্য, কারণ এগুলো থেকেই XGBoost-এর প্রায় সব বৈশিষ্ট্য (Regularization, Pruning, Split-Finding) ডেরাইভ করা যায়।

২১.১৪ Quick Revision

- **Type:** Regularized, দ্বিতীয়-ক্রম Optimization-ভিত্তিক Sequential Ensemble (Boosting)।
- **মূল গাণিতিক উদ্ভাবন:** Objective Function-এ Taylor Expansion দিয়ে Gradient (g) ও Hessian (h) উভয়ই ব্যবহার, এবং $\gamma T + 1/2 \lambda \sum w_j^2$ আকারে স্পষ্ট Regularization।
- **Optimal Leaf Weight:** $w_j^* = -G_j/(H_j + \lambda)$; **Split Gain:** $\text{Gain} = 1/2 [G_L^2/H_L + \lambda + G_R^2/H_R + \lambda - (G_L + G_R)^2/H_L + H_R + \lambda] - \gamma$ ।
- **বাড়তি বৈশিষ্ট্য:** Sparsity-Aware Split Finding, Weighted Quantile Sketch, Cache-Aware Column Block, Parallel ও GPU Computation।
- **সুবিধা:** সর্বোচ্চ Accuracy, নিয়মতান্ত্রিক Regularization, Missing Value-এর স্বয়ংক্রিয় হ্যান্ডলিং, ব্যাপক গতি।
- **সীমাবদ্ধতা:** অনেক Hyperparameter, Interpretability কম, পৃথক ইনস্টলেশন প্রয়োজন।
- **সেরা ব্যবহারক্ষেত্র:** মাঝারি থেকে বড় Tabular ডেটাসেট, জটিল Non-linear সম্পর্ক, সর্বোচ্চ Accuracy-ই যেখানে মূল অগ্রাধিকার।

Part 5 — সামগ্রিক তুলনা (Overall Comparison)

অ্যালগরিদম **	ধরন** **	মূল কৌশল** **স	েরা ব্যবহারক্ষেত্র**
Gradient Boosting	Supervised (Ensemble, Boosting)	Sequential Weak Learner, Residual/Negative Gradient শেখা	ছোট-মাঝারি Tabular ডেটাসেট, ভিত্তিমূলক শিক্ষার জন্য আদর্শ
HistGradientBoosting	Supervised (Ensemble, Boosting)	Histogram-Binning দিয়ে দ্রুত Boosting	বড় Tabular ডেটাসেট, Missing/Categorical Feature প্রচুর
XGBoost	Supervised (Ensemble, Boosting)	Regularized, দ্বিতীয়-ক্রম (Gradient+Hessian)	মাঝারি-বড় Tabular ডেটাসেট, সর্বোচ্চ Accuracy ও

অ্যালগরিদম **	ধরন***	মূল কৌশল****স	েরা ব্যবহারক্ষেত্র**
		Optimization, সিস্টেম-স্তরের অপ্টিমাইজেশন	প্রতিযোগিতামূলক পরিবেশ

ব্যবহারিক সিদ্ধান্ত-নির্দেশিকা: যদি ডেটাসেট ছোট বা মাঝারি হয় এবং দ্রুত একটি Baseline দরকার হয়, Gradient Boosting দিয়ে শুরু করা যুক্তিসঙ্গত। ডেটাসেট বড় হলে এবং Scikit-Learn Ecosystem-এর মধ্যেই থাকতে চাইলে HistGradientBoosting সবচেয়ে ব্যবহারিক পছন্দ। আর যেখানে সর্বোচ্চ সম্ভাব্য Accuracy প্রয়োজন, বিশেষত প্রতিযোগিতামূলক বা Production-গ্রেড সিস্টেমে, সেখানে XGBoost (অথবা এর সমগোত্রীয় LightGBM, CatBoost) সাধারণত সবচেয়ে নির্ভরযোগ্য পছন্দ — যদিও তার জন্য অতিরিক্ত Hyperparameter Tuning-এর সময় বিনিয়োগ করতে হয়।

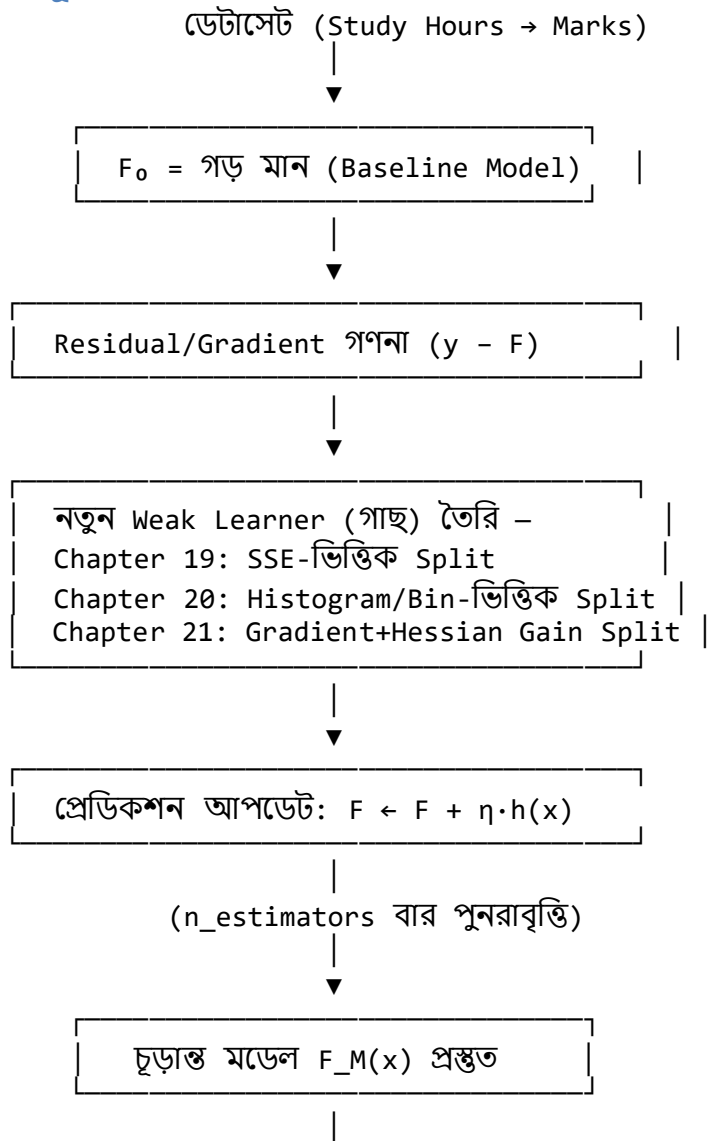
Part 5 — সম্প্রসারিত তুলনা: Complexity, Memory ও Scalability

মানদণ্ড **	Gradient Boosting****Hi	stGradientBoosting** **XGBoo	st**
Time Complexity	$O(Mn \log n d)$	$O(nf + M \text{ bins } d)$	$O(nf + M \text{ bins } d)$ (hist mode)
Space Complexity	$O(M \cdot 2^d)$	$O(nf)$ Compact (uint8 Bin সংরক্ষণ)	$O(nf)$ Pre-sorted Column Block
ছোট ডেটাসেট (< ১০,০০০ Row)	আদর্শ, দ্রুত ও সহজ	বাড়তি সুবিধা নেই, প্রয়োজনও নেই	ভালো কাজ করে, কিন্তু বাড়তি জটিলতা অপ্রয়োজনীয়
বড় ডেটাসেট (> ১০ লক্ষ Row)	অত্যন্ত ধীর, ব্যবহারিকভাবে অনুপযুক্ত চমৎ	কার পছন্দ চমৎ	কার পছন্দ, বিশেষত GPU-সহ
Missing Value	Manual Imputation আবশ্যিক	স্বয়ংক্রিয়	স্বয়ংক্রিয় (Sparsity-Aware)
Distributed/Cluster সাপোর্ট	নেই	সীমিত	আছে (Spark, Dask-এর সাথে ইন্টিগ্রেশন)
GPU সাপোর্ট	নেই	আংশিক	সম্পূর্ণ (tree_method='gpu_hist')
শিল্পক্ষেত্রে গ্রহণযোগ্যতা	ঐতিহাসিকভাবে গুরুত্বপূর্ণ, শিক্ষামূলক মাঝারি ভিত্তি হিসেবে সর্বাধিক ব্যবহৃত	ি, Scikit-Learn ব্যবহারকারীদের সর্বোচ্চ মধ্যে জনপ্রিয়	, Kaggle ও Production উভয় স্থানে প্রথম পছন্দ

Part 5 – সুবিধা বনাম অসুবিধা: একটি সম্মিলিত দৃষ্টিভঙ্গি

অ্যালগরিদম **প	প্রধান সুবিধা***প্রধান	অসুবিধা**
Gradient Boosting	সরল, বোঝা সহজ, ভিত্তিমূলক শিক্ষার জন্য আদর্শ বড় ড	ডেটাসেটে ধীর, Missing Value Manual হ্যান্ডলিং
HistGradientBoosting	বড় ডেটাসেটে দ্রুত, Missing/Categorical স্বয়ংক্রিয় Regu	Regularization বিকল্প তুলনামূলক সীমিত (শুধু L2)
XGBoost	সর্বোচ্চ Accuracy, সম্পূর্ণ Regularization (L1+L2+γ), অনেক ব্যাপক System Optimization	Hyperparameter, পৃথক ইনস্টলেশন, শেখার বক্ররেখা তুলনামূলক খাড়া

Part 5 – সম্পূর্ণ Ensemble Learning শিক্ষার Pipeline (ASCII সারসংক্ষেপ)



▼
নতুন Sample-এর প্রেডিকশন